



Master Thesis

Adaptive and Fault-Tolerant Leader Election in Resource-Constrained IoT and Robotic Clusters

Pushpit Bhardwaj

Born on: 22.04.2002 in Jharmajri, India

Course: Distributed Systems Engineering

Matriculation number: 5194985

Matriculation year: 2023

to achieve the academic degree

Master of Science (MSc)

Referee

Dr. Ing. Thomas Springer

Supervisor

Leonard Herbst M.Sc.

Supervising professor

Prof. Dr. Matthias Wählisch

Submitted on: 16.03.2026



Task for the preparation of a Master Thesis

Course:	Distributed Systems Engineering
Name:	Pushpit Bhardwaj
Matriculation number:	5194985
Matriculation year:	2023
Title:	Adaptive and Fault-Tolerant Leader Election in Resource-Constrained IoT and Robotic Clusters

Objectives of work

The objective of this work is to design, implement, and evaluate an adaptive and fault-tolerant leader election protocol for clusters of resource-constrained IoT devices and robotic nodes. Building on the PraSLE (Practical Self-Stabilizing Leader Election) protocol, the thesis aims to enhance leader election with context-aware decision-making while preserving lightweight operation and self-stabilization guarantees. The overarching goal is to realize a practically deployable solution that remains robust under unreliable communication, limited energy budgets, and dynamic network conditions, and that can be evaluated using realistic IoT simulation environments without relying on heavyweight assumptions or centralized coordination.

Focus of work

- Implement the baseline PraSLE leader election protocol in Contiki-NG and evaluate its behavior in static and lossy IoT network scenarios using the Cooja simulator, establishing reference metrics for convergence time, communication overhead, energy consumption, and fault recovery.
- Extend the PraSLE state machine with energy-aware and link-quality-aware metrics, enabling context-sensitive leader selection that reflects the practical constraints of IoT and robotic clusters.
- Design and implement a controlled leader rotation mechanism that proactively hands over leadership when energy thresholds or sustainability criteria are violated, preventing premature depletion of individual nodes.
- Introduce adaptive timeout mechanisms for failure detection that dynamically adjust to varying link qualities and network conditions, improving responsiveness while avoiding unnecessary communication overhead.
- Implement a backup-based fault recovery strategy that maintains a ranked list of successor candidates, enabling rapid and stable leader replacement in the presence of node or link failures.



- Rigorously evaluate the proposed extensions through systematic simulation studies in Cooja, comparing the extended protocol against baseline PraSLE and classical leader election algorithms such as Bully and Ring with respect to robustness, fairness, energy efficiency, and overall network lifetime.

Referee:	Dr. Ing. Thomas Springer
Supervisor:	Leonard Herbst M.Sc.
Issued on:	13.10.2025
Due date for submission:	16.03.2026

Prof. Dr. Matthias Wählisch
Supervising professor

Statement of authorship

I hereby certify that I have authored this document entitled *Adaptive and Fault-Tolerant Leader Election in Resource-Constrained IoT and Robotic Clusters* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. There were no additional persons involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 16.03.2026

A handwritten signature in black ink, appearing to read 'Pushpit Bhardwaj', with a stylized flourish at the end.

Pushpit Bhardwaj

Abstract

This thesis investigates the design, implementation, and evaluation of an adaptive and fault-tolerant leader election protocol for clusters of resource-constrained IoT devices and robotic nodes. Leader election is a fundamental coordination primitive in distributed systems, yet many classical solutions assume reliable communication and sufficient computational resources, rendering them unsuitable for energy-limited and unreliable environments. Building on PraSLE, a practical self-stabilizing leader election protocol designed for IoT networks, this work introduces an extended variant, referred to as Adaptive-PraSLE, which incorporates context-aware and sustainability-oriented mechanisms while preserving lightweight operation and self-stabilization guarantees.

The proposed approach augments the base protocol with energy-aware and link-quality-aware leader selection, controlled leader rotation to avoid premature node depletion, adaptive timeout mechanisms for efficient fault detection, and backup-based recovery to ensure rapid stabilization after failures. These extensions are designed to improve reliability, fairness, and long-term network operation without adding excessive communication or computational overhead.

The protocol is implemented in Contiki-NG and evaluated using the Cooja simulator, which provides realistic models for constrained hardware platforms, wireless communication, and energy consumption. An experimental study compares Bully, Ring, PraSLE, and Adaptive-PraSLE across a range of network conditions and fault scenarios. The evaluation focuses on convergence time, communication overhead, energy efficiency, leader stability, fairness of leader rotation, and overall network lifetime. The results show that Adaptive-PraSLE outperforms classical leader election algorithms and improves on PraSLE in resilience and sustainability, supporting the case for adaptive, context-aware leader election in IoT and robotic clusters.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Leonard Herbst M.Sc. for his continuous support, guidance, and constructive feedback throughout this thesis. His bi-weekly meetings and careful suggestions were essential in shaping both the direction and the quality of this work.

I am also grateful to Dr. Ing. Thomas Springer for providing the research topic and offering valuable input during the implementation phase. His insights and expertise contributed significantly to the development of the system presented in this thesis.

Finally, I would like to thank my friends and family for their encouragement and support during the course of this work.

Contents

List of Figures	x
List of Tables	xi
Acronyms	xiii
1 Introduction	1
1.1 Motivation	1
1.2 Objective	2
1.3 Problem Statement	2
1.4 Structure of the Work	3
2 Background	4
2.1 Distributed Systems and Leader Election	4
2.2 Self-Stabilization	6
2.3 Internet of Things: Characteristics and Constraints	9
2.4 Energy Considerations in IoT Networks	11
2.5 Link Quality in Wireless Networks	13
2.6 Contiki-NG and the Cooja Simulator	15
2.7 Summary	18
3 Related Work	19
3.1 Classical Leader Election Algorithms	19
3.2 Consensus-Based Approaches	20
3.3 Self-Stabilizing Leader Election	21
3.4 Energy-Aware Clustering	23
3.5 Link-Quality-Aware Protocols	26
3.6 Adaptive Mechanisms in Distributed Systems	27

3.7	Leader Election in Mobile and Robotic Networks	27
3.8	Research Gaps and Positioning	28
4	Concept and Solution Approach	31
4.1	Extension Philosophy	31
4.2	Justification of Design Choices	32
4.3	The Five Extensions	34
4.4	Adaptive Timeout Mechanism	37
4.5	Protocol and Application Concerns	38
4.6	Self-Stabilization Preservation	39
4.7	Platform Choice: Contiki-NG and Cooja	40
4.8	Evaluation Methodology: Two Parameter Cases	40
4.9	Summary	42
5	Implementation	43
5.1	System Architecture	43
5.2	Algorithm Design	44
5.3	Composite Scoring Function	48
5.4	Adaptive Mechanisms	50
5.5	State Machine	52
5.6	Message Formats	52
5.7	Communication Design	54
5.8	Integration with Contiki-NG	54
5.9	Configuration and Feature Toggles	55
5.10	Summary	56
6	Evaluation	57
6.1	Experimental Methodology	57
6.2	Convergence Time Results	59
6.3	Message Overhead Results	62
6.4	Fault Tolerance Results	63
6.5	Noise Resilience Results	65
6.6	Network Partition Results	68
6.7	Trade-off Analysis	69
6.8	Algorithm Scalability	71
7	Conclusion and Further Work	72
7.1	Summary	72

7.2	Key Findings	72
7.3	Threats to Validity	73
7.4	Practical Implications	74
7.5	Further Work	75
	Appendix	81
A	Simulation Environment	81
B	Complete Configuration Reference	82
C	Experiment Configuration	82

List of Figures

5.1	Adaptive-PraSLE node architecture showing the four-layer design: monitoring collects context information, scoring computes composite rankings, election manages the PraSLE algorithm with extensions, and communication handles message exchange.	44
5.2	Structure of Adaptive-PraSLE election rounds and cycles. Each round consists of a receive phase (duration T) and an update phase. An election cycle comprises K rounds. In continuous operation mode, every third cycle triggers a reset to clear potentially stale leader values.	46
5.3	Composite scoring function components. Each component contributes a weighted portion to the final score. Lower scores indicate better leader candidates.	48
5.4	Controlled leader handover protocol. The leader initiates handover when energy falls below a critical threshold. A request-acknowledgment exchange coordinates the transfer, though a brief overlap period exists during which both the old and new leader may consider themselves leader until the acknowledgment is received and propagated.	51
5.5	Adaptive-PraSLE state machine. Nodes transition between states based on election progress, convergence, leader failure detection, and controlled handover.	52
6.1	Convergence time comparison across network sizes. Lower values are better. PraSLE and Adaptive-PraSLE variants consistently outperform classical algorithms, with the performance gap widening at larger network sizes.	59
6.2	Message overhead comparison. Lower values indicate more energy-efficient algorithms. Standard PraSLE generates the highest overhead due to unconditional broadcasting, while Adaptive-PraSLE reduces message count through conditional broadcasting.	62
6.3	Recovery time after leader crash. Lower values indicate faster fault recovery. PraSLE variants show consistent recovery times across network sizes, while Ring shows degraded performance at larger scales.	64

6.4	Convergence under 10% packet loss (90% success rate). Most algorithms handle mild interference with minimal degradation.	66
6.5	Convergence under 30% packet loss (70% success rate). Ring begins showing significant degradation at larger network sizes.	66
6.6	Convergence under 50% packet loss (50% success rate). High packet loss severely affects the Ring algorithm while PraSLE variants remain stable.	67
6.7	Network partition convergence (both partitions). Each partition should independently elect a leader. All algorithms correctly handle partitioning, with PraSLE variants showing the most consistent performance. .	68
6.8	Message overhead vs. convergence time trade-off. Each point represents one algorithm at one network size (labeled). The ideal region is bottom-left (fast convergence, few messages). Adaptive-PraSLE variants occupy favorable positions.	70
6.9	Scalability comparison showing convergence time vs. network size. Ring shows poor scalability with high variance at large sizes. PraSLE and Adaptive-PraSLE maintain consistent performance across scales. .	71

List of Tables

2.1	Comparison of classical leader election algorithms	6
2.2	Recommended K values for PraSLE by topology	9
2.3	Power consumption of the CC2538 SoC at 3.3 V [18]	12
3.1	Comparison of classical leader election algorithms	20
3.2	Comparison of leader election approaches	30
3.3	Message overhead comparison of self-stabilizing leader election	30
5.1	Topology-aware K parameter selection	48
5.2	Adaptive-PraSLE message types	53
5.3	Key configuration parameters	56
A.1	UDGM transmission range by topology (10-node configuration)	82
B.2	Complete Adaptive-PraSLE configuration constants	83
C.3	Simulation duration by experiment type, topology, and network size (seconds). Each topology row applies to all algorithm variants using that topology (PraSLE, Adaptive-PraSLE). Line and ring topologies require longer durations at large network sizes due to slower message propagation. The classical Ring algorithm uses the same durations as line topologies.	84
C.4	Noise experiment success ratio configuration	85

Acronyms

6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks
API	Application Programming Interface
CPU	Central Processing Unit
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
ETX	Expected Transmission Count
EWMA	Exponentially Weighted Moving Average
FIFO	First In, First Out
GHS	Gallager-Humblet-Spira
GUI	Graphical User Interface
HEED	Hybrid Energy-Efficient Distributed Clustering
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IoT	Internet of Things
IPv6	Internet Protocol version 6
ISM	Industrial, Scientific and Medical
LE	Leader Election
LEACH	Low-Energy Adaptive Clustering Hierarchy
LPM	Low-Power Mode
LQI	Link Quality Indicator
MAC	Medium Access Control
OS	Operating System
PBFT	Practical Byzantine Fault Tolerance
PraSLE	Practical Self-Stabilizing Leader Election
RAM	Random Access Memory
RFC	Request for Comments
RPL	Routing Protocol for Low-Power and Lossy Networks
RSSI	Received Signal Strength Indicator
RTO	Retransmission Timeout
RTT	Round-Trip Time
RTTVAR	Round-Trip Time Variation
SRTT	Smoothed Round-Trip Time
UDP	User Datagram Protocol
WSN	Wireless Sensor Network

1 Introduction

1.1 Motivation

Coordination in distributed systems often requires a designated leader, a single node responsible for making decisions, aggregating data, or directing group behavior. From database replication to robotic swarms, leader election underpins countless applications. Classical protocols for this problem, such as the Bully algorithm [1] and the Chang-Roberts ring algorithm [2], assume reliable communication channels and stable network membership. But IoT and robotic networks operate over lossy wireless links without synchronized clocks, and must conserve energy rather than spend it on repeated election rounds. These conditions violate the assumptions on which classical protocols rely.

IoT devices run on batteries that must last months or years. They have kilobytes of RAM, not gigabytes. Their wireless links drop packets, fade unpredictably, and suffer from cross-technology interference. Nodes lose contact during temporary obstructions and rejoin without warning. No global clock synchronizes them; no administrator restarts them when something goes wrong. A leader election protocol for these environments cannot simply be correct; it must be lightweight enough to run on a microcontroller, adaptive enough to handle variable network conditions, and able to recover from arbitrary state corruption.

The textbook algorithms (Bully [1], Ring [2], and their variants) prioritize simplicity over robustness, assuming reliable communication and stable process groups rather than the lossy, dynamic conditions typical of IoT networks. When these assumptions are violated, the algorithms may elect multiple leaders or trigger repeated elections. Consensus protocols like Paxos [3] and Raft [4] provide fault-tolerant agreement and have been adapted for IoT settings, but they implement full distributed state machines rather than just leader election. Their multi-round message exchanges and persistent state requirements introduce overhead that is unnecessary when the only goal is to elect a single leader. What IoT networks need for this narrower task is something in between: robust enough to handle lossy wireless links, yet lightweight enough to avoid wasting scarce resources.

PraSLE (Practical Self-Stabilizing Leader Election) [5] represents a promising direction. It recovers automatically from arbitrary transient faults, a property called self-stabilization [6], and runs with minimal overhead. But PraSLE picks leaders based solely on node identifiers, ignoring whether a node has enough battery to serve as leader or whether its radio links are reliable. It uses fixed timeouts that may be too aggressive for congested networks or too conservative for quiet ones. And once a node becomes leader, it stays leader until it fails, potentially draining its battery while neighbors sit idle. These limitations suggest room for improvement.

1.2 Objective

This thesis aims to design, implement, and evaluate Adaptive-PraSLE, an extension of PraSLE [5] that incorporates context-aware leader selection while preserving self-stabilization guarantees. The specific objectives are:

- **Energy-aware leader selection:** Incorporate remaining energy into the leader ranking so that nodes with depleted batteries do not become leaders.
- **Link-quality-aware leader selection:** Factor in link reliability so that the elected leader can communicate reliably with its neighbors.
- **Adaptive timeouts:** Replace static timeout parameters with timeouts that adjust to observed network conditions, reducing both false failure suspicions and detection delays.
- **Controlled leader rotation:** Allow leaders to hand off their role when their energy runs low, rather than serving until failure.
- **Backup leader tracking:** Maintain backup candidates so that recovery after leader failure does not require a full re-election cycle.

Adaptive-PraSLE is implemented in the Contiki-NG operating system [7] and evaluated using the Cooja simulator. The evaluation compares it against Bully, Ring, and the original PraSLE across multiple topologies and failure scenarios, measuring convergence time, message overhead, energy consumption, leader stability, and network lifetime.

1.3 Problem Statement

Existing leader election protocols do not fully meet the needs of constrained IoT networks. Classical algorithms such as Bully and Ring lack mechanisms to distinguish between failed nodes and temporary communication disruptions, which can trigger unnecessary re-elections [1, 2]. Consensus protocols such as Paxos and Raft serve a broader purpose than leader election, implementing full distributed state machines [3, 4], and their overhead is disproportionate for the narrower task of electing a single leader. PraSLE [5], designed specifically for resource-constrained IoT networks, addresses self-stabilization but leaves the following problems unsolved:

- **Context-blind leader selection.** PraSLE elects the node with the lowest identifier [5], without considering whether that node has sufficient remaining energy

or reliable links to its neighbors. A node with a nearly depleted battery may become leader and fail shortly after election, triggering a complete re-election cycle.

- **Fixed timing.** PraSLE uses a static timeout parameter that does not adapt to changing network conditions [5]: if the timeout is too short, nodes may falsely suspect leader failure; if it is too long, actual failures go undetected for extended periods.
- **Permanent leadership.** Once elected, a PraSLE leader serves until it fails [5], concentrating energy consumption on a single node while other nodes conserve their batteries. Over time, this asymmetry increases the probability of leader failure.

This thesis addresses the following research questions:

- RQ1** What impact does extending self-stabilizing leader election [5, 6] with energy awareness, link-quality awareness, and adaptive timing have on convergence behavior, leader stability, and network lifetime in resource-constrained IoT networks?
- RQ2** How does Adaptive-PraSLE perform relative to classical (Bully, Ring) and self-stabilizing (PraSLE) baselines across varying network sizes, topologies, and packet-loss conditions?
- RQ3** What resource overhead do the extensions introduce in per-node state and message size, and how does this scale relative to the performance gains?

1.4 Structure of the Work

Chapter 2 covers the background: distributed systems fundamentals, classical leader election algorithms, the self-stabilization paradigm, IoT constraints, and the Contiki-NG platform. Chapter 3 surveys related work in depth, from consensus protocols to energy-aware clustering, and identifies the gaps that motivate this thesis. Chapter 4 presents the concept and solution approach, explaining the five extensions and how self-stabilization is preserved. Chapter 5 covers the implementation, including data structures, message formats, and Contiki-NG integration. Chapter 6 evaluates the protocol experimentally, comparing it against Bully, Ring, and PraSLE across topologies and failure scenarios. Chapter 7 concludes with a summary of contributions and directions for future work.

2 Background

This chapter introduces the foundational concepts that underpin the rest of the thesis: distributed systems and leader election, self-stabilization as a fault-tolerance paradigm, the resource constraints specific to IoT devices, and the Contiki-NG platform used for implementation and evaluation.

2.1 Distributed Systems and Leader Election

2.1.1 Distributed System Model

A distributed system is a collection of processes that communicate by passing messages, with no shared memory and no global clock [8]. IoT networks operate under partial synchrony: messages are usually delivered within a bounded time, but this bound may be unknown or may vary due to congestion, interference, or node failures. Processes therefore cannot reliably distinguish between a slow process and a failed one [9]. Fischer et al. demonstrated that in a purely asynchronous system, no deterministic algorithm can guarantee consensus if even a single process may fail [10], motivating the use of timeouts and failure detectors in practical systems.

2.1.2 The Leader Election Problem

Leader election is a standard coordination problem in distributed systems. The goal is for a set of processes to collectively agree on a single process, called the leader or coordinator, that assumes a distinguished role. The leader may be responsible for making decisions, coordinating actions, aggregating data, or managing shared resources on behalf of the group [1].

Formally, a leader election algorithm must satisfy the following properties [9]:

- **Safety (Agreement):** At any point in time, at most one process considers itself the leader.
- **Liveness (Termination):** Eventually, exactly one process becomes the leader and all other processes acknowledge this leader.

- **Validity:** Only a participating process can be elected as the leader.

Leader election is closely related to other distributed computing problems such as consensus and mutual exclusion. In fact, leader election can be used to solve consensus, since a leader can propose a value that all processes then adopt. Conversely, consensus can be used to implement leader election by having processes agree on the identity of the leader [9].

Leader election arises in many distributed settings. Database replicas elect a primary to sequence writes [4], sensor networks elect aggregators to compress readings [11], and robot swarms elect coordinators to direct movement [12]. The pattern recurs because many problems become simpler once a single process assumes a coordinating role, even if that process might fail and require replacement [8].

2.1.3 Classical Leader Election Algorithms

Several classical algorithms have been developed for leader election in distributed systems. Two of the most influential are the Bully algorithm [1] and the Ring algorithm [2], both of which assume that each process has a unique identifier and that the process with the highest (or lowest) identifier should become the leader.

The Bully Algorithm

The Bully algorithm [1] assumes reliable crash detection and synchronous communication. When a process suspects leader failure, it sends ELECTION messages to all higher-identifier processes. If none responds, it declares itself leader; otherwise, it defers to the highest responder. The algorithm has $O(n^2)$ worst-case and $O(n)$ best-case message complexity, with $O(d)$ time complexity where d is the maximum message delay [1].

The Ring Algorithm

The Chang-Roberts Ring algorithm [2] arranges processes in a logical ring where each node forwards election messages to its successor, replacing the candidate identifier if its own is higher. The message circulates until it returns to the initiator, who becomes leader. The algorithm has $O(n^2)$ worst-case message complexity and $O(n \cdot d)$ time complexity. It requires a pre-established ring topology and is vulnerable to ring breaks caused by node failures.

Comparison of Classical Approaches

Table 2.1 summarizes the Bully and Ring algorithms.

Both algorithms assume reliable failure detection and relatively stable network conditions. Neither is inherently self-stabilizing, meaning they cannot recover from an arbitrary corrupted state without external intervention. This limitation is particularly

Table 2.1: Comparison of classical leader election algorithms

Property	Bully	Ring
Message complexity (worst case)	$O(n^2)$	$O(n^2)$
Message complexity (best case)	$O(n)$	$O(n)$
Time complexity	$O(d)$	$O(n \cdot d)$
Topology requirement	None	Ring
Communication pattern	Broadcast	Point-to-point

problematic in IoT environments, where transient faults due to interference, memory corruption, or power fluctuations are common.

2.2 Self-Stabilization

2.2.1 The Self-Stabilization Paradigm

Dijkstra introduced self-stabilization in 1974: design systems that reach correct behavior from *any* starting state [6]. No matter how badly corrupted the initial configuration (whether from power glitches, cosmic rays, or electromagnetic interference), a self-stabilizing system eventually recovers without anyone pressing a reset button.

Formally, a self-stabilizing algorithm satisfies two properties [13]:

- **Convergence:** Starting from any arbitrary state, the system eventually reaches a legitimate state.
- **Closure:** Once the system reaches a legitimate state, it remains in legitimate states as long as no further faults occur.

Self-stabilization guarantees recovery from any reachable state, regardless of how that state arose. Whether a sensor node reboots after a brown-out, a robot's memory is corrupted by radiation, or a gateway loses track of its neighbors after interference, the system converges back to correct behavior without external intervention. This generality distinguishes self-stabilization from traditional fault-tolerance techniques, which target specific failure models such as crash failures, Byzantine behavior, or particular message loss patterns.

This property is particularly valuable for systems that operate unattended in environments where physical maintenance is impractical or impossible. Self-stabilization provides the theoretical foundation for protocols that recover autonomously, a requirement for long-lived IoT deployments.

2.2.2 Self-Stabilizing Leader Election

Applying self-stabilization to leader election means the system must recover from states where multiple processes believe they are the leader, no process considers itself the leader, processes have inconsistent views, or state variables have been

corrupted [13]. Classical algorithms like Bully and Ring cannot handle these cases because they assume a clean initial state.

Self-stabilizing leader election algorithms address this by having each process maintain a local estimate of the leader's identity and periodically exchange this information with neighbors. Through repeated communication, the system converges to agreement. Their correctness is measured by stabilization time: the maximum time to reach a legitimate state from any arbitrary initial configuration, typically expressed in terms of the network diameter D , the number of processes n , and the algorithm's timing parameters. Section 3.3 surveys specific algorithms in detail.

2.2.3 PraSLE: Practical Self-Stabilizing Leader Election

Conard *et al.* [5] proposed PraSLE (Practical Self-Stabilizing Leader Election), a self-stabilizing leader election algorithm designed specifically for constrained IoT devices on lossy links. PraSLE forms the foundation for this thesis; understanding its mechanics is necessary to follow the extensions introduced in Chapter 4.

Algorithm Overview

PraSLE is a round-based algorithm where each node maintains a pair $(\min_i, \text{leader}_i)$. The ranking value $\min_i$ represents the best (lowest) ranking the node has seen so far, and leader_i stores the identifier of the node that produced that ranking. Initially, each node sets its ranking value to its own ranking (typically derived from its node identifier) and considers itself the leader. In each round, nodes broadcast their pair to neighbors. When a node receives a pair with a lower ranking value, it adopts that pair, effectively forwarding the candidacy of a better-ranked node. After K rounds, the pair with the globally lowest ranking value has propagated through the entire network, and all nodes agree on the same leader.

The algorithm takes two parameters:

- T : The maximum network latency, which determines the duration of each round.
- K : A diameter-proportional value that determines the number of rounds needed for convergence.

Algorithm Description

Algorithm 1 shows the complete PraSLE procedure for a single node p_i , adapted from [5]. All pair comparisons use lexicographic ordering: $(\min_a, \text{leader}_a) < (\min_b, \text{leader}_b)$ holds if $\min_a < \min_b$, or $\min_a = \min_b$ and $\text{leader}_a < \text{leader}_b$. This ensures that the node with the lowest ranking value becomes the leader, with ties broken by node identifier.

The first round (when $\text{round} = K + 1$) skips the receive phase because the while-loop condition $\text{round} < K + 1$ is false, allowing nodes to bootstrap by immediately broadcasting their initial values.

Algorithm 1 Practical Self-Stabilizing Leader Election (process p_i)

Initialization:

```

1: round  $\leftarrow K + 1$  ▷  $K$  can be the network diameter
2: neighbors $i$   $\leftarrow$  getListOfNeighbors()
3: min $i$   $\leftarrow N + 1$  ▷  $N$  is the max number of processes
4: temp_min $i$   $\leftarrow$  getRankingValue()
5: leader $i$   $\leftarrow$  ID $i$ 
6: temp_leader $i$   $\leftarrow$  ID $i$ 
7:  $T \leftarrow 1.0$  ▷ Tunable round duration

Main loop:
8: loop ▷ Runs indefinitely
9:   recvTimer  $\leftarrow T$ 
10:  while recvTimer > 0 and round <  $K + 1$  do ▷ Receive phase
11:    upon recv (min $j$ , leader $j$ ) from  $p_j \in$  neighbors $i$  do
12:      if (min $j$ , leader $j$ ) < (temp_min $i$ , temp_leader $i$ ) then
13:        temp_min $i$   $\leftarrow$  min $j$ 
14:        temp_leader $i$   $\leftarrow$  leader $j$ 
15:      end if
16:    end upon
17:  end while ▷ Update phase

18:  round  $\leftarrow$  round - 1
19:  if (temp_min $i$ , temp_leader $i$ ) < (min $i$ , leader $i$ ) then
20:    min $i$   $\leftarrow$  temp_min $i$ 
21:    leader $i$   $\leftarrow$  temp_leader $i$ 
22:    for all  $p_j \in$  neighbors $i$  do
23:      send( $p_j$ , min $i$ , leader $i$ )
24:    end for
25:  else if round  $\leq 0$  then
26:    return leader $i$ 
27:  end if
28: end loop

```

Unreliable Network Mode

For networks with message loss, PraSLE provides an unreliable mode with two modifications to Algorithm 1: the send loop (lines 22–24) is moved outside the if-condition on line 19, so that nodes broadcast their values every round regardless of whether values changed; and the termination check (line 25–26) is removed, so the algorithm runs continuously. This continuous operation provides self-stabilization: election cycles repeat indefinitely, and if any node’s state becomes corrupted, subsequent rounds propagate correct values and restore consistency.

Parameter Selection

The parameter K must be at least equal to the network diameter to guarantee convergence. Table 2.2 shows recommended values for different topologies.

Table 2.2: Recommended K values for PraSLE by topology

Topology	Diameter	Recommended K
Clique (fully connected)	1	$K \geq 2$
Ring	$\lfloor n/2 \rfloor$	$K \geq \lceil (n+1)/2 \rceil$
Line	$n-1$	$K \geq n$
Mesh (square grid)	$2(\sqrt{n}-1)$	$K \geq 2\sqrt{n}$

The parameter T should exceed the maximum expected message delay. The total convergence time is $O(K \cdot T)$.

Complexity Analysis

PraSLE's complexity for an arbitrary connected topology with n nodes, maximum degree Δ , and diameter-proportional parameter K is:

- **Message complexity:** $O(K \cdot n \cdot \Delta)$ per election cycle. In unreliable mode, this overhead repeats every cycle.
- **Time complexity:** $O(K \cdot T)$, linear in the number of rounds.
- **Space complexity:** $O(1)$ per node, plus neighbor list storage.

2.3 Internet of Things: Characteristics and Constraints

2.3.1 Resource-Constrained IoT Devices

The Internet of Things encompasses a wide range of networked devices, from powerful single-board computers to simple microcontroller-based sensors. This thesis focuses on the constrained end of this spectrum. RFC 7228 [14] defines three classes of constrained devices based on available memory: Class 0 devices have much less than 10 KiB of RAM and 100 KiB of flash, Class 1 devices have roughly 10 KiB of RAM and 100 KiB of flash, and Class 2 devices have roughly 50 KiB of RAM and 250 KiB of flash. Devices in these classes share common characteristics that distinguish them from traditional computing platforms: they are equipped with low-power microcontrollers, often battery-powered or energy-harvesting (though some, such as smart building sensors, may be mains-powered), and frequently deployed in environments where physical access for maintenance is difficult or impossible. Energy is often the most critical constraint, as the radio transceiver can draw 20–30 mA during transmission or reception, compared to just 1–10 μ A during sleep modes [15]. This disparity motivates duty cycling, where the radio is turned off for most of the time and activated only when needed.

2.3.2 Wireless Communication in IoT

Constrained IoT devices communicate using a variety of low-power wireless technologies, including IEEE 802.15.4 [16], Bluetooth Low Energy (BLE), and LoRa. This thesis

uses IEEE 802.15.4, which provides a physical layer and medium access control (MAC) layer for low-rate wireless personal area networks. Operating in the 2.4 GHz ISM band, it offers data rates up to 250 kbps with a typical range of 10–100 meters.

Communication Characteristics

Wireless communication in IoT networks differs substantially from wired networks. Key characteristics include:

- **Packet loss:** Wireless links are inherently unreliable. Packet loss rates of 10–30% are common [17], and links may experience bursts of losses due to interference or fading. Protocols must be designed to tolerate message loss without compromising correctness.
- **Asymmetric links:** Unlike wired connections, wireless links may be asymmetric. A node may be able to receive packets from a neighbor but not transmit to it successfully, or vice versa. This complicates neighbor discovery and routing.
- **Variable link quality:** Link quality fluctuates over time due to environmental factors such as temperature, humidity, and physical obstructions. A link that works well at one moment may become unreliable minutes later.
- **Cross-technology interference:** IoT radios share spectrum with other wireless technologies. For example, IEEE 802.15.4 and BLE operate alongside Wi-Fi in the 2.4 GHz band, and even sub-GHz technologies can suffer from interference by other devices in the same frequency range. Coexistence with these technologies requires careful channel selection and adaptive mechanisms.
- **Broadcast nature:** Wireless transmissions are inherently broadcast; all nodes within range receive every transmission. This can be exploited for efficient flooding [17] but also means that communication overhead scales with node density.

Medium Access Control

The MAC layer manages access to the shared wireless medium. Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) is the standard approach in IEEE 802.15.4 [16]. Before transmitting, a node senses the channel to check if it is idle. If the channel is busy, the node backs off for a random period before trying again. This approach reduces collisions but cannot eliminate them entirely, especially under high load.

2.3.3 Network Topologies

IoT networks can be organized in various topologies, each with different implications for algorithm design and performance [17].

- **Star topology:** All nodes communicate directly with a central hub. Simple and efficient for small networks but creates a single point of failure and limits scalability.

- **Mesh topology:** Nodes can communicate with multiple neighbors, and messages may traverse multiple hops to reach their destination. Provides redundancy and extends range but requires routing protocols.
- **Ring topology:** Nodes are arranged in a logical ring, with each node having exactly two neighbors (predecessor and successor). Simple for certain algorithms but vulnerable to link failures.
- **Clique (fully connected):** Every node can communicate directly with every other node. Maximizes redundancy but limits scalability as broadcast overhead grows quadratically.

The choice of topology has a direct impact on algorithm design. For example, the Ring algorithm [2] achieves $O(n)$ best-case message complexity on a ring topology, while the Bully algorithm [1] has a worst-case complexity of $O(n^2)$ messages on a clique topology. The network diameter, defined as the maximum number of hops between any two nodes, determines the minimum time required for information to propagate throughout the network.

2.4 Energy Considerations in IoT Networks

2.4.1 Energy Consumption Model

Designing energy-efficient protocols requires understanding the energy consumption profile of a node. The total energy consumed can be decomposed into several components:

- **Radio transmission (TX):** Transmitting packets consumes significant power, typically 15–30 mA depending on the transmission power level [15]. Higher transmission power increases range but also increases energy consumption.
- **Radio reception (RX):** Receiving packets consumes nearly as much power as transmitting, typically 15–25 mA [15]. This is a critical observation: a node listening for incoming packets consumes almost as much energy as one actively transmitting.
- **Idle listening:** When the radio is on but no packets are being transmitted or received, the node still consumes power at the RX rate. Idle listening is a major source of energy waste in duty-cycled networks.
- **Processing (CPU):** The microcontroller consumes power during computation, typically 1–10 mA when active [15]. Modern microcontrollers offer various low-power modes that reduce consumption to microamps.
- **Sleep:** During deep sleep, consumption drops to 1–10 μ A [15]. Maximizing sleep time is the most effective strategy for extending battery life.

Table 2.3 shows power consumption values for the CC2538 SoC [18], a representative IoT platform.

The implication for protocol design is clear: minimize radio-on time. Each message sent or received has a significant energy cost. Protocols that require frequent message exchanges or continuous listening will drain batteries quickly.

Table 2.3: Power consumption of the CC2538 SoC at 3.3 V [18]

Mode	Current (mA)	Power (mW)
Radio TX (0 dBm)	24	79
Radio RX	20	66
CPU active	7	23
Sleep (PM2)	0.001	0.003

2.4.2 Software-Based Energy Estimation

Measuring actual energy consumption requires specialized hardware that is not always available, particularly during simulation-based development. An alternative is software-based energy estimation, which tracks the time spent in different power states and calculates energy consumption using known power values for each state.

By periodically sampling state counters, an application can estimate its energy consumption and the remaining battery capacity. This information can then be used to make energy-aware decisions, such as deferring non-critical tasks or adjusting transmission power. Several IoT operating systems provide built-in support for this approach, including Contiki-NG through its Energest module [19] (described in Section 2.6).

For leader election, energy awareness means that nodes with more remaining energy should be preferred as leaders. A node that becomes leader will likely consume more energy than followers due to additional coordination responsibilities. If a low-energy node is elected leader, it may fail prematurely, triggering costly re-elections. By incorporating energy into the leader selection criteria, the protocol can distribute leadership burden more fairly and extend overall network lifetime.

2.4.3 Energy-Aware Design Principles

Several design principles help minimize energy consumption in IoT protocols [15]:

- **Minimize message count:** Each message consumes energy for both transmission and reception. Reducing the number of messages reduces energy consumption proportionally.
- **Minimize message size:** Smaller messages require less time to transmit, reducing radio-on time. Protocol overhead (headers, acknowledgments) should be kept minimal.
- **Duty cycling:** Turning off the radio when not needed can reduce energy consumption by orders of magnitude [15]. Protocols should be designed to work with duty-cycled MACs.
- **Local communication:** Multi-hop communication consumes more energy than single-hop. Protocols that rely on local information rather than global state are more energy-efficient.
- **Load balancing:** Concentrating responsibilities on a few nodes drains their batteries faster. Rotating leadership and distributing tasks can balance energy consumption across the network.

2.5 Link Quality in Wireless Networks

2.5.1 Wireless Link Characteristics

Wireless links in IoT networks exhibit complex behavior that differs substantially from the idealized models often assumed in algorithm design [17]. Protocols that ignore these effects will fail in practice.

Path Loss and Fading

Radio signals attenuate as they travel through space, a phenomenon known as path loss. The signal strength decreases with distance according to a power law, typically proportional to $d^{-\alpha}$ where d is distance and α (the path loss exponent) ranges from 2 in free space to 4 or more in cluttered indoor environments [17].

Fading introduces additional variability. Multipath fading occurs when signals reach the receiver via multiple paths (reflections from walls, floors, and objects), causing constructive or destructive interference depending on the relative phases [17]. Temporal fading causes link quality to vary over time due to changes in the environment, such as moving people or objects.

The Gray Zone

Between the regions of reliable and unreliable communication lies a “gray zone” (also called the transitional region) where packet reception is probabilistic [20]. Links in the gray zone may deliver 30–70% of packets, making them partially useful but unreliable for critical communication. The gray zone is often surprisingly large, sometimes spanning 50% or more of the communication range [20].

Protocols that assume binary connectivity (either a link works or it does not) perform poorly in practice. Effective protocols must account for the probabilistic nature of wireless communication and adapt to varying link qualities.

2.5.2 Link Quality Metrics

Several metrics are used to characterize wireless link quality, each capturing different aspects of link behavior.

Received Signal Strength Indicator (RSSI)

RSSI measures the power of the received signal, typically reported in decibels relative to one milliwatt (dBm) [16]. Typical values range from -30 dBm (very strong) to -100 dBm (very weak), with receiver sensitivity around -90 to -100 dBm. RSSI provides an instantaneous snapshot of signal strength and is available for every received packet.

RSSI correlates with distance and packet reception probability but is not a perfect predictor. Two links with similar RSSI values may have different packet delivery rates due to interference, noise floors, or multipath effects. Nevertheless, RSSI is useful as a fast, low-overhead indicator of link quality.

In practice, links with RSSI above -60 dBm are typically reliable, while those below -85 dBm are often unreliable [20]. The region between -60 and -85 dBm corresponds roughly to the gray zone.

Link Quality Indicator (LQI)

LQI is a hardware-provided metric that estimates the quality of the received signal based on factors such as the chip error rate. The IEEE 802.15.4 standard [16] defines LQI as an integer from 0 to 255, with higher values indicating better quality. Unlike RSSI, which measures signal strength, LQI attempts to capture signal quality, accounting for noise and interference.

The relationship between LQI and packet delivery rate varies across hardware platforms, limiting its portability. However, on a given platform, LQI can provide useful information complementary to RSSI.

Expected Transmission Count (ETX)

ETX estimates the number of transmissions (including retransmissions) required to successfully deliver a packet across a link [21]. A perfect link has ETX of 1.0, while a link that delivers half of packets has ETX of 2.0. ETX is calculated from the forward and reverse delivery ratios:

$$\text{ETX} = \frac{1}{d_f \times d_r} \quad (2.1)$$

where d_f is the forward delivery ratio (probability that a packet from A reaches B) and d_r is the reverse delivery ratio (probability that an acknowledgment from B reaches A).

ETX is widely used in routing protocols, including RPL [22], to select paths that minimize expected transmission cost. A lower ETX indicates a more reliable link that requires fewer transmissions.

2.5.3 Link Freshness and Confidence

Link quality measurements become stale over time as wireless conditions change. A measurement taken minutes ago may no longer reflect current link conditions. Protocols must account for this by assigning confidence levels to link quality estimates based on their age.

One approach described in the link quality estimation literature is to maintain a freshness counter for each neighbor [23]. The counter is reset when a new

measurement is received and decremented periodically. Older measurements are given less weight in routing and leader selection decisions. If no measurement has been received for an extended period, the link may be considered lost.

This concept of freshness is important for leader election. A node should not be selected as leader based on outdated information about its link quality. By incorporating freshness into the scoring function, the protocol can prefer nodes with recent, confident measurements over those with stale data.

2.6 Contiki-NG and the Cooja Simulator

2.6.1 Contiki-NG Overview

Several open-source operating systems target constrained IoT devices, including RIOT, Zephyr, and Contiki-NG. All three offer small memory footprints, event-driven architectures, and IPv6 networking stacks. The distinguishing factor for this thesis is simulation support: Contiki-NG ships with the Cooja network simulator (Section 2.6.5), which enables controlled, reproducible experiments with accurate energy accounting on emulated hardware. Neither RIOT nor Zephyr provides an equivalent integrated simulator, making Contiki-NG the most practical choice for evaluating distributed algorithms before hardware deployment.

Contiki-NG [24] evolved from the original Contiki OS [25] and supports a variety of hardware platforms, including devices based on ARM Cortex-M and Texas Instruments CC26xx/CC13xx system-on-chip families.

Key features of Contiki-NG include:

- **Small footprint:** The core operating system requires only a few kilobytes of RAM and tens of kilobytes of Flash, making it suitable for severely constrained devices.
- **Event-driven architecture:** Applications are structured as event handlers that respond to system events such as timer expiration, message arrival, or sensor readings. This model is efficient for the interrupt-driven nature of IoT applications.
- **Full IPv6 networking:** Contiki-NG provides a complete IPv6 stack including 6LoWPAN header compression [26] and the RPL routing protocol [22].
- **Extensive library support:** Libraries for common tasks such as timer management, sensor access, and power management are included.
- **Active development:** The project is actively maintained with regular releases, bug fixes, and platform support updates.

2.6.2 Programming Model: Protothreads

Contiki-NG uses protothreads [27], a lightweight threading abstraction that provides a sequential programming style while maintaining the efficiency of event-driven execution. Protothreads are stackless, meaning they do not require a separate stack for each thread. This cuts memory overhead sharply compared to traditional threading.

A protothread is implemented as a C function that can yield at designated points. When a protothread yields (waiting for an event), control returns to the scheduler, which can run other protothreads or put the device to sleep. When the awaited event occurs, the protothread resumes from where it left off. Because protothreads are stackless, local variables are not preserved across yield points and must be declared static to retain their values.

The following example illustrates a simple protothread that waits for a timer. The `PROCESS_THREAD` macro expands to a function definition whose parameters are the process name, an event variable `ev`, and a data pointer `data`. The macros `PROCESS_BEGIN` and `PROCESS_END` delimit the protothread body:

```
PROCESS_THREAD(example_process, ev, data)
{
    static struct etimer timer;
    PROCESS_BEGIN();

    while(1) {
        etimer_set(&timer, CLOCK_SECOND * 5);
        PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&timer));
        printf("Timer expired\n");
    }

    PROCESS_END();
}
```

This programming model is well-suited for implementing distributed algorithms, where a process must periodically send messages and wait for responses. The protothread abstraction allows the algorithm logic to be written in a clear, sequential style while the underlying implementation remains event-driven and memory-efficient.

2.6.3 Networking Stack

Contiki-NG provides a complete networking stack for IPv6 communication over low-power wireless links [24]. The stack includes:

- **IEEE 802.15.4 MAC layer:** Implements CSMA/CA for medium access control, with optional acknowledgments and retransmissions.
- **6LoWPAN adaptation layer:** Compresses IPv6 headers to fit within the limited payload size of IEEE 802.15.4 frames (127 bytes maximum).
- **IPv6 network layer:** Provides standard IPv6 addressing and packet handling, including link-local and global addresses.
- **RPL routing:** Implements the IPv6 Routing Protocol for Low-Power and Lossy Networks, which builds a destination-oriented directed acyclic graph (DODAG) rooted at a border router.
- **UDP transport:** The `simple-udp` library provides a lightweight interface for sending and receiving UDP datagrams.

2.6.4 Energy Estimation with Energest

Contiki-NG includes the Energest module [19], which provides software-based energy accounting. Energest tracks the time each node spends in several power states:

- `ENERGEST_TYPE_CPU`: Time the CPU is active
- `ENERGEST_TYPE_LPM`: Time in low-power mode
- `ENERGEST_TYPE_TRANSMIT`: Time spent transmitting
- `ENERGEST_TYPE_LISTEN`: Time spent receiving or idle listening

By periodically reading these counters and multiplying by the known current draw for each state (e.g., from a device datasheet such as [18]), an application can estimate its cumulative energy consumption without external measurement hardware.

2.6.5 The Cooja Simulator

Cooja is a network simulator designed specifically for Contiki-NG applications [28]. It allows developers to test and debug their applications in simulation before deploying them on real hardware.

Key features of Cooja include [24, 28]:

- **Cross-level simulation:** Cooja can simulate at different levels of abstraction. At the highest level, it emulates entire nodes as Java objects. At the lowest level, it uses MSPSim to provide cycle-accurate emulation of MSP430 microcontrollers.
- **Radio medium models:** Several radio propagation models are available, including the Unit Disk Graph Medium (UDGM), which models ideal radio propagation with a configurable transmission range, and the Multi-path Ray-tracer Medium (MRM), which provides more realistic modeling of obstacles and multipath effects.
- **Visualization:** A graphical interface shows node positions, radio transmissions, and allows interactive control of the simulation.
- **Scripting:** Simulations can be controlled through JavaScript plugins that can inject faults, move nodes, or collect measurements at specified times.
- **Headless operation:** Simulations can be run without a graphical interface, enabling automated experiments and batch processing.

For evaluating leader election protocols, Cooja provides several advantages. Network topology can be precisely controlled by positioning nodes and configuring radio ranges. Faults can be injected by stopping nodes or introducing packet loss. Energy consumption can be estimated through Energest integration. And experiments can be repeated with identical conditions, ensuring reproducibility.

2.6.6 Why Contiki-NG for Leader Election Research

Contiki-NG and Cooja were selected as the implementation platform for this thesis for several reasons:

- **Realistic constraints:** Applications developed for Contiki-NG run on real hardware with real memory and energy constraints. This reduces the effort required to move protocols from simulation to actual IoT devices.
- **Energy measurement:** The Energest module provides software-based energy accounting, enabling energy-aware protocol evaluation without specialized measurement hardware.
- **Link quality metrics:** The networking stack provides access to RSSI, LQI, and ETX measurements, which link-quality-aware leader election requires.
- **Reproducibility:** Cooja enables reproducible experiments with controlled conditions.
- **Community and documentation:** Contiki-NG has an active community and extensive documentation, reducing the barrier to entry for new developers.
- **Comparison baseline:** PraSLE [5], the foundation for this thesis, also targets constrained IoT devices. Implementing it in Contiki-NG and evaluating it in Cooja enables a direct comparison under identical conditions.

2.7 Summary

This chapter introduced the foundational concepts required for the rest of the thesis. Distributed systems coordinate through message passing without shared memory or global clocks. Leader election selects a single coordinator while tolerating failures. Self-stabilization guarantees recovery from arbitrary initial states, a property well suited to unattended IoT deployments.

Constrained IoT devices as classified by RFC 7228 [14] operate under tight resource limits: kilobytes of RAM, battery or harvested power, and lossy wireless links. Energy consumption is dominated by the radio transceiver, making every message costly. Link quality varies over time, and metrics such as RSSI and ETX require continuous refreshing to remain useful.

Contiki-NG and Cooja provide the implementation and evaluation platform for this thesis. Contiki-NG's protothread model and simple-udp API make it practical to implement distributed algorithms on constrained devices. Cooja enables controlled, reproducible experiments with accurate energy accounting through Energest and realistic radio simulation.

The next chapter critically examines existing approaches to leader election, including classical algorithms, consensus protocols, self-stabilizing techniques, and energy-aware clustering, and identifies the gaps that Adaptive-PraSLE addresses. Chapter 4 then presents the design.

3 Related Work

This chapter surveys existing approaches to leader election and related coordination problems, focusing on their applicability to constrained IoT networks. It examines classical leader election algorithms and their limitations, consensus-based protocols, energy-aware clustering schemes, and self-stabilizing approaches. For each category, the discussion highlights what researchers have proposed and where their solutions fall short of what IoT deployments require. The chapter concludes by identifying the specific gaps that Adaptive-PraSLE addresses.

3.1 Classical Leader Election Algorithms

Chapter 2.1.3 introduced the Bully algorithm [1] and the Chang-Roberts Ring algorithm [2]. Both assume reliable failure detection and stable communication, properties that do not hold in lossy wireless IoT networks. The Bully algorithm's $O(n^2)$ worst-case message complexity and requirement for direct communication between all processes make it expensive on constrained devices. The Ring algorithm requires a pre-established ring topology; any link failure breaks the ring and halts the election.

Beyond these two, several other classical approaches have been proposed. Gallager *et al.* [29] developed an algorithm for constructing a minimum-weight spanning tree, which implicitly elects a leader as the tree's root. GHS achieves $O(n \log n)$ message and time complexity but requires edge weights and reliable FIFO channels. Le Lann [30] introduced token-based coordination, where a circulating token grants temporary leadership. This concept influenced the Chang-Roberts ring algorithm [2], the Hirschberg-Sinclair bidirectional ring algorithm [31], and the IEEE 802.5 Token Ring standard [32]. Token-based approaches share the Ring algorithm's vulnerability to token loss and ring breaks.

Rahman [33] surveyed leader election algorithms specifically for IoT and wireless sensor networks. The survey categorizes leader election approaches by their communication model, fault tolerance properties, and intended function. Rahman identifies that most existing protocols still assume reliable communication and do not account

for the transient faults common in wireless deployments. The survey highlights energy constraints, unreliable wireless communication, dynamic topology changes, and scalability to thousands of nodes as the primary challenges for IoT leader election. It also identifies self-stabilizing approaches and multi-leader election as promising directions that remain underexplored.

On the theoretical side, Kutten *et al.* [34] demonstrated that randomized leader election in general synchronous networks can be made *singularly near-optimal*, simultaneously achieving near-optimal time ($O(D)$) and message complexity ($O(m \log n)$) with high probability. Kutten *et al.* [35] extended this result to general asynchronous networks, achieving $O(D + \log^2 n)$ time and $O(m \log^2 n)$ messages. These results show that the classical complexity bounds can be significantly improved. However, both algorithms assume reliable point-to-point communication and are not designed for lossy wireless environments.

Table 3.1 summarizes the classical algorithms discussed above. None provides self-stabilization or recovery from arbitrary state corruption.

Table 3.1: Comparison of classical leader election algorithms

Algorithm	Messages	Time	Topology	Fault Recovery
Bully	$O(n^2)$	$O(d)$	Clique	No
Ring	$O(n) - O(n^2)$	$O(n \cdot d)$	Ring	No
GHS	$O(n \log n)$	$O(n \log n)$	Weighted graph	No
Token Ring	$O(n)$	$O(n \cdot d)$	Ring	No
Kutten et al. (sync)	$O(m \log n)$	$O(D)$	General	No
Kutten et al. (async)	$O(m \log^2 n)$	$O(D + \log^2 n)$	General	No

3.2 Consensus-Based Approaches

Consensus and leader election are related but distinct problems. Consensus guarantees agreement on arbitrary values with total ordering and durable decisions, tolerating process crashes or, in stronger models, arbitrary (Byzantine) faults. Leader election only requires that all processes identify the same coordinator, with no ordering and no persistence requirement; if the leader fails, a new one is elected. Although leader election can be reduced to consensus, the guarantees that consensus provides are disproportionate to this narrower problem. This section examines prominent consensus protocols and explains why their overhead exceeds what IoT leader election requires.

Paxos [3, 36] requires at least two message round-trips per decision, majority quorums among $2f+1$ acceptors, and partial synchrony for progress. Raft [4] provides equivalent fault tolerance with a clearer structure but adds a replicated log and routes all operations through a strong leader, both unnecessary for leader election. PBFT [37] tolerates Byzantine faults using $3f+1$ nodes and $O(n^2)$ messages per decision with cryptographic authentication, but IoT failures are typically transient (interference, low battery) rather than malicious. All three protocols require majority quorums and multiple message round-trips, overhead that is disproportionate when the only goal is to agree on a coordinator identity.

3.2.1 Applicability of Consensus to IoT Leader Election

Consensus solves a harder problem than leader election requires. It guarantees agreement on arbitrary values with total ordering and durable decisions. Leader election only requires agreement on an identity, with no ordering and no persistence requirement. If the leader fails, a new one is elected. The guarantees that consensus provides are disproportionate to this narrower problem.

More practically, the message and storage overhead of consensus protocols is difficult to justify on constrained devices when only a coordinator identity needs to be agreed upon. A protocol that provides eventual agreement, recovers automatically from faults, and minimizes communication is better suited to IoT. This reasoning leads to self-stabilization (Section 3.3).

3.3 Self-Stabilizing Leader Election

Self-stabilization (introduced in Section 2.2) offers an alternative to the fault prevention used by classical leader election algorithms and the fault masking through replication. Rather than assuming faults do not occur or hiding them behind redundant copies, self-stabilizing algorithms accept that faults may corrupt system state and guarantee recovery to a correct state within bounded time.

3.3.1 Theoretical Foundations

Dijkstra's original self-stabilizing algorithms [6] showed that distributed systems could be designed to recover from arbitrary initial states. His work on token ring stabilization showed that even without explicit initialization, a system could converge to a state with exactly one token.

Dolev's textbook [13] established self-stabilization as a mature field. Key results include:

- **Composition:** Self-stabilizing algorithms can be composed to build complex systems while preserving stabilization properties.
- **Complexity bounds:** Lower bounds on stabilization time and space have been established for various problems.
- **Fault models:** Self-stabilization tolerates transient faults that corrupt state but does not handle permanent failures without additional mechanisms.

Arora and Gouda [38] formalized the concepts of closure (remaining in legitimate states) and convergence (reaching legitimate states), providing a framework for reasoning about self-stabilizing algorithms.

3.3.2 Population Protocols

Population protocols [39, 40] model distributed computation as pairwise agent interactions chosen by an adversarial scheduler. Beauquier *et al.* [41] developed self-stabilizing leader election for this model, achieving stabilization in $O(n^3)$ expected interactions. However, the pairwise interaction assumption does not match the broadcast nature of wireless IoT communication, and the $O(n^3)$ convergence bound is impractical for networks where encounters are infrequent.

3.3.3 Recent Advances in Self-Stabilizing Leader Election

Since these early results, self-stabilizing leader election has progressed in two directions: reducing complexity bounds and expanding the class of networks for which solutions exist.

Altisen *et al.* [42] proposed the first silent self-stabilizing leader election algorithm for arbitrary identified networks with proven polynomial step complexity. Their algorithm works under the distributed unfair daemon in the locally shared memory model, requires no global knowledge of the network (neither an upper bound on the diameter nor the number of processes), and achieves stabilization in $\Theta(n^3)$ steps with $\Theta(\log n)$ bits of memory per process, which is asymptotically optimal. This result is relevant to IoT deployments where topology information may be unavailable.

Altisen *et al.* [43] extended this line of work to directed networks, characterizing exactly when self-stabilizing leader election is possible in digraphs. They showed that leader election can be self-stabilizingly solved if and only if the network contains a unique source component. This result is particularly relevant to IoT networks where asymmetric radio ranges create directed communication links: a node may hear a neighbor but not vice versa.

Altisen *et al.* [44] addressed self-stabilizing leader election in highly dynamic time-varying graphs, defining three classes of network dynamics (bounded diameter, quasi-bounded diameter, and recurrent connectivity) and providing algorithms for each class. This work bridges the gap between static self-stabilization and the reality of mobile or changing IoT networks.

Sudo *et al.* [45] achieved time-optimal self-stabilizing leader election in the population protocol model, showing that the loosely-stabilizing variant can converge in polylogarithmic time. Their work also established that strictly self-stabilizing leader election in population protocols is impossible unless the exact number of agents is known a priori.

Mo *et al.* [46] proposed a self-stabilizing leader election algorithm based on aggregate computing, where leaders are elected through locally computed distance and pseudo-diameter estimates that converge to near-optimal values. The algorithm achieves $O(\text{diameter})$ stabilization time with small messages and no flooding, making it suitable for large-scale IoT deployments such as smart cities.

None of these algorithms incorporates energy awareness or link quality into the leader selection decision. They elect leaders based on identifiers or topological properties, not runtime conditions.

3.3.4 PraSLE: Practical Self-Stabilizing Leader Election

PraSLE [5], introduced in Section 2.2.3, forms the foundation for this thesis. This section focuses on what distinguishes PraSLE from prior work and identifies its limitations.

Key Contributions

PraSLE differs from the approaches discussed in the preceding sections. Unlike population protocols [41], which assume pairwise interactions and an adversarial scheduler, PraSLE uses broadcast communication and operates in synchronous rounds, matching the communication patterns of wireless sensor networks. Unlike the GHS algorithm [29], which requires edge weights and reliable FIFO channels, PraSLE works on any connected topology with only basic message passing and timers. Its unreliable mode provides self-stabilization through continuous election cycles, tolerating message loss without requiring reliable delivery. The algorithm achieves $O(K \cdot n \cdot \Delta)$ message complexity per cycle, $O(K \cdot T)$ stabilization time, and $O(1)$ space per node [5], making it practical for Class 0 and Class 1 devices [14].

Limitations

PraSLE has limitations that motivate this thesis. The following are identified through analysis of the original algorithm [5]:

- **Static selection criteria:** The ranking function uses node identifiers, ignoring energy levels and link quality. A node with low battery may become leader and fail prematurely. Other approaches have demonstrated the value of richer selection criteria: HEED [47] uses residual energy and communication cost, LEACH [11] rotates the coordinator role probabilistically, and SEALEA [48] selects k -leaders based on energy awareness. However, none of these are self-stabilizing.
- **Fixed timeouts:** The parameter T does not adapt to network conditions. In variable environments, this leads to either slow fault detection (if T is too large) or false failures (if T is too small).
- **No controlled rotation:** Leadership is permanent until failure, causing uneven energy depletion at the leader.
- **Full re-election on failure:** When the leader fails, the entire K -round election must complete before a new leader is established.
- **High message overhead:** Unreliable mode broadcasts every round, even when the network is stable.

3.4 Energy-Aware Clustering

A separate line of research addresses energy efficiency in wireless sensor networks through clustering, where nodes organize into groups and select cluster heads that aggregate data and communicate with a base station. Cluster-head selection and leader election serve different purposes:

- **Clustering:** Organizes nodes for efficient data aggregation and communication with a base station. It typically operates within a centralized architecture where a base station coordinates the process, and it produces multiple cluster heads (one per cluster).
- **Leader election:** Selects a single coordinator for the entire network (or partition). The leader makes decisions or coordinates actions on behalf of all nodes, without relying on a centralized base station.

Despite these structural differences, the two problems share the challenge of selecting coordinators under energy constraints, and the techniques developed for energy-aware cluster-head selection are relevant to leader election.

3.4.1 LEACH

LEACH (Low-Energy Adaptive Clustering Hierarchy) [11] is the foundational energy-aware clustering protocol. LEACH operates in rounds, with each round consisting of a setup phase (cluster formation) and a steady-state phase (data transmission).

In the setup phase, each node decides whether to become a cluster head based on a probability that depends on the desired percentage of cluster heads and whether the node has recently been a cluster head. This probabilistic approach distributes the cluster-head burden over time.

LEACH's contributions relevant to leader election include:

- **Rotation:** By rotating cluster heads, LEACH distributes energy consumption across nodes.
- **Probabilistic selection:** The selection mechanism avoids explicit coordination, reducing overhead.
- **Round-based operation:** The concept of rounds with distinct phases influences protocol design.

However, LEACH assumes single-hop communication to the base station and uniform node distribution, limiting its applicability.

3.4.2 HEED

HEED (Hybrid Energy-Efficient Distributed clustering) [47] improves upon LEACH by using a hybrid criterion for cluster-head selection: a primary parameter (residual energy) and a secondary parameter (node degree or proximity to neighbors).

HEED's multi-criteria approach is directly relevant to the composite scoring function used in Adaptive-PraSLE (Chapter 4). However, HEED focuses on clustering for data aggregation rather than leader election for coordination.

3.4.3 PEGASIS

PEGASIS (Power-Efficient Gathering in Sensor Information Systems) [49] organizes nodes into a chain rather than clusters. Data is aggregated along the chain, with one node designated as the leader to transmit to the base station. The leader role rotates to balance energy consumption.

Lindsey *et al.* demonstrated that chain-based organization with rotating leadership can outperform cluster-based approaches in some scenarios. However, chain construction requires global knowledge or significant coordination.

3.4.4 TEEN and APTEEN

TEEN (Threshold-sensitive Energy Efficient sensor Network protocol) [50] and APTEEN [51] are hierarchical clustering protocols designed for time-critical applications. They use hard and soft thresholds to control when nodes transmit data, reducing unnecessary transmissions.

While focused on data-driven reporting, these protocols illustrate the importance of threshold-based decision-making in resource-constrained networks.

3.4.5 Energy-Aware Leader Election

Beyond clustering, several protocols have been proposed that address energy consumption specifically in the context of leader election rather than cluster-head selection.

Haddar [48] proposed SEALEA, a scalable and energy-aware algorithm for electing k leaders in IoT wireless sensor networks. SEALEA selects leaders based on energy levels to ensure high availability despite battery exhaustion and node failures.

Bounceur *et al.* [52] proposed BROGO (Branch Optima to Global Optimum), which constructs a spanning tree and elects branch-local leaders before selecting a global leader at the root. BROGO achieves $O(n)$ message complexity and reduces energy consumption compared to the classical minimum-finding algorithm by limiting the number of broadcast messages. The same authors proposed LOGO [53] (Local Optima to Global Optimum), which uses a similar local-to-global strategy where each node sends exactly one message, achieving minimum message complexity.

The same research group introduced two additional approaches. WBS (Wait-Before-Starting) [54] uses a staggered start strategy where each node waits a time proportional to its value before initiating flooding. When two spanning trees meet, the one with the better leader value absorbs the other. Each node sends exactly one message, making WBS both energy-efficient and fault-tolerant. DoTRo (Dominating Tree Routing) [55] extends this idea with a tree routing protocol designed for disconnected and failure-prone networks, achieving energy reductions while handling node failures and network disconnection.

These protocols demonstrate that energy-aware leader election is an active area of research. However, none of them provides self-stabilization or recovery from corrupted state.

3.4.6 Comparison with Leader Election

Several mechanisms developed for clustering have influenced the design of Adaptive-PraSLE. LEACH's round-based rotation of the coordinator role inspired the controlled leader rotation mechanism. HEED's use of residual energy as a primary selection criterion informed the composite scoring function that incorporates energy levels. The threshold-based transmission control in TEEN and APTEEN is conceptually related to conditional broadcasting, where messages are sent only when relevant state changes occur.

However, existing clustering protocols provide energy awareness without self-stabilization. If a cluster head fails or state becomes corrupted, none of these protocols can recover without restarting the clustering process. Conversely, self-stabilizing protocols such as PraSLE (Section 3.3.4) provide fault recovery without energy awareness. This gap motivates combining both properties.

3.5 Link-Quality-Aware Protocols

Link quality has a direct impact on wireless network performance. Several metrics and protocols have been developed to exploit link quality information.

3.5.1 Link Quality Metrics in Routing

The ETX metric [21] (introduced in Section 2.5) is widely used in routing protocols to select paths that minimize transmission cost. The RPL routing protocol [22] introduced the concept of pluggable objective functions for path selection. Two objective functions have been defined in separate specifications: OF0 [56], which minimizes hop count, and MRHOF [57], which uses ETX to minimize expected transmissions. These demonstrate how link quality metrics can be integrated into protocol decisions. However, these applications are limited to routing rather than leader election.

3.5.2 Link Quality in Leader Selection

Among the leader election protocols discussed in this chapter, none incorporates link quality into the leader selection decision. The Bully and Ring algorithms use node identifiers, LEACH uses probabilistic rotation, HEED uses residual energy and communication cost, and PraSLE uses a static ranking value. A leader with poor connectivity to its neighbors will have difficulty coordinating the network regardless of its identifier or energy level.

Adaptive-PraSLE addresses this by incorporating link quality into its scoring function, preferring nodes with good average connectivity to neighbors.

3.6 Adaptive Mechanisms in Distributed Systems

Adaptive mechanisms allow protocols to adjust their behavior based on observed conditions rather than relying on fixed parameters.

Jacobson [58] introduced the smoothed RTT estimator for TCP, which maintains an exponentially weighted moving average of round-trip times with variance tracking to compute retransmission timeouts. This approach produces aggressive timeouts in stable networks and conservative timeouts in variable ones. The technique has become a standard building block in networking but has not been applied to leader election timeout management.

Chandra *et al.* [59] formalized failure detectors as an abstraction for distributed systems. A failure detector is not a specific implementation but a specification of properties that any failure detection mechanism must satisfy, classified by completeness (every failed process is eventually suspected) and accuracy (correct processes are not falsely suspected). Chandra *et al.* defined a hierarchy of failure detector classes with varying capabilities, ranging from the perfect failure detector $\mathcal{P}$ (which never makes mistakes) to weaker classes such as the eventually perfect detector $\diamond\mathcal{P}$ and the eventual leader detector Ω . Their subsequent work [60] showed that Ω is the weakest failure detector sufficient for solving consensus. Ω eventually outputs the same correct process at all correct processes, amounting to eventual leader election. This result establishes a theoretical link between failure detection and leader election but does not address how to implement efficient failure detection on constrained devices with lossy links.

Existing self-stabilizing leader election protocols, including PraSLE, use fixed timing parameters that must be configured conservatively to avoid false failure detections. No prior work has combined adaptive timeout estimation with self-stabilizing leader election for IoT. Chapter 4.4 describes how Adaptive-PraSLE addresses this.

3.7 Leader Election in Mobile and Robotic Networks

Leader election has also been studied in mobile settings. These approaches illustrate how leader selection criteria vary with the application domain.

3.7.1 Lunar Leader

The Lunar Leader algorithm [12] addresses leader election in multi-robot exploration teams. The algorithm selects leaders based on mission-relevant criteria such as position, communication capability, and task suitability. It is designed for scenarios where the leader must be physically well-positioned to coordinate the team.

Lunar Leader demonstrates the importance of context-aware leader selection in robotics but addresses a different set of constraints than IoT networks.

3.7.2 Stabilizing Flocking

Canepa *et al.* [61] investigated self-stabilizing leader election as a component of flocking algorithms for robot swarms. In their model, robots must elect a leader to determine the flock's direction. The work demonstrates that self-stabilization is valuable in mobile scenarios as well as static ones.

Flocking algorithms [62] coordinate group behavior without explicit leader election, using local rules to achieve emergent coordination. However, many practical scenarios require a designated leader for decision-making.

3.7.3 Topology-Aware Leader Election in Dynamic Networks

Favier *et al.* [63] proposed CEL, a topology-aware leader election algorithm for dynamic networks. Each node maintains local knowledge of the communication graph and uses it to elect the node with the highest closeness centrality as leader. Electing a topologically central leader means information can spread faster from the leader to all nodes, reducing communication overhead. CEL uses cross-layer neighbor detection and a self-pruning mechanism based on topological knowledge combined with probabilistic gossip for efficient broadcast. Favier *et al.* [64] extended this work with improved algorithms for eventually electing one leader per connected component in dynamic evolving networks.

These topology-aware approaches are relevant to IoT because they optimize leader placement for communication efficiency. However, they focus on topological centrality and do not incorporate energy awareness or link quality into the selection decision.

3.7.4 Multi-Leader and Hierarchical Election

Khalid *et al.* [65] proposed algorithms for multi-leader election in dynamic sensor networks, where multiple leaders partition the network to distribute coordination load and save energy. Their approach uses token handover when a leader's energy drops below a threshold, achieving $O(N_{\text{radius}} \cdot \log N_{\text{radius}} + D)$ time complexity. Multi-leader election is relevant to large IoT deployments where a single leader creates a bottleneck, but it introduces additional complexity for inter-leader coordination that this thesis does not address.

3.8 Research Gaps and Positioning

The preceding sections identify several capabilities that IoT leader election requires but no existing protocol provides. Leader election is not restricted to stationary networks, as the mobile and robotic approaches above demonstrate, but this thesis targets static or quasi-static IoT deployments where energy constraints and lossy links are the primary challenges.

3.8.1 Identified Gaps

Gap 1: No protocol combines energy awareness with self-stabilization. PraSLE [5] provides self-stabilization but ignores energy. LEACH [11] and HEED [47] consider energy but cannot recover from corrupted state. No existing protocol provides both properties.

Gap 2: Link quality is not used in leader selection. Routing protocols have used ETX [21] and RSSI for path selection, yet leader election protocols do not incorporate link quality. A leader with poor connectivity to its neighbors will perform poorly regardless of its identifier or energy level.

Gap 3: Fixed timeouts in variable wireless environments. Jacobson [58] demonstrated decades ago that adaptive timeouts outperform fixed ones for TCP. Self-stabilizing leader election protocols still use static timeout parameters, even though wireless conditions fluctuate more than wired ones. Fixed timeouts that are too short cause false failure detections; timeouts that are too long delay recovery.

Gap 4: No graceful leadership transition. In PraSLE, a leader serves until failure. LEACH rotates cluster heads randomly without considering which node should lead next. Neither approach supports a controlled handover when a leader's energy is running low but it has not yet failed.

Gap 5: Full re-election on every failure. When a PraSLE leader fails, the network must complete K full rounds before a new leader is established. Maintaining backup candidates could reduce recovery time.

3.8.2 Summary Comparison

Table 3.2 compares the approaches across all categories, including energy-aware clustering protocols LEACH [11] and HEED [47] for reference.

Table 3.3 provides a detailed comparison of message overhead between the self-stabilizing approaches implemented in this thesis.

The larger message size of Adaptive-PraSLE (14 bytes vs. 8 bytes for PraSLE) is due to the additional fields for energy level, average RSSI, neighbor count, and status flags. Adaptive-PraSLE uses conditional broadcasting (sending only when values change), which introduces a trade-off: while it reduces the number of messages in stable networks, a corrupted value at a receiving node will only be corrected once the sender's value changes again. However, since the algorithm operates in continuous election cycles, the reset at the beginning of each cycle re-initializes all values and triggers a fresh propagation, preserving the self-stabilization guarantee. Both algorithms achieve identical recovery times since they use the same reset-cycle mechanism for fault tolerance. A detailed comparison of message overhead is provided in Chapter 6.

Table 3.2: Comparison of leader election approaches

Approach	Self-Stab.	Energy	Link	Adaptive	Rotation	Msg. Complexity
Bully	No	No	No	No	No	$O(n^2)$
Ring	No	No	No	No	No	$O(n)$
Paxos/Raft	No	No	No	No	No	$O(n)-O(n^2)$
LEACH	No	Yes	No	No	Yes	$O(n)$
HEED	No	Yes	Partial	No	Yes	$O(n \log n)$
BROGO	No	Yes	No	No	No	$O(n)$
WBS	No	Yes	No	No	No	$O(n)$
SEALEA	No	Yes	No	No	No	—
Altisen et al.	Yes	No	No	No	No	—
Beal et al.	Yes	No	No	No	No	$O(\text{diam.})$
CEL	Eventual	No	No	No	No	—
PraSLE	Yes	No	No	No	No	$O(K \cdot n)$
Adaptive-PraSLE	Yes	Yes	Yes	Yes	Yes	$O(K \cdot n)$

Table 3.3: Message overhead comparison of self-stabilizing leader election

Algorithm	Broadcast Strategy	Message Size	Recovery Time
Bully	On-demand	4–8 bytes	$O(1)$
Ring	Sequential	4–8 bytes	$O(n)$
PraSLE	Unconditional	8 bytes	$O(K \cdot T)$
Adaptive-PraSLE	Conditional	14 bytes	$O(K \cdot T)$

Adaptive-PraSLE extends PraSLE with energy-aware and link-quality-aware scoring, adaptive timeout mechanisms, controlled leader rotation, and backup leader tracking to address these gaps. The design of these extensions is presented in Chapter 4.

4 Concept and Solution Approach

PraSLE [5] provides self-stabilizing leader election with low overhead, but it does not consider energy levels, link quality, or changing network conditions when selecting a leader. This chapter describes how Adaptive-PraSLE addresses these limitations through five extensions: energy-aware scoring, link-quality-aware scoring, adaptive timeouts, controlled leader rotation, and backup leader tracking. The next chapter covers implementation details; here the focus is on what the extensions do and why they are effective.

Scope. This thesis focuses on static or quasi-static networks where node positions are fixed and the set of neighbors does not change. Full mobility support, including handling frequent topology changes, position-aware leader selection, and continuous reconfiguration, is left for future work.

4.1 Extension Philosophy

Several principles guide the design of Adaptive-PraSLE.

Preservation of core guarantees. PraSLE’s round-based minimum-finding algorithm already handles leader election correctly. The extensions modify how scores are computed and how timeouts are managed, but they do not change the core propagation logic. The mathematical guarantees from PraSLE’s formal verification [5] therefore carry over.

Context awareness with minimal hardware requirements. The extensions rely on signal strength (from the radio transceiver), neighbor count (derived from received messages), and battery level. Battery monitoring is not universally available: it requires either an on-chip battery monitor (e.g., the AON_BATMON peripheral on Texas Instruments CC26xx), an ADC with a voltage divider, or an external BMS connected via I2C or UART. In the Cooja simulation environment, Contiki-NG’s Energest module [19] provides software-based energy estimation as a substitute. The composite scoring function combines these inputs into a single value that replaces PraSLE’s static node ID.

Modularity. Each extension can be independently enabled or disabled at compile time. The protocol remains correct regardless of which extensions are active; only performance characteristics change. This supports ablation studies that isolate each mechanism’s contribution.

4.2 Justification of Design Choices

The design choices of the solution implemented in the context of this thesis follow directly from the practical constraints of IoT and robotic clusters and the specific gaps identified in Chapter 3 (see Section 3.8). Each design decision (the choice of PraSLE as the base protocol, the energy and link-quality scoring metrics, and the mechanisms for adaptive timeouts, controlled rotation, and backup recovery) addresses a concrete problem and supports the overall objective of achieving leader election that is lightweight, adaptive, and fault-tolerant.

4.2.1 Choice of PraSLE as the Base Protocol

PraSLE [5] was selected because it combines self-stabilization with a lightweight design tailored to resource-constrained IoT networks, and it already provides several of the core properties this thesis builds upon.

Lightweight design suitable for constrained IoT. PraSLE was explicitly engineered for small microcontrollers with limited RAM and ROM [14], strict energy budgets, low-power wireless links, and low message complexity. This matches the constraints of the target environment (IoT motes, small robots).

Self-stabilization and fault tolerance. PraSLE guarantees convergence to a correct state from any arbitrary configuration of its protocol variables (Section 2.2). This covers transient faults that corrupt protocol state, such as power glitches resetting variables, message loss causing inconsistent views, or nodes rebooting into arbitrary states. It does not cover permanent faults such as bitflips in program code or persistent hardware failures, which require orthogonal mechanisms (e.g., code redundancy or watchdog timers). IoT clusters experience frequent packet loss [17, 20] and unpredictable reboots [13], making self-stabilization a practical necessity that lightweight IoT protocols such as LEACH [11], HEED [47], Bully [1], and Ring [2] do not provide.

Extensibility aligned with identified gaps. This thesis introduces energy-aware scoring, link-quality-aware scoring, controlled leader rotation, adaptive timeouts, and backup-based failover. PraSLE lacks these features but is modular enough to be extended, making it both a solid foundation and a suitable target for the proposed improvements.

Practicality and implementability. PraSLE can be implemented in Contiki-NG [24]/Cooja [28], which simulates IoT motes realistically, provides built-in energy models, provides wireless link-quality instrumentation, and requires minimal custom modeling.

Unsuitability of alternatives. The alternatives discussed in Chapter 3 each lack properties required for this work. Bully [1] and Ring [2] achieve comparable convergence

times but are not self-stabilizing and cannot recover from arbitrary state corruption. GHS [29] requires reliable FIFO channels. Paxos [3] and Raft [4] implement full consensus, which is unnecessary for the narrower task of leader election and requires persistent state and majority quorums. Population protocols [41] assume pairwise interactions that do not map to broadcast-based IoT communication. PraSLE combines self-stabilization with a lightweight design purpose-built for constrained IoT devices.

4.2.2 Choice of Bully and Ring as Baseline Protocols

Bully [1] and Ring [2] are included as baselines because they are canonical classical leader election algorithms covered in standard distributed systems textbooks [8, 9] and surveys [33], with different trade-offs in message complexity: Bully uses $O(n^2)$ messages but converges quickly, while Ring uses $O(n)$ messages but converges slowly. Both assume reliable communication and lack energy awareness or self-stabilization, making them representative of the limitations that IoT-specific protocols must overcome. Including them alongside PraSLE strengthens external validity by showing how self-stabilizing, context-aware design compares to classical approaches.

4.2.3 Rationale for Energy- and Link-Quality-Aware Metrics

IoT devices operate under strong energy constraints [15] and unreliable wireless links with variable packet loss, latency, and connectivity [20]. These factors have direct influence on whether a leader can perform its duties effectively.

Therefore, integrating energy (battery level, consumption rate) and link-quality metrics (RSSI, packet delivery ratio, neighborhood connectivity) [23] into the scoring function is a well-motivated extension. A leader that depletes its battery quickly or has unstable communication links reduces cluster stability, increases election frequency, and risks system failures.

Adding these metrics moves the protocol toward context-aware leader election, in which the “best” leader is chosen for current environmental and network conditions rather than static identifiers. This directly aligns with the identified gap: PraSLE lacks context-aware leader selection, despite the practical need for it in real IoT and robotic deployments.

4.2.4 Rationale for Controlled Leader Rotation

Issue: In standard PraSLE, a leader serves until it fails [5]. This concentrates energy consumption on a single node, which eventually depletes its battery and crashes, triggering a full re-election cycle. The problem worsens in networks where some nodes have lower initial energy, as the same node may be repeatedly elected and drained [15].

Solution: Controlled leader rotation allows a leader to voluntarily hand over its role when its energy drops below a configurable threshold, before failure occurs. The protocol ensures that handovers are coordinated (avoiding split-leadership states) and

that rapid oscillations are prevented through hysteresis. The threshold and successor-selection policy are application-level parameters, while the protocol guarantees correctness and stable state transitions during handover.

4.2.5 Rationale for Adaptive Timeout Tuning

Issue: PraSLE uses a fixed timeout for fault detection [5]. However, IoT networks experience fluctuating latency due to interference and contention [17, 20]. If the timeout is too short relative to current network conditions, nodes falsely suspect leader failure and trigger unnecessary re-elections. If it is too long, actual failures go undetected.

Solution: Adaptive timeout tuning replaces the fixed value with a dynamic timeout computed from observed message round-trip times using Jacobson’s algorithm [58] (detailed in Section ??). When contention or interference delays heartbeat messages, the smoothed RTT estimate increases, which automatically extends the timeout and prevents false failure detection. Conversely, when heartbeats arrive consistently on time (low SRTT and low RTTVAR), the computed timeout decreases, down to a configurable minimum bound that prevents premature detection (Section ??).

4.2.6 Rationale for Backup-Based Fault Recovery

Leader failure can disrupt coordination, especially in robotic systems where leader roles may be mission-critical. Relying on full re-election after each failure increases convergence time and communication cost.

Maintaining a small ranked backup list of successors provides fast handover when the leader fails, reduced re-election frequency, and improved stability under intermittent node failures. This design choice is justified by the requirement that leader availability is critical and that existing mechanisms are too slow or energy-inefficient under frequent failures.

4.3 The Five Extensions

Adaptive-PraSLE introduces five extensions to the base PraSLE protocol. Each addresses a specific limitation identified in Section 3.8 and can be enabled or disabled independently at compile time, preserving the original PraSLE guarantees (self-stabilization, correctness) regardless of which extensions are active.

4.3.1 Energy-Aware Leader Selection

Problem: PraSLE [5] selects leaders based solely on node IDs, which bear no relation to a node’s remaining energy. A low-ID node may have the least energy in the network yet still be elected, leading to premature leader failure and repeated re-elections.

Solution: Nodes report their remaining energy as part of their ranking score. Nodes with depleted batteries receive higher (worse) scores and are less likely to be elected. The scoring function can be tuned to weight energy heavily or lightly depending on application requirements.

Trade-off: Energy estimation requires periodic sampling, which itself consumes energy [19]. Whether the monitoring cost is offset by improved leader selection is evaluated in Chapter 6.

4.3.2 Link-Quality-Aware Leader Selection

Problem: A node may have sufficient remaining energy but poor radio links. If elected leader, it cannot reliably communicate with followers. Messages are lost, followers falsely suspect leader failure, and the network triggers unnecessary re-elections.

Solution: Nodes track link quality to their neighbors using RSSI (Received Signal Strength Indicator) and LQI (Link Quality Indicator) [23]. A node with consistently weak links receives a higher (worse) score. The scoring function also considers a freshness confidence factor: stale link quality data is penalized to avoid decisions based on outdated information.

Trade-off: Link quality changes over time. The protocol uses Exponentially Weighted Moving Averages (EWMA) to smooth short-term fluctuations while tracking longer-term trends. This introduces latency in detecting link quality changes but prevents overreaction to transient interference.

4.3.3 Composite Scoring Function

Problem: Considering multiple factors (energy, link quality, connectivity, CPU load) raises the question of how to combine them. A node might have high remaining energy but poor links, or vice versa. Simple rules such as selecting the node with the highest energy ignore other relevant factors.

Solution: A weighted sum combines four normalized components into a single composite score:

$$\text{Score} = w_E \cdot S_E + w_L \cdot S_L + w_C \cdot S_C + w_P \cdot S_P \quad (4.1)$$

where S_E , S_L , S_C , S_P are scores for energy, link quality, connectivity, and CPU respectively, and w_E , w_L , w_C , w_P are configurable weights. Lower scores indicate better candidates. The default weights (30% energy, 30% link quality, 20% connectivity, 20% CPU) were chosen based on preliminary experiments but can be adjusted for different applications.

Normalization ensures each component contributes proportionally regardless of its raw scale. Energy percentage (0-100) and RSSI values (typically -90 to -30 dBm) are both mapped to 0-100 ranges before weighting.

Trade-off: The weights are tunable but not self-tuning. An application that prioritizes energy over link quality must configure different weights. Adaptive weight adjustment based on network conditions is a potential future extension.

4.3.4 Controlled Leader Rotation

Problem: Once elected, a PraSLE leader serves until it fails [5]. This maximizes stability but concentrates energy drain on a single node. Eventually that node depletes its energy and fails, triggering re-election. With energy-aware scoring, a different node is elected, but the same pattern repeats.

Solution: When a leader's energy falls below a critical threshold, it initiates a controlled handover to a successor. The handover protocol uses a request-acknowledgment exchange to coordinate the transfer:

1. Leader detects low energy (e.g., below 20%)
2. Leader selects best successor from known candidates
3. Leader sends HANDOVER_REQ to successor
4. Successor acknowledges with HANDOVER_ACK and assumes the leader role
5. Old leader receives the acknowledgment and steps down
6. Both nodes broadcast the updated leadership to followers

A brief overlap period exists during which both nodes consider themselves leader (between step 4 and step 5). This is preferable to the alternative of stepping down before confirmation, which would create a leaderless gap that could trigger unnecessary failure detection and re-elections. The overlap is resolved within one message round-trip, and the self-stabilizing property of the protocol guarantees eventual convergence to a single leader.

The protocol includes safeguards against oscillation (minimum leadership term) and failures (acknowledgment timeout with retry, fallback to backup candidates).

Trade-off: Handover adds message overhead and introduces a brief period of uncertainty. The alternative (waiting for leader failure) causes longer disruption. In most cases, graceful handover is preferable, but applications that prioritize stability over energy balance can disable it.

4.3.5 Backup Leader List

Problem: When a leader fails unexpectedly (no handover), the network must re-elect. Standard PraSLE re-election takes K rounds [5], where K depends on network diameter. For a 10-node line topology with $K = 10$ and $T = 1$ second per round, recovery takes 10 seconds.

Solution: Each node maintains a ranked list of backup leader candidates based on observed scores during elections. When the leader fails, nodes first check if any backup candidate is available. If so, they can bypass the full election:

1. Follower detects leader failure (missed heartbeats)
2. Follower consults backup list for next-best candidate

3. If backup is reachable, followers converge on backup as new leader
4. Full election runs in background to confirm or correct

The backup list is updated after each election cycle and purged when entries become stale.

Trade-off: The backup list requires additional per-node state (one entry per tracked candidate). To keep the protocol lightweight, only the top few candidates are stored. This may miss the optimal successor if many nodes have similar scores.

4.4 Adaptive Timeout Mechanism

PraSLE uses a fixed timeout T for each election round [5]. If T is too short, nodes may time out before receiving messages from slow neighbors, leading to incorrect elections. If T is too long, the protocol wastes time waiting when the network is idle.

Adaptive-PraSLE replaces the fixed timeout with a dynamic one based on observed network latency. The approach is inspired by TCP congestion control [58]:

1. Track round-trip times (RTT) for message exchanges
2. Maintain a smoothed RTT estimate using exponential averaging
3. Set timeout to smoothed RTT plus a safety margin based on RTT variance

The algorithm uses Jacobson's RTT estimation formula [58]:

$$\text{SRTT} \leftarrow (1 - \alpha) \cdot \text{SRTT} + \alpha \cdot \text{RTT}_{\text{sample}} \quad (4.2)$$

$$\text{RTTVAR} \leftarrow (1 - \beta) \cdot \text{RTTVAR} + \beta \cdot |\text{RTT}_{\text{sample}} - \text{SRTT}| \quad (4.3)$$

$$\text{Timeout} \leftarrow \text{SRTT} + 4 \cdot \text{RTTVAR} \quad (4.4)$$

where $\text{RTT}_{\text{sample}}$ is a single observed round-trip time, SRTT (Smoothed Round-Trip Time) is the exponentially weighted moving average of past samples, RTTVAR (Round-Trip Time Variation) tracks the mean deviation, and $\alpha = 1/8$ and $\beta = 1/4$ are the smoothing factors.

Upper and lower bounds prevent extreme values. The minimum timeout ensures nodes have time to process messages. The maximum prevents indefinite waiting when the network is partitioned.

4.4.1 Connection to Failure Detector Theory

The adaptive timeout mechanism is grounded in failure detector theory. As discussed in Section 3.6, Chandra and Toueg [59] defined completeness and accuracy properties for failure detectors, and the Ω failure detector [60] reduces to eventual leader election. In practice, failure detectors are implemented using heartbeats and timeouts. The challenge is balancing fast detection (low timeout) against avoiding false suspicions (high timeout). Adaptive-PraSLE's Jacobson-style timeout mechanism addresses this trade-off by adjusting timeouts based on observed network conditions.

4.5 Protocol and Application Concerns

Leader election serves applications, but the protocol should not dictate application behavior. Adaptive-PraSLE separates concerns into two layers.

4.5.1 Protocol Layer

The protocol layer handles:

- Score computation and propagation
- Timeout management
- Convergence detection
- Failure detection (heartbeat monitoring)
- Handover coordination

These mechanisms are independent of the application. The protocol layer is responsible for determining which node is the current leader; the application layer defines how that leader role is used.

4.5.2 Application Layer

The application layer handles:

- Defining the leader role (coordinator, aggregator, relay, etc.)
- Implementing leader-specific functionality
- Deciding when leader rotation should happen (beyond energy thresholds)
- Integrating with domain-specific systems

For example, in a sensor network, the application might configure frequent rotation to distribute energy consumption. In a robotic swarm, the application might prefer stable leadership even at higher energy cost.

4.5.3 Configuration Interface

The separation is enforced through a configuration interface:

- **Scoring weights:** Application specifies relative importance of factors
- **Rotation thresholds:** Application specifies when handover triggers
- **Timeout bounds:** Application specifies acceptable latency range
- **Feature toggles:** Application enables/disables individual mechanisms

4.6 Self-Stabilization Preservation

PraSLE's self-stabilization guarantee [5] ensures that "starting from any arbitrary initial values of $\min_i$ and leader_i , the entire network will eventually elect a unique leader whose identity will be known for each process." The extensions must preserve this property.

4.6.1 Why Extensions Could Break Stabilization

The following risks must be addressed:

1. **Score instability:** If scores change too rapidly, the minimum-finding algorithm might never converge. A node's score must be stable long enough for it to propagate.
2. **Timeout mismatches:** If some nodes use different timeouts, they may process rounds at different rates, violating synchronization assumptions.
3. **Handover cycles:** If two nodes repeatedly hand off leadership to each other, the system oscillates without converging.
4. **Backup list corruption:** If backup lists contain stale or incorrect entries, fast recovery might select the wrong leader.

4.6.2 How Extensions Preserve Stabilization

Each risk is addressed:

Score stability: Scores are computed at the start of each election cycle and held constant throughout. The EWMA smoothing of link quality and energy measurements reduces noise. Within a single election, scores do not change.

Timeout convergence: If a transient fault corrupts a node's timeout to an arbitrary value, the upper and lower bounds defined in Section 4.4 clamp it back into a valid range within one adjustment step. This prevents infinite waiting, but does not by itself synchronize nodes whose timeouts have diverged. Synchronization is restored by the underlying reset cycle: after at most R cycles, all nodes reset their election state and begin a fresh round with timeouts recomputed from current network measurements.

Handover limits: A minimum leadership term prevents immediate re-handover. A node cannot hand off within 30 seconds of becoming leader. This prevents oscillation cycles.

Backup list validation: The backup list is advisory, not authoritative. Fast recovery proposes a candidate, but the full election runs in background and corrects any incorrect selection.

4.6.3 Reset-Cycle Recovery

The original PraSLE [5] broadcasts messages every round, unconditionally. This ensures recovery from any corruption but generates continuous traffic.

Adaptive-PraSLE uses conditional broadcasting: messages are sent only when values change. This reduces overhead but could leave corrupted state undetected. To compensate, periodic reset cycles force all nodes to reinitialize the election:

1. Every R cycles (default: $R = 3$), nodes reset their state
2. All nodes set their score to their own current value
3. All nodes set their leader to themselves
4. A full K -round election runs
5. The network re-converges to the correct leader

The default value $R = 3$ is a conservative choice that balances message efficiency against recovery time. With $R = 3$, the system completes one initial election cycle, operates for one steady-state cycle with conditional broadcasting, and then resets in the third cycle. Setting $R = 2$ would leave no steady-state cycle, negating the benefit of conditional broadcasting entirely. Higher values of R would increase steady-state efficiency but extend the window during which corrupted state could persist undetected. For a 10-node clique topology with $T = 1.0$ s and $K = 2$, the worst-case recovery time is $R \times K \times T = 6$ s. Finding the optimal R for different network conditions and application requirements is identified as future work in Section 7.5.

This mechanism provides the same recovery guarantee as continuous broadcasting but with lower average message rate.

4.7 Platform Choice: Contiki-NG and Cooja

The implementation uses Contiki-NG [24] and the Cooja simulator [28], described in Section 2.6. Contiki-NG's Energest module [19] provides the software-based energy accounting needed for energy-aware scoring, and its lightweight protothread model [27] fits the memory constraints of IoT devices. Cooja's deterministic simulation enables reproducible cross-algorithm comparison, with programmable fault injection for systematic fault tolerance testing. Simulation limitations (simplified radio models, timing approximation, energy model approximation) are accepted trade-offs for the scalability and controllability that simulation provides over hardware testbeds.

4.8 Evaluation Methodology: Two Parameter Cases

Comparing leader election algorithms fairly is challenging because each algorithm was designed with different assumptions and tuned for different scenarios. To address this, the evaluation framework supports two distinct parameter configurations.

4.8.1 Case 1: Original Paper Parameters

In Case 1, each algorithm uses the timing parameters specified in its original publication:

- **Bully/Ring:** ALIVE_INTERVAL = 2s, COORDINATOR_TIMEOUT = 4s (based on Garcia-Molina's model [1] where T is the upper bound for message delivery)
- **PraSLE/Adaptive-PraSLE:** T = 1.0s (from Conard and Ebneenasir's Algorithm 1 [5], Line 8: "T := 1.0")

This configuration is useful for:

- Validating implementations against published benchmarks
- Comparing performance under each algorithm's "optimal" configuration
- Reproducing results from the original papers

4.8.2 Case 2: Standardized Parameters

In Case 2, all algorithms use identical timing parameters to enable fair cross-algorithm comparison:

- **All algorithms:** Heartbeat/Round period T = 2.0s, Startup delay max = 2.0s
- **Bully/Ring:** COORDINATOR_TIMEOUT = 4.0s (unchanged)

In Case 1, PraSLE broadcasts twice as frequently (T=1s) as Bully/Ring (heartbeat=2s). This gives PraSLE faster convergence at the cost of higher message overhead. Standardizing T=2s for all algorithms in Case 2 enables direct comparison of:

- Message efficiency: How many messages does each algorithm need to converge?
- Convergence efficiency: Given equal broadcast rates, which algorithm converges faster?
- Fault recovery: Which algorithm recovers faster from leader failures?

4.8.3 Case Selection

Case 1 evaluates how each algorithm performs with the parameters specified in its original publication. Case 2 evaluates algorithmic efficiency under equal broadcast rates, isolating algorithm design from parameter tuning.

The evaluation in Chapter 6 uses Case 2, as standardized timing parameters enable fair cross-algorithm comparison by ensuring that performance differences reflect algorithmic design rather than parameter tuning.

4.9 Summary

Adaptive-PraSLE extends PraSLE with five mechanisms: energy-aware scoring, link-quality-aware scoring, a composite scoring function, controlled leader rotation, and backup leader tracking. An adaptive timeout mechanism replaces fixed intervals. The extensions are modular and preserve PraSLE's self-stabilization guarantee.

The design separates protocol concerns (election, convergence, failure detection) from application concerns (what the leader does, when to rotate). This allows the same protocol to serve different applications with different priorities.

The evaluation uses standardized timing parameters (Case 2), where all algorithms use identical broadcast intervals ($T = 2s$) to enable fair cross-algorithm comparison.

The implementation uses Contiki-NG for its energy instrumentation and IoT focus, and the evaluation uses Cooja for reproducible and controlled simulation. The next chapter describes the implementation in detail: system architecture, algorithm design, composite scoring, adaptive mechanisms, state machine, message formats, communication design, and integration with Contiki-NG.

5 Implementation

This chapter presents the technical implementation of Adaptive-PraSLE. The previous chapter described the conceptual approach: the five extensions, adaptive timeouts, and how self-stabilization is preserved. This chapter covers the system architecture, algorithm details, data structures, message formats, and Contiki-NG integration.

5.1 System Architecture

Figure 5.1 illustrates the high-level architecture of an Adaptive-PraSLE node. The system is organized into four layers: monitoring, scoring, election, and communication.

5.1.1 Monitoring Layer

The monitoring layer collects context information from the node and its environment. Four monitors run periodically: an **energy monitor** that tracks remaining battery capacity via Contiki-NG's Energest module [19] using radio duty cycle as a proxy; a **link quality monitor** that maintains per-neighbor RSSI, LQI, and ETX metrics smoothed via EWMA; a **connectivity monitor** that tracks valid neighbor count and reachability; and a **CPU monitor** that measures processor utilization using Energest's CPU time counters.

5.1.2 Scoring Engine

The scoring engine combines monitoring inputs into a single composite score (weighted sum of four normalized components, lower is better) that determines leader eligibility. Section 5.3 provides the detailed algorithm.

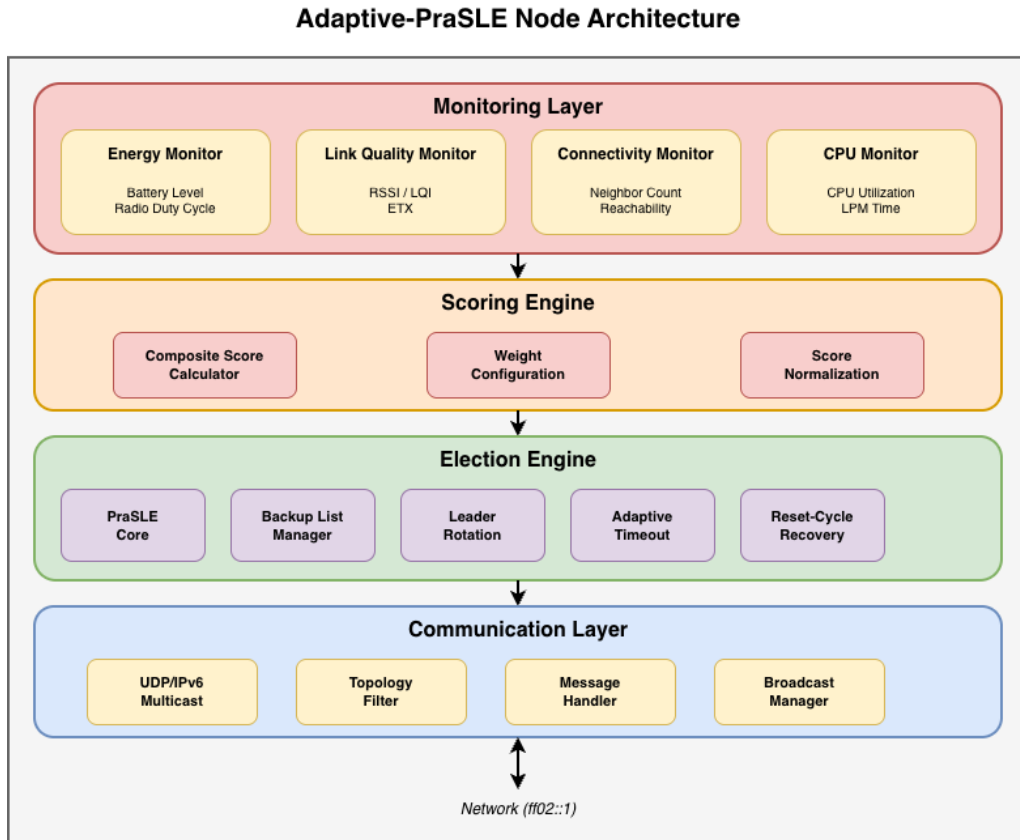


Figure 5.1: Adaptive-PraSLE node architecture showing the four-layer design: monitoring collects context information, scoring computes composite rankings, election manages the PraSLE algorithm with extensions, and communication handles message exchange.

5.1.3 Election Engine

The election engine implements PraSLE’s round-based minimum-finding algorithm with the five extensions described in Chapter 4: backup list tracking for fast recovery, controlled leader rotation, adaptive timeout adjustment, and periodic reset-cycle recovery to maintain self-stabilization.

5.1.4 Communication Layer

The communication layer handles message exchange using UDP over IPv6 with link-local multicast. All election messages are broadcast to the all-nodes multicast address (ff02::1), with topology enforcement performed through receiver-side filtering.

5.2 Algorithm Design

Adaptive-PraSLE extends the PraSLE algorithm while preserving its self-stabilization guarantees. This section describes the core algorithm and its extensions.

5.2.1 PraSLE Foundation

PraSLE operates in rounds, with each round consisting of a receive phase and an update phase. Each node maintains a pair $(\min_i, \text{leader}_i)$ representing the best known score and the corresponding leader identity. The algorithm uses lexicographic comparison:

$$(\min_1, \text{leader}_1) < (\min_2, \text{leader}_2) \iff (\min_1 < \min_2) \vee ((\min_1 = \min_2) \wedge (\text{leader}_1 < \text{leader}_2)) \quad (5.1)$$

Algorithm 2 presents the Adaptive-PraSLE election procedure. The two key subroutines, `GetCompositeScore` and `GetAdaptiveTimeout`, are detailed in Sections 5.3 and 5.4, respectively.

Algorithm 2 Adaptive-PraSLE Election Round

```

1: Input:  $K$  (rounds),  $T$  (timeout), neighbors list
2: Output: Elected leader identity
3: round  $\leftarrow K + 1$ 
4:  $\min_i \leftarrow N + 1$  ▷ Null value
5: temp_min_i  $\leftarrow$  GetCompositeScore
6: leader_i  $\leftarrow i$ 
7: temp_leader_i  $\leftarrow i$ 
8: while round  $> 0$  do
9:   round  $\leftarrow$  round  $- 1$ 
10:  if round  $< K + 1$  then ▷ Skip receive on first round
11:    timeout  $\leftarrow$  GetAdaptiveTimeout
12:    wait timeout and receive messages from neighbors
13:    for all received  $(\min_j, \text{leader}_j)$  do
14:      if  $(\min_j, \text{leader}_j) < (\text{temp\_min}_i, \text{temp\_leader}_i)$  then
15:        temp_min_i  $\leftarrow \min_j$ 
16:        temp_leader_i  $\leftarrow \text{leader}_j$ 
17:      end if
18:    end for
19:  end if
20:  if  $(\text{temp\_min}_i, \text{temp\_leader}_i) < (\min_i, \text{leader}_i)$  then
21:    min_i  $\leftarrow \text{temp\_min}_i$ 
22:    leader_i  $\leftarrow \text{temp\_leader}_i$ 
23:    BroadcastToNeighbors(min_i, leader_i)
24:  end if
25: end while
26: return leader_i

```

5.2.2 Key Modifications from PraSLE

Adaptive-PraSLE makes three key modifications to the base PraSLE algorithm:

1. **Composite scoring:** The ranking value is computed dynamically from multiple context factors rather than using static node identifiers.

2. **Conditional broadcasting:** Messages are sent only when values change, rather than unconditionally every round. Self-stabilization is maintained through periodic reset cycles.
3. **Adaptive timeouts:** The timeout value T is dynamically adjusted based on observed network latency rather than being fixed.

5.2.3 Election Rounds and Cycles

The election process in Adaptive-PraSLE is organized hierarchically into rounds and cycles. This structure determines how the algorithm achieves convergence while maintaining self-stabilization. Figure 5.2 illustrates the complete timing structure.

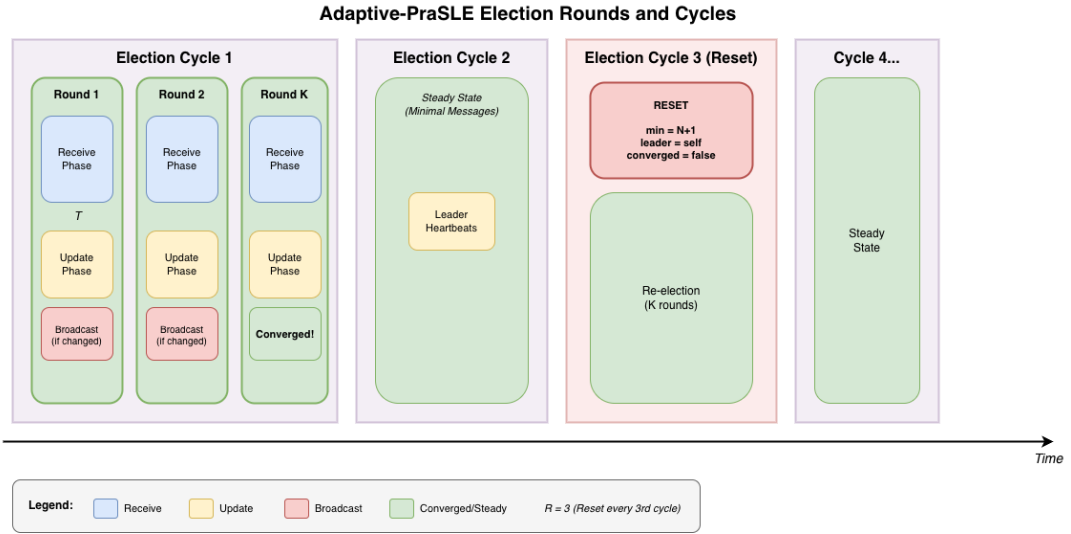


Figure 5.2: Structure of Adaptive-PraSLE election rounds and cycles. Each round consists of a receive phase (duration T) and an update phase. An election cycle comprises K rounds. In continuous operation mode, every third cycle triggers a reset to clear potentially stale leader values.

Round Structure

Each election round consists of two phases:

1. **Receive Phase (duration T):** The node listens for messages from neighbors. During this phase, incoming (min, leader) pairs are collected and compared against the node's current temporary values. The timeout T determines how long the node waits before proceeding. Setting it too short risks missing messages, while setting it too long delays convergence.
2. **Update Phase:** After the receive phase completes, the node updates its state. If any received pair is better (lexicographically smaller) than the current temporary values, the node adopts the new values. If the values changed, the node broadcasts its updated pair to neighbors.

The key optimization in Adaptive-PraSLE is *conditional broadcasting*: messages are sent only when values change, not unconditionally every round. This reduces message overhead significantly in stable networks while still propagating changes quickly when they occur.

Election Cycle

An election cycle consists of K consecutive rounds, where K is the propagation parameter. After K rounds, if the algorithm has converged, all nodes agree on the same leader. The value of K must be at least equal to the network diameter to ensure the best score can propagate from any node to all other nodes.

During the first election cycle (Cycle 1 in Figure 5.2), nodes actively exchange messages to find the minimum score. Once convergence is reached at the end of Round K , the system enters a steady state.

Steady State Operation

In the steady state (Cycle 2 in Figure 5.2), message traffic is minimal:

- The elected leader sends periodic heartbeat messages to signal liveness
- Followers monitor for heartbeats and maintain their backup lists
- No election messages are exchanged since all nodes already agree

In this mode, the system has converged and maintains the leadership relationship with minimal overhead.

Reset Cycle Mechanism

To maintain self-stabilization guarantees, every R -th cycle (default $R = 3$) triggers a reset. This value provides one steady-state cycle between resets, balancing message efficiency against recovery time (see Chapter 4 for the justification of $R = 3$). During a reset cycle (Cycle 3 in Figure 5.2):

1. All nodes reset their state: $\text{min} \leftarrow N + 1$, $\text{leader} \leftarrow \text{self}$, $\text{converged} \leftarrow \text{false}$
2. A fresh election runs for K rounds
3. The network re-converges (typically to the same leader if conditions are stable)

This periodic reset serves two purposes:

- **Fault recovery:** If the system entered an inconsistent state due to message loss or transient failures, the reset clears stale values and allows recovery.
- **Adaptation:** If network conditions have changed (e.g., a node's energy dropped significantly), the reset allows the system to elect a new, more suitable leader.

After the reset cycle completes, the system returns to steady state operation (Cycle 4 and beyond), and the pattern repeats.

Propagation Parameter K

The parameter K must be at least equal to the network diameter to guarantee that the best score propagates to all nodes. Table 5.1 shows the topology-aware K values used in the implementation.

Table 5.1: Topology-aware K parameter selection

Topology	K Formula	K (N=10)
Clique (fully connected)	$K = 2$	2
Ring	$K = \lceil (N + 1)/2 \rceil$	5
Line	$K = N$	10
Mesh (square grid)	$K \approx 2\sqrt{N}$	6

5.3 Composite Scoring Function

The scoring function determines leader eligibility. PraSLE uses static node identifiers for ranking, which does not account for runtime conditions such as energy depletion or link degradation. Adaptive-PraSLE instead computes a composite score from four factors: energy, link quality, connectivity, and CPU availability. Lower scores indicate better candidates.

5.3.1 Score Components

The composite score is computed as:

$$\text{Score} = w_E \cdot S_E + w_L \cdot S_L + w_C \cdot S_C + w_P \cdot S_P \quad (5.2)$$

where $w_E = 0.30$, $w_L = 0.30$, $w_C = 0.20$, $w_P = 0.20$ are the component weights, and S_E , S_L , S_C , S_P are the normalized scores for energy, link quality, connectivity, and CPU respectively.

Figure 5.3 illustrates the scoring components and their weights.

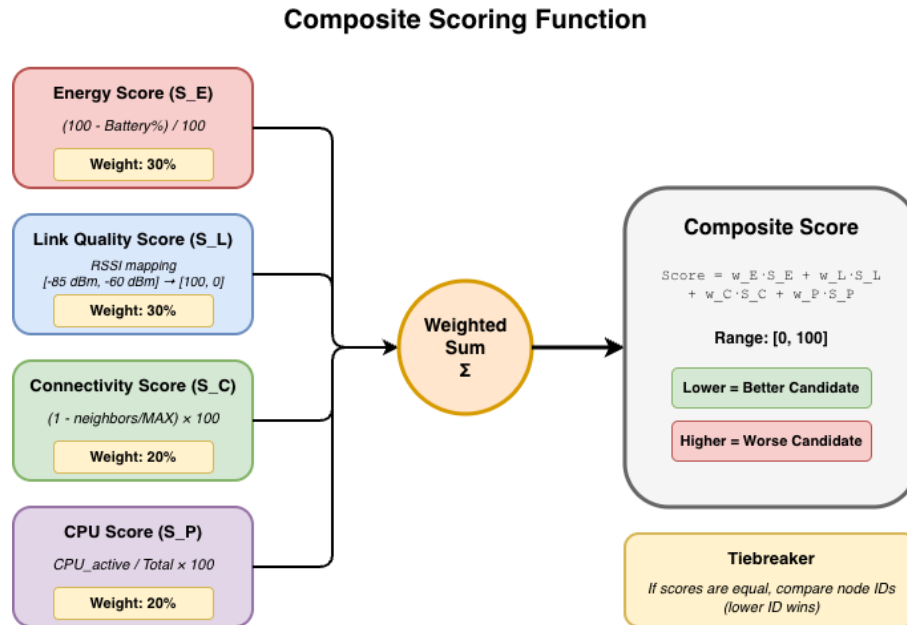


Figure 5.3: Composite scoring function components. Each component contributes a weighted portion to the final score. Lower scores indicate better leader candidates.

Energy Score (S_E)

The energy score penalizes nodes with depleted batteries:

$$S_E = \frac{100 - E_{\text{remaining}}}{100} \times 100 \quad (5.3)$$

where $E_{\text{remaining}}$ is the estimated remaining battery percentage (0–100). A node with 100% battery receives $S_E = 0$; a node with 0% battery receives $S_E = 100$.

Energy estimation uses the radio duty cycle as a proxy:

$$\text{duty_cycle} = \frac{t_{\text{TX}} + t_{\text{RX}}}{t_{\text{total}}} \times 100 \quad (5.4)$$

The battery level is then decremented proportionally to the duty cycle over each measurement interval.

Link Quality Score (S_L)

The link quality score penalizes nodes with poor connectivity to neighbors. The thresholds -60 dBm and -85 dBm correspond to the connected and transitional regions identified in empirical studies of IEEE 802.15.4 link quality [20, 23]: above -60 dBm, packet reception is near-perfect; below -85 dBm, links become unreliable. Both thresholds are configurable (Table 5.3).

$$S_L = \begin{cases} 0 & \text{if } \text{RSSI}_{\text{avg}} \geq -60 \text{ dBm} \\ 100 & \text{if } \text{RSSI}_{\text{avg}} \leq -85 \text{ dBm} \\ 100 - \frac{(\text{RSSI}_{\text{avg}} - (-85)) \times 100}{25} & \text{otherwise} \end{cases} \quad (5.5)$$

The raw link score is modulated by a freshness confidence factor:

$$S_L^{\text{effective}} = S_L \times \frac{100}{f_{\text{confidence}}} \quad (5.6)$$

where $f_{\text{confidence}}$ ranges from 50% (stale data, freshness < 4) to 100% (fresh data, freshness ≥ 16). This penalizes nodes whose link quality information is uncertain.

Connectivity Score (S_C)

The connectivity score penalizes nodes with few neighbors:

$$S_C = \left(1 - \frac{N_{\text{neighbors}}}{\text{MAX_NEIGHBORS}} \right) \times 100 \quad (5.7)$$

A node with the maximum number of neighbors receives $S_C = 0$; an isolated node receives $S_C = 100$.

CPU Score (S_P)

The CPU score penalizes nodes with high processor utilization:

$$S_P = \frac{t_{\text{CPU_active}}}{t_{\text{total}}} \times 100 \quad (5.8)$$

Nodes with lower CPU usage have more available capacity for leader coordination tasks.

5.3.2 Tiebreaking

When two nodes have identical composite scores, the lexicographic comparison falls back to node IDs:

$$(\text{score}_1, \text{id}_1) < (\text{score}_2, \text{id}_2) \iff (\text{score}_1 < \text{score}_2) \vee ((\text{score}_1 = \text{score}_2) \wedge (\text{id}_1 < \text{id}_2)) \quad (5.9)$$

This ensures deterministic leader selection even when context factors are equal.

5.4 Adaptive Mechanisms

Adaptive-PraSLE incorporates three adaptive mechanisms that respond to changing network conditions.

5.4.1 Adaptive Timeout Management

Fixed timeouts present a trade-off: values that are too short cause nodes to miss messages from slower neighbors, leading to incorrect elections, while values that are too long delay convergence. Adaptive-PraSLE uses Jacobson's algorithm [58] to dynamically adjust timeouts based on observed network latency.

The algorithm maintains a smoothed RTT estimate (SRTT) and RTT variance (RTTVAR):

$$\text{SRTT} \leftarrow \frac{7}{8} \cdot \text{SRTT} + \frac{1}{8} \cdot \text{RTT}_{\text{sample}} \quad (5.10)$$

$$\text{RTTVAR} \leftarrow \frac{3}{4} \cdot \text{RTTVAR} + \frac{1}{4} \cdot |\text{RTT}_{\text{sample}} - \text{SRTT}| \quad (5.11)$$

$$\text{RTO} \leftarrow \text{SRTT} + 4 \cdot \text{RTTVAR} \quad (5.12)$$

The retransmission timeout (RTO) is clamped to the range $[\text{RTO}_{\min}, \text{RTO}_{\max}]$ to prevent extreme values:

$$\text{Timeout} = \text{clamp}(\text{RTO} \times \text{safety_margin}, \text{RTO}_{\min}, \text{RTO}_{\max}) \quad (5.13)$$

Default parameters are: $SRTT_{init} = 100$ ms, $RTTVAR_{init} = 25$ ms, $RTO_{min} = 150$ ms, $RTO_{max} = 5000$ ms, $safety_margin = 1.2$.

5.4.2 Controlled Leader Rotation

When a leader's energy falls below a critical threshold, it proactively initiates handover to prevent unexpected failure. Figure 5.4 illustrates the handover protocol.

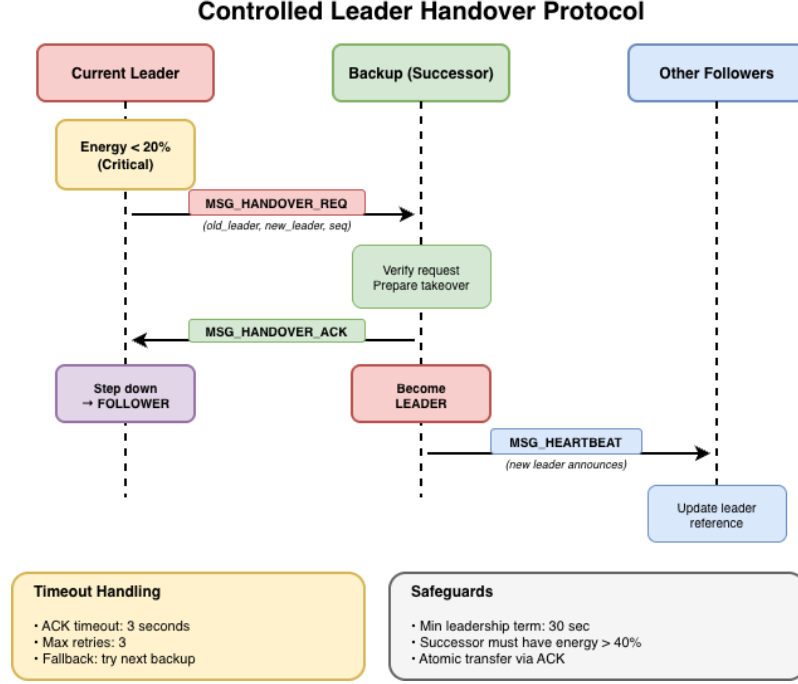


Figure 5.4: Controlled leader handover protocol. The leader initiates handover when energy falls below a critical threshold. A request-acknowledgment exchange coordinates the transfer, though a brief overlap period exists during which both the old and new leader may consider themselves leader until the acknowledgment is received and propagated.

The handover protocol includes several safeguards:

- **Minimum leadership term:** A node must remain leader for at least 30 seconds before initiating handover, preventing oscillation between candidates with similar scores.
- **Acknowledgment timeout:** If the successor does not acknowledge within 3 seconds, the request is retried up to 3 times.
- **Backup fallback:** If the primary successor fails to respond, the next candidate from the backup list is tried.

5.4.3 Reset-Cycle Recovery

To maintain self-stabilization while reducing message overhead, Adaptive-PraSLE uses periodic reset cycles. Every R election cycles (default: $R = 3$), all nodes reset their $(min_i, leader_i)$ values to initial state, as described in Section 4. This mechanism provides the same recovery guarantees as PraSLE's continuous broadcasting mode

but with lower message overhead. Between resets, messages are sent only when values change.

5.5 State Machine

Figure 5.5 shows the state machine governing Adaptive-PraSLE node behavior.

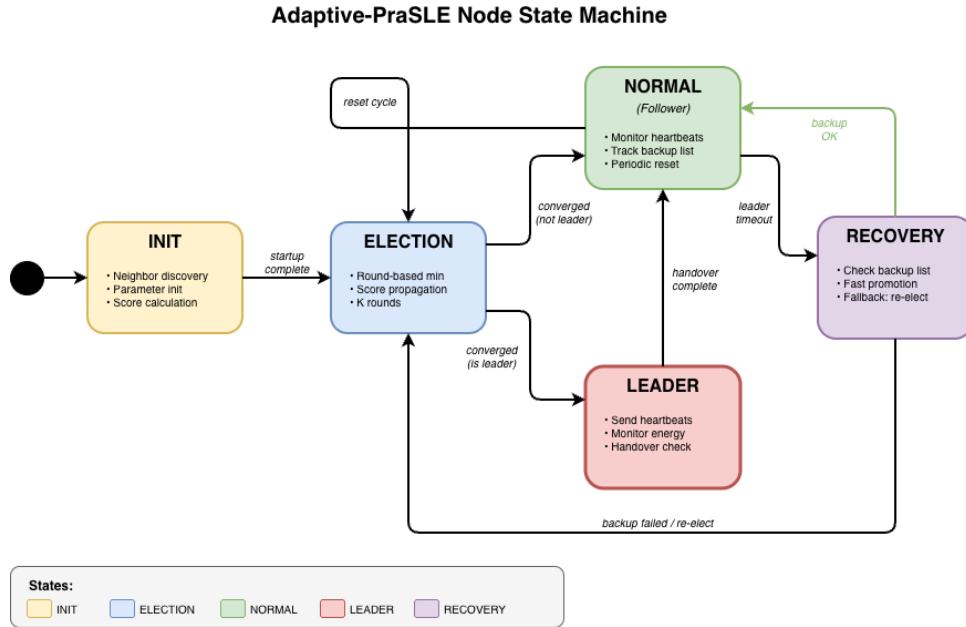


Figure 5.5: Adaptive-PraSLE state machine. Nodes transition between states based on election progress, convergence, leader failure detection, and controlled handover.

The states are:

- **INIT:** Initial state after node startup. Neighbor discovery and parameter initialization occur here.
- **ELECTION:** Active election in progress. The node participates in round-based score propagation.
- **NORMAL:** Election converged and this node is a follower. Monitors for leader failure and periodic resets.
- **LEADER:** Election converged and this node is the leader. Sends heartbeats and monitors for handover triggers.
- **RECOVERY:** Leader failure detected, attempting fast recovery from backup list.

5.6 Message Formats

Adaptive-PraSLE uses four message types, defined in Table 5.2.

Table 5.2: Adaptive-PraSLE message types

Type	Name	Size	Purpose
0	MSG_ELECTION	14 bytes	Score propagation during election
1	MSG_HEARTBEAT	14 bytes	Leader liveness signal
2	MSG_HANOVER_REQ	8 bytes	Initiate controlled rotation
3	MSG_HANOVER_ACK	8 bytes	Confirm handover acceptance

5.6.1 Election Message Format

The election message extends PraSLE's format with additional context fields:

```

1 typedef struct {
2     uint8_t msg_type;           // Message type (0 = ELECTION)
3     uint8_t padding;           // Alignment padding
4     uint16_t min_value;        // Composite ranking score
5     uint16_t leader_id;       // Current leader ID
6     uint16_t sender_id;       // Sender node ID
7     uint8_t energy_level;      // Sender's battery % (0-100)
8     int8_t avg_rssi;           // Sender's average RSSI
9     uint8_t neighbor_count;    // Number of valid neighbors
10    uint8_t flags;              // Status flags
11    uint16_t seq_num;           // Sequence number
12 } __attribute__((packed)) adaptive_prasle_msg_t;

```

Listing 5.1: Election message structure

The status flags encode important node state:

- FLAG_IS_LEADER (0x01): Sender is the current leader
- FLAG_LOW_ENERGY (0x02): Sender has low energy
- FLAG_HANOVER_PENDING (0x04): Handover in progress

5.6.2 Handover Message Format

Handover messages are smaller since they only need to identify the parties involved:

```

1 typedef struct {
2     uint8_t msg_type;           // MSG_HANOVER_REQ or MSG_HANOVER_ACK
3     uint8_t padding;           // Alignment padding
4     uint16_t old_leader;       // Current leader ID
5     uint16_t new_leader;      // Proposed new leader
6     uint16_t seq_num;         // Handover sequence number
7 } __attribute__((packed)) handover_msg_t;

```

Listing 5.2: Handover message structure

5.7 Communication Design

5.7.1 UDP/IPv6 Multicast

All messages are sent via UDP to the IPv6 link-local all-nodes multicast address `ff02::1`. This design choice offers several advantages:

1. **Energy efficiency:** One multicast transmission reaches all neighbors with a single radio operation. Unicast to k neighbors would require k transmissions.
2. **Simplicity:** No routing tables or address resolution required.
3. **Topology independence:** The same code works for any logical topology.

5.7.2 Topology Enforcement

Since multicast reaches all nodes, logical topology constraints are enforced at the receiver:

```

1 static void handle_message(const uint8_t *data, uint16_t len) {
2     adaptive_prasle_msg_t *msg = (adaptive_prasle_msg_t *)data;
3
4     // Filter self-messages (broadcast echo)
5     if (msg->sender_id == my_node_id) return;
6
7     // Filter non-neighbors (topology enforcement)
8     if (!is_logical_neighbor(msg->sender_id)) return;
9
10    // Process message from valid neighbor...
11 }
```

Listing 5.3: Receiver-side topology filtering

The `is_logical_neighbor()` function checks a pre-computed neighbor list based on the configured topology (ring, line, mesh, or clique).

5.8 Integration with Contiki-NG

5.8.1 Process Model

Adaptive-PraSLE is implemented as a single Contiki-NG protothread [27] process. On startup, the process initializes the neighbor table and registers a UDP connection for message exchange. The election loop uses an etimer whose duration is set by the adaptive timeout mechanism. Each timer expiration triggers one election round. Incoming messages are handled asynchronously via a UDP receive callback.

5.8.2 Energest Integration

Energy monitoring uses Contiki-NG's Energest module:

```

1 static uint8_t get_energy_level(void) {
2     energest_flush();
3
4     uint64_t tx_time = energest_type_time(ENERGEST_TYPE_TRANSMIT);
5     uint64_t rx_time = energest_type_time(ENERGEST_TYPE_LISTEN);
6     uint64_t total = energest_type_time(ENERGEST_TYPE_CPU) +
7                     energest_type_time(ENERGEST_TYPE_LPM);
8
9     // Calculate duty cycle and drain battery
10    uint64_t active = tx_time + rx_time;
11    uint8_t duty_cycle = (active * 100) / total;
12
13    // Drain proportional to duty cycle
14    my_energy_level -= (duty_cycle * DRAIN_RATE) / 100;
15
16    return my_energy_level;
17 }

```

Listing 5.4: Energy estimation using Energest

5.8.3 Link Quality Metrics

Link quality is obtained from the radio driver after each received packet:

```

1 radio_value_t rssi_val, lqi_val;
2 NETSTACK_RADIO.get_value(RADIO_PARAM_LAST_RSSI, &rssi_val);
3 NETSTACK_RADIO.get_value(RADIO_PARAM_LAST_LINK_QUALITY, &lqi_val);
4
5 // Update neighbor with EWMA smoothing
6 nbr->rssi = (nbr->rssi * 7 + rssi_val) / 8;
7 nbr->lqi = (nbr->lqi * 7 + lqi_val) / 8;

```

Listing 5.5: Link quality measurement

5.9 Configuration and Feature Toggles

Adaptive-PraSLE provides extensive configuration options through compile-time constants. Table 5.3 lists the parameters.

The feature toggles enable ablation studies by selectively disabling individual mechanisms. The parameter configuration described in Section 4.8 is selected at compile time via the `PARAM_CASE` build flag, and the Makefile propagates this to all components.

Table 5.3: Key configuration parameters

Parameter	Default	Description
<i>Algorithm Parameters</i>		
K_ROUNDS	topology-dep.	Election rounds (must $\geq$ diameter)
T_SECONDS	1.0 / 0.1	Round duration (normal / fast mode)
MAX_NEIGHBORS	8	Maximum tracked neighbors
N_MAX	100	Maximum network size
<i>Scoring Weights</i>		
ENERGY_WEIGHT	30	Energy component weight (%)
LINK_QUALITY_WEIGHT	30	Link quality weight (%)
CONNECTIVITY_WEIGHT	20	Connectivity weight (%)
CPU_WEIGHT	20	CPU availability weight (%)
<i>Thresholds</i>		
ENERGY_CRITICAL	20%	Trigger handover threshold
ENERGY_LOW	40%	Avoid as new leader threshold
RSSI_GOOD	−60 dBm	Excellent link threshold
RSSI_POOR	−85 dBm	Poor link threshold
<i>Feature Toggles</i>		
ADAPTIVE_ENERGY_AWARE	1	Enable energy scoring
ADAPTIVE_LINK_QUALITY_AWARE	1	Enable link quality scoring
ADAPTIVE_LEADER_ROTATION	1	Enable controlled handover
ADAPTIVE_TIMEOUTS	1	Enable adaptive timeouts
ADAPTIVE_BACKUP_LIST	1	Enable backup list
ADAPTIVE_RESET_CYCLES	1	Enable periodic resets

5.10 Summary

Adaptive-PraSLE extends PraSLE with five mechanisms: energy-aware scoring, link-quality-aware scoring, a composite scoring function, controlled leader rotation, and backup leader tracking. An adaptive timeout mechanism replaces fixed intervals, and reset-cycle recovery preserves self-stabilization while reducing message overhead.

The implementation integrates with Contiki-NG [24]. Energy monitoring uses the Energest module [19]. Link quality is obtained from the radio driver. Communication uses simple-udp with IPv6 multicast. Feature toggles allow each mechanism to be enabled or disabled independently, supporting the ablation studies in Chapter 6.

The design involves trade-offs. Composite scoring adds complexity compared to static ID-based ranking. Conditional broadcasting reduces messages but requires periodic resets to maintain fault tolerance. The handover protocol introduces additional message types not present in PraSLE. Whether these costs are justified is evaluated in Chapter 6.

6 Evaluation

This chapter presents the experimental evaluation of Adaptive-PraSLE and compares it against baseline algorithms. The evaluation examines convergence time, message overhead, fault tolerance, and energy efficiency across different network sizes and topologies.

6.1 Experimental Methodology

6.1.1 Simulation Environment

All experiments are conducted using the Cooja network simulator [28] with Contiki-NG [24]. Simulation was chosen over hardware testbed experimentation because it provides precise control over network topology, deterministic repeatability through configurable random seeds, and programmable fault injection, all of which are necessary for systematic cross-algorithm comparison across 6,400 simulation runs. Cooja’s cycle-accurate MSP430 emulation executes unmodified Contiki-NG binaries, and its Energest integration [19] enables software-based energy accounting without specialized measurement hardware. However, simulation has limitations: Cooja’s Unit Disk Graph radio model simplifies real-world propagation effects such as multipath fading and environmental interference, timing behavior may differ from physical hardware under interrupt-heavy workloads, and the energy model approximates rather than measures actual power consumption. These limitations are discussed further in Section 7 as threats to validity. The platform selection rationale is described in detail in Section 4.7.

6.1.2 Reproducibility and Communication Stack

Each experiment uses a unique Cooja random seed per trial, so that the 100 trials per configuration represent statistically independent network conditions (different interference patterns, timing jitter, and startup sequences). All algorithms use the same IPv6/UDP communication stack: Bully and PraSLE variants use UDP multicast

(ff02::1), while Ring uses unicast UDP. This ensures that measured performance differences reflect algorithmic behavior rather than network-layer variations.

6.1.3 Parameter Cases

The parameter configuration used in all experiments follows Case 2 as described in Section 4.8, where all algorithms use standardized timing parameters ($T = 2s$). This case was chosen over Case 1 (original paper parameters) for the following reasons.

In Case 1, PraSLE uses $T = 1s$ while Bully and Ring use a heartbeat interval of 2s, meaning PraSLE broadcasts twice as frequently. This gives PraSLE a structural advantage in convergence speed that stems from parameter tuning rather than algorithmic design. Standardizing $T = 2s$ for all algorithms in Case 2 equalizes the broadcast rate and ensures that measured differences in convergence time, message overhead, and fault recovery reflect the algorithms' inherent efficiency rather than differences in how aggressively they communicate. Case 2 therefore provides the fairest basis for cross-algorithm comparison, which is the primary goal of this evaluation.

6.1.4 Experiment Types

Four experiment categories evaluate different aspects of algorithm behavior. **Convergence** experiments measure time from startup until all nodes agree on a leader, testing network sizes of 5, 10, 50, and 100 nodes across 100 trials with unique random seeds. **Fault tolerance** experiments measure recovery time after leader crash: the simulation runs until convergence, the leader is terminated, and recovery time is measured until a new leader is elected. **Noise resilience** experiments measure convergence under packet loss at three levels (10%, 30%, and 50% loss). **Network partition** experiments test split-brain behavior by dividing the network into two isolated groups, each of which should independently elect its own leader. Detailed experiment configurations are provided in Appendix C.

With 10 algorithm variants, 4 network sizes, 4 experiment types (including 3 noise levels), and 100 trials per configuration, the full evaluation comprises 6,400 individual simulation runs per parameter case. Each simulation takes between 60 and 180 seconds depending on network size. The complete evaluation for both parameter cases required approximately 25 hours of wall-clock time on a machine equipped with an Apple M4 Pro processor (14 cores) and 48 GB of RAM, using parallel trial execution.

6.1.5 Algorithms Evaluated

The evaluation covers 10 algorithm variants:

Baseline algorithms:

- Bully [1] – classical priority-based election
- Ring [2] – classical ring-based election (Chang-Roberts)

PraSLE variants (4 topologies):

- PraSLE (clique) – fully connected logical topology
- PraSLE-line – linear chain topology
- PraSLE-ring – ring logical topology
- PraSLE-mesh – 2D grid topology

Adaptive-PraSLE variants (4 topologies):

- Adaptive-PraSLE (clique)
- Adaptive-PraSLE-line
- Adaptive-PraSLE-ring
- Adaptive-PraSLE-mesh

6.2 Convergence Time Results

Convergence time measures how long it takes from network startup until all nodes agree on a single leader. Fast convergence is important because nodes cannot coordinate effectively until a leader is established.

Figure 6.1 shows convergence time across all algorithms and network sizes. Each bar represents the mean of 100 trials with unique random seeds; error bars show standard deviation.

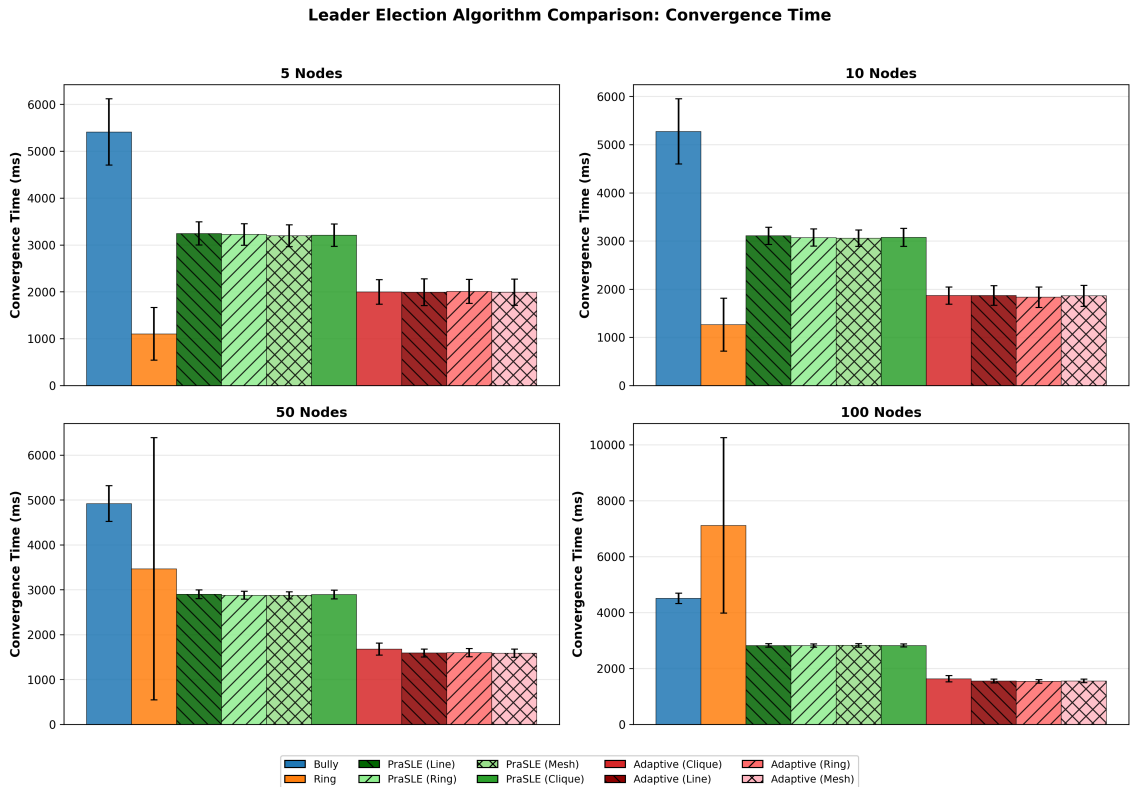


Figure 6.1: Convergence time comparison across network sizes. Lower values are better. PraSLE and Adaptive-PraSLE variants consistently outperform classical algorithms, with the performance gap widening at larger network sizes.

Convergence times in these experiments are dominated by configured timer values rather than by algorithm execution or network propagation delays. In the Cooja simulation environment with ideal link conditions, actual message delivery is on the order of milliseconds. However, each algorithm must wait for its configured timeouts before declaring convergence: Bully waits up to $2T = 4$ seconds (the coordinator timeout from Garcia-Molina’s design [1]), while PraSLE requires $K \times T$ seconds to complete its rounds. Since all algorithms use $T = 2s$ in Case 2, the broadcast rate is equalized, and differences reflect algorithmic efficiency rather than parameter tuning. Additionally, all algorithms include a random startup delay (up to 2 seconds) to prevent synchronized election storms at boot time. These conservative timer values are intentional and provide tolerance for message loss and retransmissions in real 802.15.4 wireless networks. Reducing T would proportionally reduce convergence times. The measured convergence times therefore reflect the safety margins built into the timeout parameters, and the relative differences between algorithms are more meaningful than the absolute values.

6.2.1 Small Networks (5–10 nodes)

At 5 nodes, all algorithms converge within 1–6 seconds. Bully takes 5.4 seconds, reflecting its timeout-based failure detection approach. Ring converges faster at small scale (1.1 seconds) because the token-passing mechanism completes quickly with few nodes. PraSLE variants converge in 3.2 seconds, and Adaptive-PraSLE in 2.0 seconds.

At 10 nodes, the pattern holds with all algorithms maintaining similar performance. Standard PraSLE converges in 3.1 seconds. Adaptive-PraSLE is faster at 1.9 seconds, as conditional broadcasting reduces message overhead even in ideal conditions.

6.2.2 Large Networks (50–100 nodes)

At larger network sizes, Ring’s performance diverges from other algorithms. At 50 nodes, Ring shows a mean convergence time of 3.5 seconds with high variance (standard deviation of 2.9 seconds). At 100 nodes, the mean increases to 7.1 seconds with a standard deviation of 3.1 seconds. The large error bars visible in Figure 6.1 reflect occasional trials where convergence exceeds 10–13 seconds. The sequential message propagation causes this variability: if a message is lost or delayed early in the ring, recovery requires a full ring traversal. While most trials complete quickly, this unpredictability makes Ring less suitable for applications that need consistent convergence guarantees.

Bully maintains relatively stable convergence (4.5–5.4 seconds) because its timeout-based approach does not depend heavily on network size once the highest-priority node responds. Convergence time decreases slightly at larger network sizes, likely because the highest-priority node is found sooner when more nodes participate.

PraSLE variants scale well. Even at 100 nodes, convergence times remain under 3 seconds. Standard PraSLE with clique topology converges in 2.8 seconds; the fundamental $O(K)$ round complexity keeps times bounded regardless of network size.

Adaptive-PraSLE performs even better, converging in 1.6 seconds at 100 nodes. Conditional broadcasting reduces both message overhead and convergence time by avoiding redundant transmissions.

6.2.3 Topology Effects

Each topology restricts both physical radio range and logical neighbor sets. In clique, all nodes are within radio range and process all messages. In line, ring, and mesh, the radio range is configured so that nodes can only physically reach their logical neighbors (e.g., 52 units for line vs. 420 units for clique at 100 nodes). Nodes also filter incoming messages at the application layer, ignoring broadcasts from non-logical-neighbors. The K parameter (rounds per election cycle) is set to the graph diameter of the logical topology:

- **Clique** ($K=2$): Each node has $N-1$ logical neighbors. All broadcasts are processed. The short cycle length (2 rounds) means the election resets frequently.
- **Ring** ($K=\lceil(N+1)/2\rceil$): Each node has exactly 2 logical neighbors (predecessor and successor). The logical ring topology should not be confused with the Ring algorithm (Chang-Roberts); PraSLE-ring still uses broadcast communication.
- **Line** ($K=N$): Each node has 1–2 logical neighbors (left and right along the chain). This gives the longest cycle among all topologies.
- **Mesh** ($K\approx 2\sqrt{N}$): Each node has up to 4 logical neighbors (orthogonal grid connections).

Despite these structural differences, convergence time is topology-independent for standard PraSLE under Case 2 parameters. At 100 nodes, all four topologies converge within 2,818–2,823 milliseconds. Each election cycle lasts T seconds total, divided into K rounds of T/K seconds each. Since $T = 2s$ is the same for all topologies, the total propagation time per cycle is $K \times (T/K) = T$ regardless of topology. The topology choice therefore affects message overhead (discussed in Section 6.3) rather than convergence speed.

For Adaptive-PraSLE, topology has a small effect on convergence because conditional broadcasting interacts with the neighbor structure. In clique, once any node broadcasts the new minimum, all $N-1$ neighbors receive it simultaneously and no further broadcasts are needed. In line, ring, and mesh, changes must propagate hop-by-hop through the logical graph, triggering a re-broadcast at each hop. This leads to slightly faster convergence for non-clique topologies (1,536–1,553 milliseconds at 100 nodes vs. 1,630 milliseconds for clique) but at the cost of higher message overhead (Section 6.3).

In real deployments where the physical network topology constrains which nodes can communicate, the topology choice would have a more direct impact on convergence. The standardized T in Case 2 equalizes convergence by design to enable fair cross-algorithm comparison.

6.3 Message Overhead Results

Message overhead directly impacts energy consumption because radio transmission is the dominant energy cost in IoT devices [15]. Figure 6.2 shows total messages sent during election.

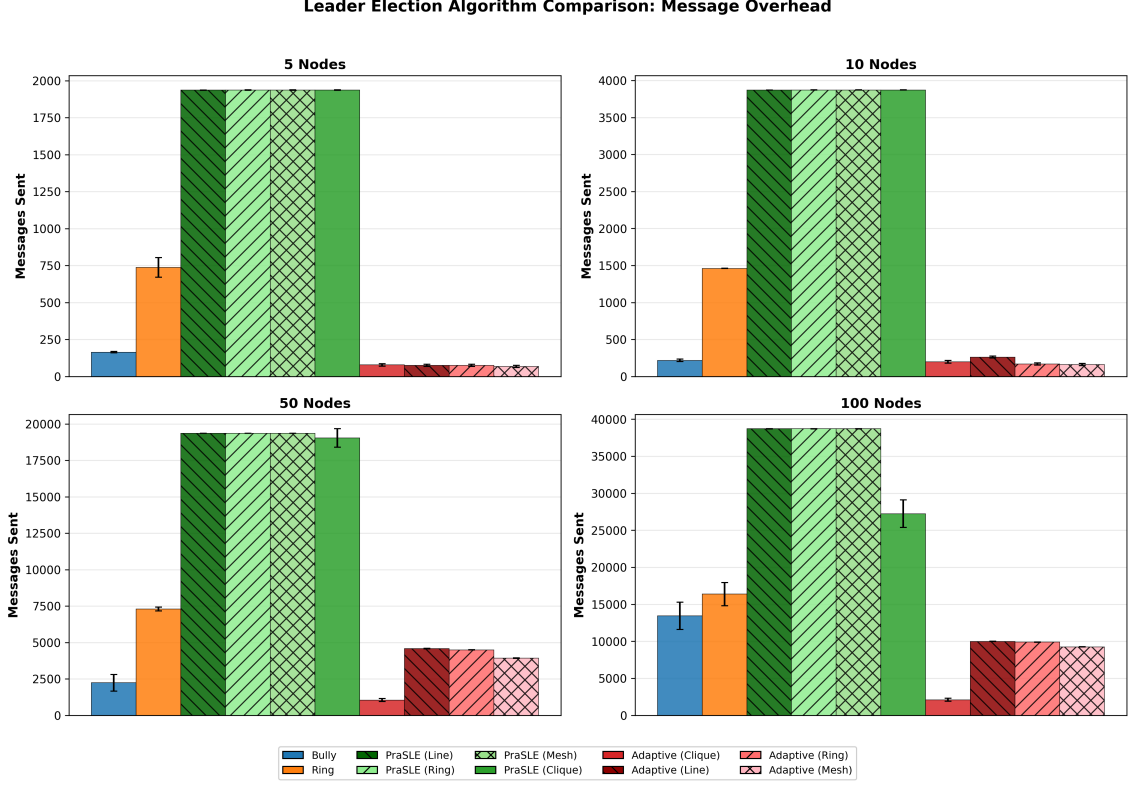


Figure 6.2: Message overhead comparison. Lower values indicate more energy-efficient algorithms. Standard PraSLE generates the highest overhead due to unconditional broadcasting, while Adaptive-PraSLE reduces message count through conditional broadcasting.

6.3.1 Classical Algorithms

Bully generates relatively few messages at small scale (164 at 5 nodes) but scales to 13,400 at 100 nodes. The algorithm’s coordinator announcement and acknowledgment pattern limits message proliferation at small scale, but the overhead grows with network size.

Ring shows moderate message counts at small scale but poor scaling. At 100 nodes, Ring generates 16,400 messages, reflecting the sequential forwarding pattern where each election message visits every node.

6.3.2 Standard PraSLE Message Overhead

Standard PraSLE generates high message counts due to unconditional broadcasting. At 5 nodes, all topologies generate 1,900 messages. At 100 nodes, the line, ring, and mesh topologies generate 38,700 messages each, while the clique topology generates

27,200 messages. The lower clique count at scale reflects its shorter cycle length ($K = 2$ rounds), which means fewer broadcasts accumulate between resets. Regardless, all PraSLE topologies produce higher message overhead than classical algorithms because every node broadcasts unconditionally each round.

6.3.3 Adaptive-PraSLE Efficiency

Adaptive-PraSLE shows much lower message counts than standard PraSLE. At 5 nodes, Adaptive-PraSLE generates only 68–78 messages compared to standard PraSLE’s 1,900, a reduction of over 95%. Conditional broadcasting causes this reduction: messages are sent only when the local (min, leader) pair changes, not unconditionally every round.

At 100 nodes, the topology differences become visible in Adaptive-PraSLE:

- **Clique topology:** 2,100 messages (92% reduction from standard PraSLE clique’s 27,200). All nodes hear each broadcast simultaneously, so once the minimum propagates, no further broadcasts are needed.
- **Line, ring, and mesh topologies:** 9,300–10,000 messages (74% reduction from standard PraSLE’s 38,700). These topologies still require more rounds for values to propagate through the network.

The periodic reset mechanism (every 3 cycles) ensures self-stabilization is maintained while reducing steady-state overhead.

6.4 Fault Tolerance Results

Leader failure is expected in long-running IoT deployments. The fault tolerance experiments measure how quickly algorithms recover after the current leader crashes. Figure 6.3 shows recovery times.

6.4.1 Recovery Mechanisms

Each algorithm family uses a different recovery approach:

Bully relies on timeout-based failure detection. When nodes stop receiving coordinator heartbeats, they initiate a new election. Recovery time is bounded by the timeout value (4 seconds) plus election duration, resulting in recovery times of 7 seconds across all network sizes.

Ring must propagate failure information around the ring and establish a new leader. At small scale (5 nodes), recovery takes 6.4 seconds. At 10 nodes, recovery increases to 11.4 seconds. At larger scale, Ring’s recovery time increases to over 53 seconds at 50 nodes and exceeds 102 seconds at 100 nodes. Ring also fails to recover in 13–21% of trials, depending on network size.

PraSLE recovers through the same round-based mechanism used for initial election. When nodes detect missing heartbeats, they reset their state and re-run the election.

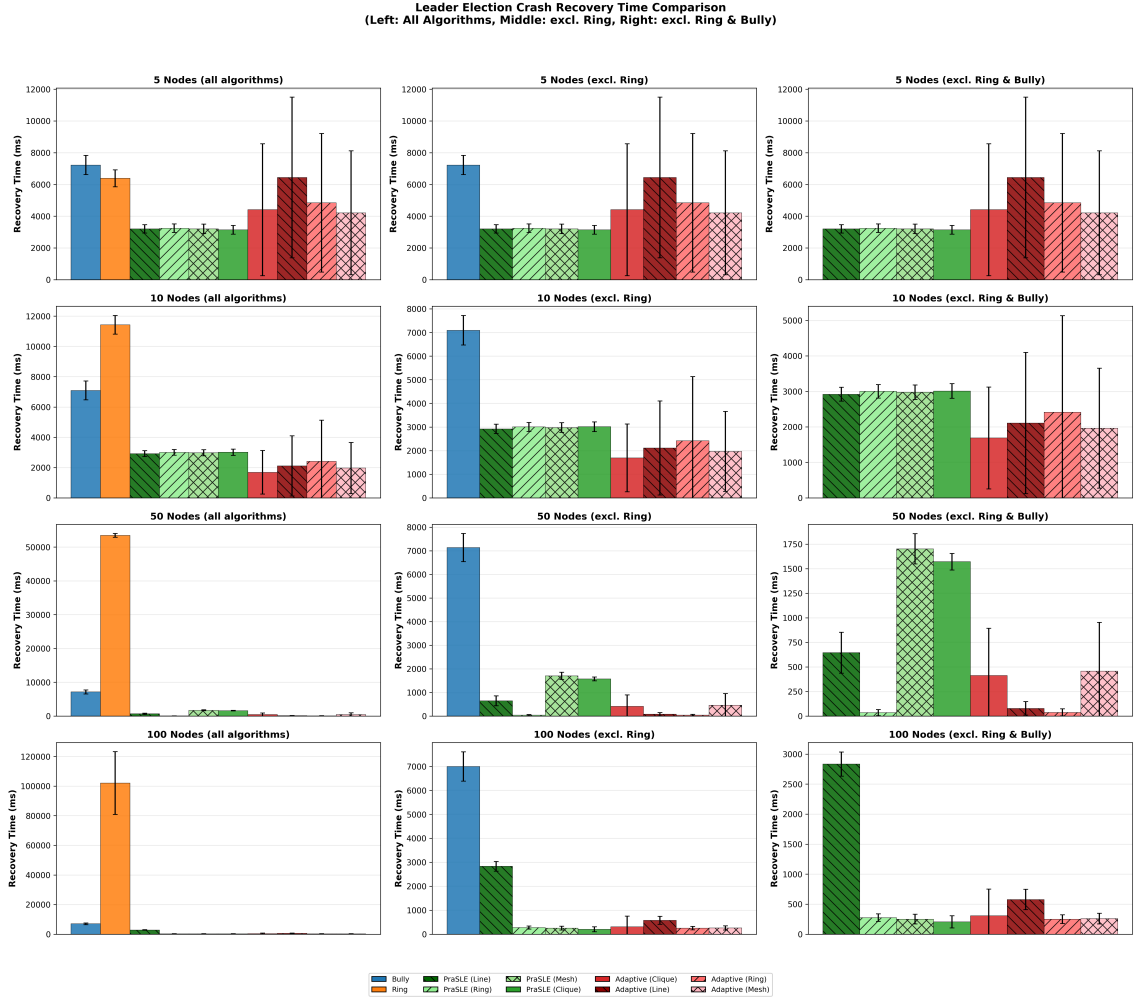


Figure 6.3: Recovery time after leader crash. Lower values indicate faster fault recovery. PraSLE variants show consistent recovery times across network sizes, while Ring shows degraded performance at larger scales.

At small network sizes (5–10 nodes), recovery takes 3 seconds, comparable to the initial convergence time since the full election cycle must complete again. At larger sizes, recovery improves: at 50 nodes, PraSLE-ring recovers in 34 milliseconds, while clique takes 1.6 seconds and mesh 1.7 seconds. At 100 nodes, clique, ring, and mesh topologies all achieve sub-second recovery (206–276 milliseconds). The line topology is an exception at 100 nodes (2.8 seconds) due to its slower propagation pattern. At larger sizes, the reset-cycle mechanism re-establishes consensus quickly because many nodes participate simultaneously.

Adaptive-PraSLE adds backup list recovery. When the leader fails, followers can promote a known backup without full re-election. At small scale (5 nodes), Adaptive-PraSLE recovery takes 4–6 seconds with high variance, slightly slower than standard PraSLE due to additional state management overhead. At 10 nodes, Adaptive-PraSLE becomes competitive, with the clique topology recovering in 1.7 seconds. At 50–100 nodes, most Adaptive-PraSLE topologies recover in under a second (37–458 milliseconds). The ring topology is fastest (37 milliseconds at 50 nodes, 250 milliseconds at 100 nodes), while the line topology at 100 nodes takes 578 milliseconds. The backup list provides greater value for anticipated handovers (controlled leader rotation) rather than sudden crashes. Note that these experiments do not isolate

the individual contributions of the backup list and the reset-cycle mechanism to recovery performance; both mechanisms operate simultaneously in Adaptive-PraSLE. An ablation study disabling the backup list while keeping other extensions active would clarify the backup list's independent contribution, but this is left for future work.

6.4.2 Scalability of Recovery

Unlike convergence time, which remains relatively constant, recovery time actually *improves* with network size for PraSLE variants. At 5–10 nodes, recovery takes 3 seconds (comparable to initial convergence). At 50–100 nodes, most topologies recover in under 1 second, with PraSLE-ring achieving recovery in 34 milliseconds at 50 nodes. Larger networks have more nodes simultaneously detecting the failure and participating in the reset-cycle, which leads to faster re-convergence.

Ring shows the opposite trend: recovery degrades from 6.4 seconds at 5 nodes to over 102 seconds at 100 nodes. Bully remains constant at 7 seconds regardless of network size, bounded by its timeout-based detection.

For applications that need high availability, the difference matters. A 102-second recovery window exceeds acceptable bounds for coordinated robotic systems or real-time sensor networks, whereas PraSLE's sub-second recovery at scale meets tight availability requirements.

6.5 Noise Resilience Results

Real IoT deployments experience packet loss from interference, collisions, and environmental factors. The noise experiments evaluate convergence under controlled packet loss rates. Figures 6.4, 6.5, and 6.6 show results at 10%, 30%, and 50% packet loss.

6.5.1 Mild Interference (10% Loss)

At 10% packet loss, PraSLE and Adaptive-PraSLE variants show virtually no degradation: convergence times change by less than 1% compared to ideal conditions, well within statistical variance. Bully also remains stable, with convergence dominated by its fixed timeout rather than message delivery. Ring, however, is hit much harder: at 50 nodes, convergence increases from 3.5 seconds to 58 seconds (a $17\times$ increase), and at 100 nodes from 7.1 seconds to 117 seconds (a $16\times$ increase). Several Ring trials at these sizes failed to converge entirely (recorded as NA): 36% at 50 nodes and 24% at 100 nodes. Even mild packet loss disrupts Ring's sequential propagation pattern, where a single lost message delays the entire election cycle.

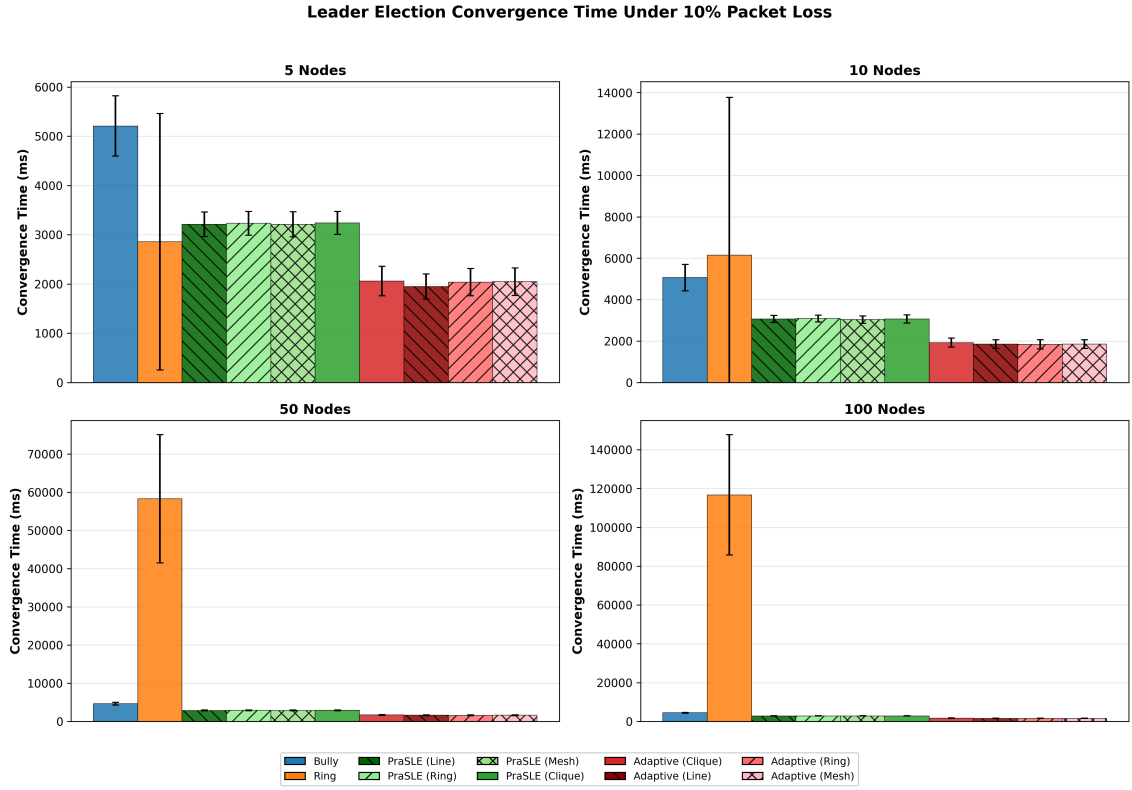


Figure 6.4: Convergence under 10% packet loss (90% success rate). Most algorithms handle mild interference with minimal degradation.

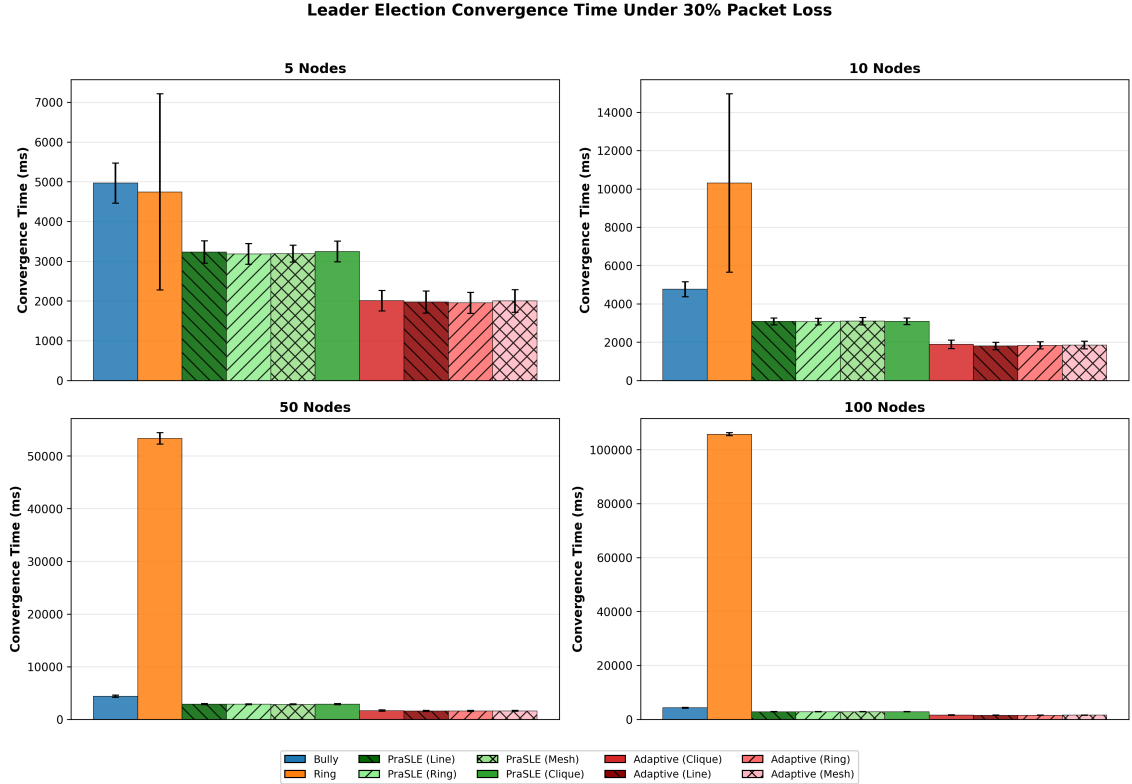


Figure 6.5: Convergence under 30% packet loss (70% success rate). Ring begins showing significant degradation at larger network sizes.

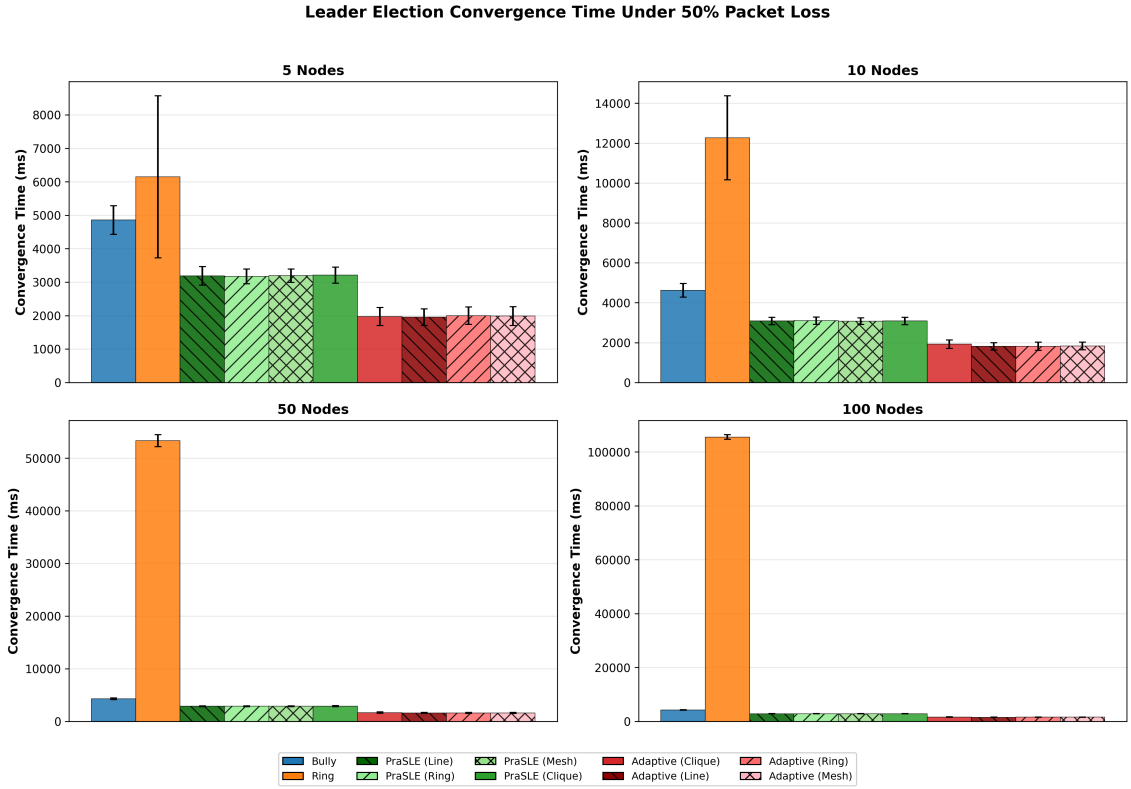


Figure 6.6: Convergence under 50% packet loss (50% success rate). High packet loss severely affects the Ring algorithm while PraSLE variants remain stable.

6.5.2 Moderate Interference (30% Loss)

At 30% packet loss, Ring's degradation becomes more pronounced. At 50 nodes, Ring convergence time jumps to 53 seconds (compared to 2.9 seconds for PraSLE variants, an $18\times$ difference). At 100 nodes, Ring exceeds 105 seconds, making it unusable for time-sensitive applications.

PraSLE variants handle 30% loss well. Convergence times remain under 3 seconds at 100 nodes (2.8 seconds), showing negligible degradation from ideal conditions. The broadcast communication pattern provides inherent redundancy: each round's broadcast gives every message a fresh chance to reach neighbors, and the self-stabilizing mechanism ensures eventual convergence regardless of individual packet losses.

Adaptive-PraSLE outperforms standard PraSLE under noise at all network sizes, converging in 1.6–2.0 seconds at 30% loss (2.0 seconds at 5 nodes, 1.6 seconds at 100 nodes). This represents a 38–43% improvement over standard PraSLE's 2.8–3.2 seconds. The conditional broadcasting mechanism reduces channel contention, which becomes increasingly valuable as packet loss rises.

6.5.3 Severe Interference (50% Loss)

At 50% packet loss, Ring convergence time exceeds practical limits at scale. At 50 nodes, Ring convergence exceeds 53 seconds; at 100 nodes it exceeds 105 seconds. Ring's

performance at 50% loss is nearly identical to its 30% loss performance, suggesting saturation of the failure mode. Once enough packets are lost to disrupt the sequential ring traversal, additional loss has diminishing marginal effect.

PraSLE variants are largely unaffected by severe packet loss. At 100 nodes, convergence times remain at 2.8 seconds, virtually identical to ideal conditions (2.8 seconds). The self-stabilizing design explains this: the algorithm continues broadcasting each round regardless of acknowledgments, and the periodic reset mechanism ensures eventual convergence even when most packets are lost.

Adaptive-PraSLE maintains its advantage over standard PraSLE under severe loss, converging in 1.6 seconds at 100 nodes (44% faster than standard PraSLE's 2.8 seconds). The performance gap between Adaptive-PraSLE and standard PraSLE remains consistent across all noise levels, which suggests that the advantage comes from the conditional broadcasting mechanism rather than noise-specific adaptations.

6.6 Network Partition Results

Network partitions occur when connectivity fails between groups of nodes, creating isolated sub-networks. Correct behavior requires each partition to independently elect its own leader. Figure 6.7 shows partition convergence times.

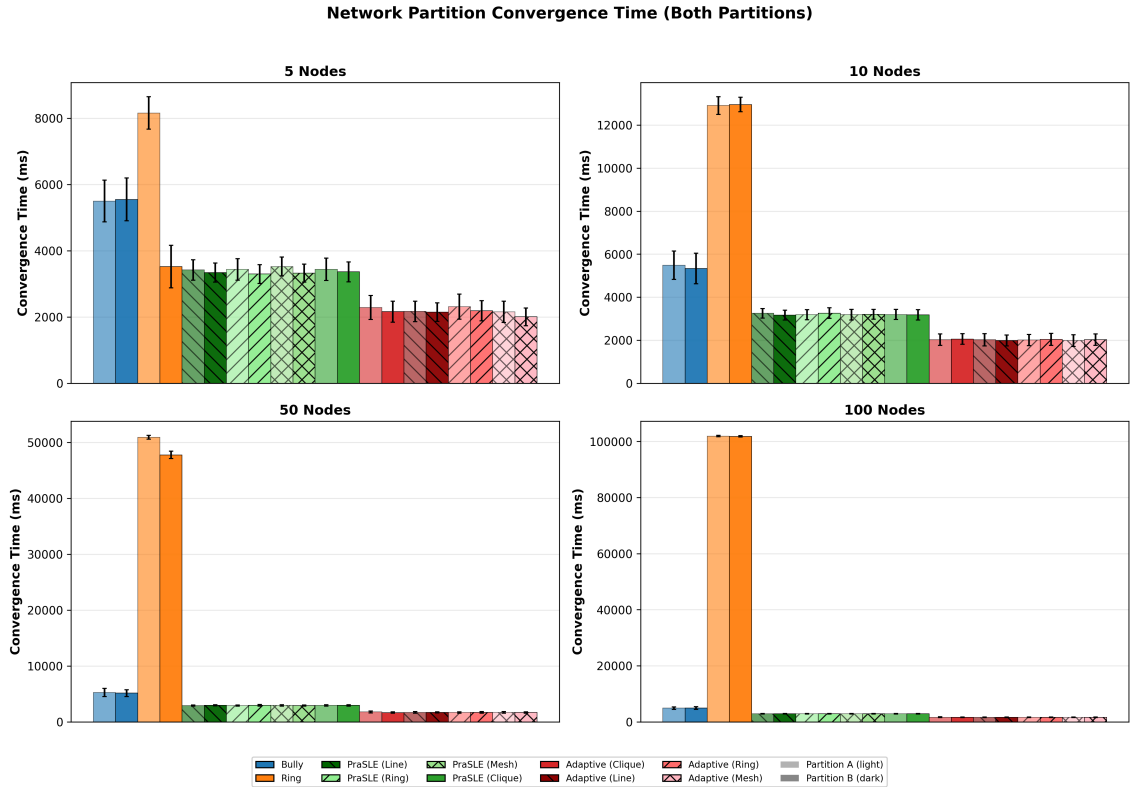


Figure 6.7: Network partition convergence (both partitions). Each partition should independently elect a leader. All algorithms correctly handle partitioning, with PraSLE variants showing the most consistent performance.

6.6.1 Partition Handling

All evaluated algorithms correctly handle network partitions. Each isolated group elects its own leader without interference. This is expected behavior for algorithms designed with partition tolerance in mind.

The convergence times in partitioned networks follow similar patterns to non-partitioned cases, scaled for the smaller effective network size in each partition. A 10-node network split into two 5-node partitions converges at a similar rate to a 5-node non-partitioned network.

6.6.2 Split-Brain Avoidance

None of the algorithms exhibit split-brain behavior where multiple leaders claim authority over the same nodes. The partition experiments confirm that leader scope is correctly bounded to reachable nodes.

When partitions heal (not tested in these experiments), re-convergence to a single leader would require additional mechanisms. PraSLE's self-stabilization guarantees eventual convergence when connectivity is restored, though the healing process may take several election cycles.

6.7 Trade-off Analysis

The previous sections looked at individual metrics separately. In practice, system design requires balancing multiple objectives. Figure 6.8 visualizes the trade-off between message overhead and convergence time.

6.7.1 Pareto Frontier

The Pareto frontier (the set of points not dominated by any other) shows the fundamental trade-offs:

Bully has moderate convergence time (4.5–5.4 seconds) with message overhead between Ring and PraSLE variants (164–13,400 messages depending on network size).

Ring at 100 nodes has both the highest convergence time (7.1 seconds) and 16,400 messages, dominated by all PraSLE variants in both metrics.

PraSLE (all topologies) converges in 2.8 seconds at 100 nodes but generates 27,200–38,700 messages due to unconditional broadcasting.

Adaptive-PraSLE (Clique) has the lowest message count at large scale (2,100 at 100 nodes) with 1.6 second convergence. **Adaptive-PraSLE (Line, Ring, Mesh)** converge in 1.5 seconds at 100 nodes with 9,300–10,000 messages, still below Bully's 13,400 messages at the same scale.

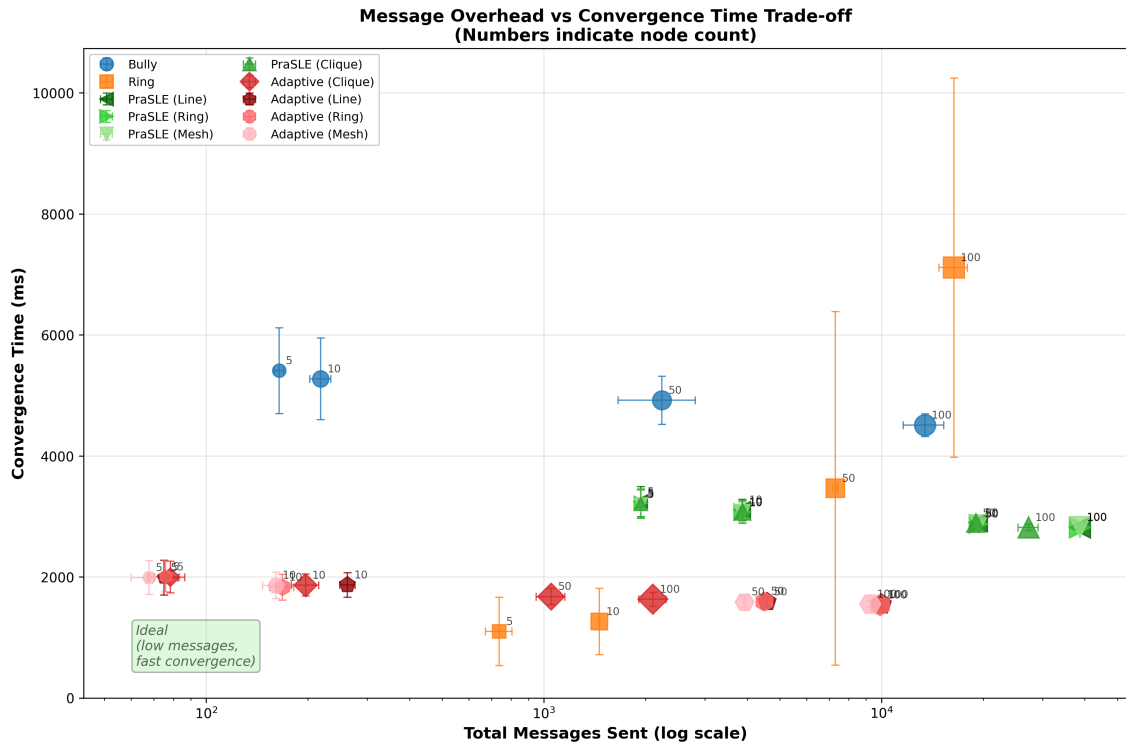


Figure 6.8: Message overhead vs. convergence time trade-off. Each point represents one algorithm at one network size (labeled). The ideal region is bottom-left (fast convergence, few messages). Adaptive-PraSLE variants occupy favorable positions.

6.7.2 Recommendations by Use Case

Based on the trade-off analysis:

For small networks (5–10 nodes): Any algorithm works reasonably well. Bully is simplest to implement. Adaptive-PraSLE with clique topology provides fastest convergence with the fewest messages if implementation complexity is acceptable.

For large networks (50+ nodes): Avoid Ring algorithm entirely. Adaptive-PraSLE with clique topology provides the best balance of speed and efficiency (2,100 messages, 1.6 seconds at 100 nodes). The other Adaptive-PraSLE topologies also outperform classical algorithms.

For lossy environments: PraSLE variants clearly outperform classical algorithms. Adaptive-PraSLE's conditional broadcasting provides 44% faster convergence than standard PraSLE with fewer messages.

For energy-constrained deployments: Adaptive-PraSLE with clique topology minimizes message overhead (2,100 messages at 100 nodes) while maintaining fast convergence. Among standard PraSLE variants, clique topology also has the lowest message count (27,200 at 100 nodes) due to its short cycle length.

6.8 Algorithm Scalability

Figure 6.9 shows how convergence time scales with network size for the four main algorithm families.

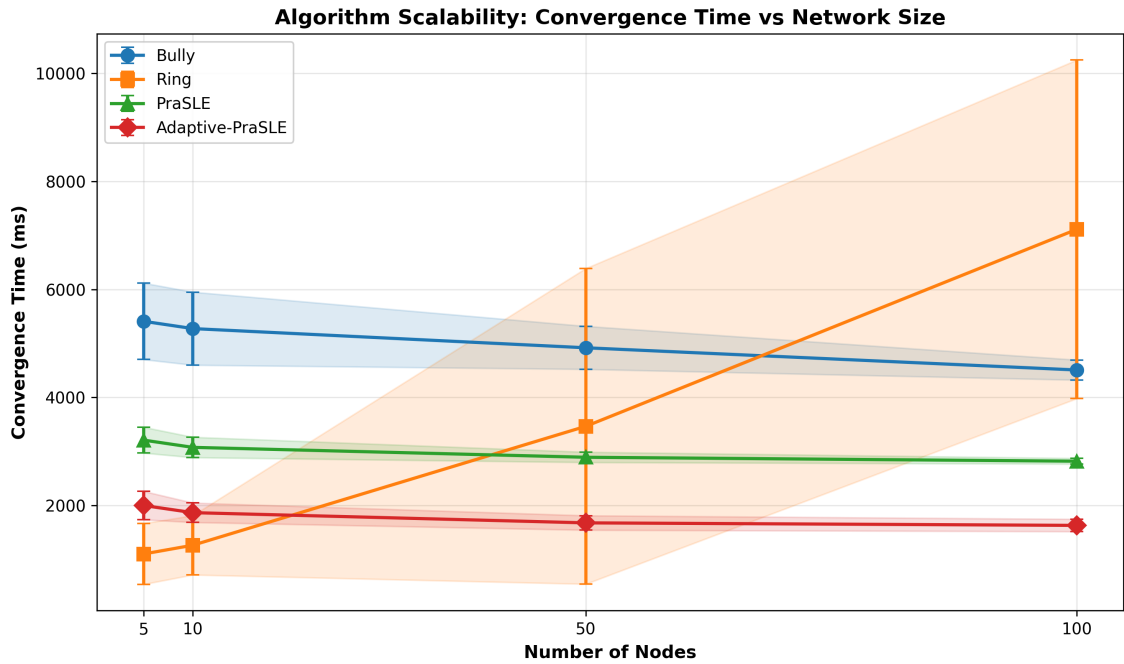


Figure 6.9: Scalability comparison showing convergence time vs. network size. Ring shows poor scalability with high variance at large sizes. PraSLE and Adaptive-PraSLE maintain consistent performance across scales.

6.8.1 Scalability Characteristics

Bully shows a slight decrease in convergence time as network size grows (5.4 seconds at 5 nodes to 4.5 seconds at 100 nodes). The timeout-dominated approach means network size has limited impact on convergence.

Ring exhibits linear or worse scaling. Convergence time grows proportionally with network size, and variance increases sharply at 50+ nodes. The shaded confidence region in Figure 6.9 highlights the unpredictability of Ring at scale.

PraSLE maintains nearly constant convergence time. The $O(K)$ round complexity bounds total time regardless of network size, assuming K is chosen appropriately for the topology.

Adaptive-PraSLE matches PraSLE's scalability while adding context-aware leader selection. The adaptive mechanisms add minimal overhead to the base algorithm.

7 Conclusion and Further Work

7.1 Summary

This thesis set out to investigate the impact of extending self-stabilizing leader election with energy awareness, link-quality awareness, and adaptive mechanisms on convergence behavior, leader stability, and network lifetime in resource-constrained IoT networks. The experimental evaluation in Chapter 6 shows that these extensions yield substantial improvements, with important trade-offs.

Adaptive-PraSLE extends the base PraSLE protocol with five mechanisms that address concrete limitations of identifier-based leader election. Energy-aware scoring prevents nearly-dead nodes from becoming leader. Link quality awareness favors nodes with reliable radio connections to their neighbors. Adaptive timeouts, based on Jacobson’s algorithm from TCP, replace fixed timing values with dynamic ones that track actual network conditions. Controlled rotation allows a leader to hand off to a successor before its battery runs out, rather than serving until failure. Backup list tracking enables fast recovery by promoting a known successor without running a full re-election.

The protocol was implemented in Contiki-NG (Chapter 5) and evaluated in the Cooja simulator across 10 algorithm variants, 4 network sizes (5 to 100 nodes), and multiple fault and noise conditions (Chapter 6). The evaluation compared Adaptive-PraSLE against classical Bully and Ring algorithms and the original PraSLE across four topologies (clique, line, ring, mesh).

7.2 Key Findings

The experimental evaluation produced four main results.

1. **Classical algorithms have limited applicability in IoT.** Ring’s sequential message propagation makes it fragile at scale: convergence time grows to 7.1 seconds at 100 nodes under ideal conditions and exceeds 117 seconds with just 10% packet loss, with 24% of trials failing to converge. At 50% packet loss, Ring’s convergence exceeds 105 seconds. Bully converges consistently (4.5–5.4 seconds) but its

identifier-based selection produced the same leader regardless of node energy levels across all trials, offering no mechanism to extend network lifetime.

2. **PraSLE and Adaptive-PraSLE scale well.** Standard PraSLE converges in under 3 seconds at all tested network sizes. Adaptive-PraSLE is 44% faster, converging in 1.6 seconds at 100 nodes due to conditional broadcasting that avoids redundant transmissions. Both tolerate packet loss well: standard PraSLE's convergence times change by less than 0.4% even at 50% packet loss, and Adaptive-PraSLE varies by up to 3%, because the self-stabilizing broadcast mechanism retries automatically each round.
3. **Topology choice primarily affects message overhead, not convergence time.** For standard PraSLE, all topologies converge at the same speed (2.8 seconds at 100 nodes), but clique generates fewer messages (27,200) than line, ring, and mesh (38,700 each) due to its shorter cycle length ($K=2$ rounds). For Adaptive-PraSLE, clique has the lowest message overhead (2,100 messages at 100 nodes) but converges slightly slower (1.6 seconds) than line, ring, and mesh topologies (1.5 seconds), which generate 9,300–10,000 messages. Adaptive-PraSLE reduces message overhead by 74–92% compared to standard PraSLE across all topologies through conditional broadcasting: for instance, at 100 nodes with clique topology, Adaptive-PraSLE sends 2,100 messages while standard PraSLE requires 27,200 messages to converge.
4. **Fault recovery improves with network size for PraSLE variants.** At 5–10 nodes, both standard PraSLE and Adaptive-PraSLE recover in 3–6 seconds. At larger scales the gap narrows and both achieve sub-second recovery: at 100 nodes, standard PraSLE-clique recovers in 206 milliseconds while Adaptive-PraSLE-ring recovers in 250 milliseconds; at 50 nodes, PraSLE-ring achieves 34 milliseconds and Adaptive-PraSLE-ring 37 milliseconds. Adaptive-PraSLE's backup list adds greater value for controlled leader rotation than for sudden crashes, where the reset-cycle mechanism dominates recovery in both variants. Ring shows the opposite pattern: recovery degrades from 6.4 seconds at 5 nodes to 102 seconds at 100 nodes. Bully remains constant at 7 seconds.

7.3 Threats to Validity

Several factors limit the generalizability of these results.

Simulation vs. real hardware. All experiments were conducted in Cooja, which provides cycle-accurate emulation but cannot capture all real-world effects such as environmental interference, hardware variation, and battery degradation over time. Results should be validated on physical IoT deployments.

Uniform node capabilities. All experiments assume homogeneous nodes with identical hardware and radio characteristics. Heterogeneous networks with varying radio ranges, processing capabilities, or energy budgets may show different behavior, particularly for the composite scoring function.

Static topology. Nodes remain stationary throughout all experiments. Mobile networks with frequent topology changes would stress the algorithms differently. Adaptive-PraSLE's timeout mechanism may help in such scenarios, but this was not tested.

Parameter sensitivity. Results depend on specific timeout values, scoring function weights (energy 30%, link quality 30%, connectivity 20%, CPU 20%), and topology configurations. The current weights were chosen based on preliminary experiments that balanced energy preservation with communication reliability, but no systematic sensitivity analysis was conducted. Increasing the energy weight (e.g., to 50%) could improve battery lifetime at the cost of occasionally selecting leaders with poorer connectivity, while increasing the link quality weight could improve coordination reliability in noisy environments at the expense of faster energy depletion at well-connected nodes. A formal sensitivity analysis varying these weights across deployment scenarios would strengthen confidence in the default configuration.

Increased protocol complexity. Adaptive-PraSLE increases message size from 8 bytes (standard PraSLE) to 14 bytes and adds state for backup tracking, scoring history, and rotation logic. While message count decreases through conditional broadcasting, the per-message cost is higher. Whether this trade-off is favorable depends on the deployment scenario.

7.4 Practical Implications

For practitioners deploying leader election in IoT systems:

- Use PraSLE or Adaptive-PraSLE for networks larger than 10 nodes. Classical algorithms do not scale.
- For Adaptive-PraSLE, choose clique topology when message overhead is the priority. It generates only 2,100 messages at 100 nodes (4–5× fewer than other topologies) with a minor convergence trade-off (1.6 seconds vs. 1.5 seconds for line, ring, and mesh).
- For standard PraSLE, clique topology also generates the fewest messages at large scale (27,200 at 100 nodes vs. 38,700 for line, ring, and mesh) due to its shorter cycle length.
- Use mesh, ring, or line topology when clique connectivity is not available. All three show similar convergence and message overhead in standard PraSLE. In Adaptive-PraSLE, mesh generates slightly fewer messages (9,300) than line (10,000) and ring (9,900).
- Avoid the Ring algorithm for production deployments. Its performance degrades unpredictably under packet loss.
- Prefer Adaptive-PraSLE over standard PraSLE when nodes have heterogeneous energy levels or when leader stability matters. The 44% convergence improvement and 74–92% message reduction come at the cost of slightly larger messages and additional protocol state.
- Standard PraSLE may be preferable over Adaptive-PraSLE when nodes are homogeneous (identical hardware and energy budgets), the network is small (under 10 nodes), or implementation simplicity is a priority. In such cases, the additional complexity of composite scoring, adaptive timeouts, and backup tracking provides limited benefit, since all nodes are equally suitable as leaders and the overhead of the extensions (larger messages, additional state) outweighs the gains.

7.5 Further Work

Several directions remain open for future research.

Hardware testbed validation. The most immediate next step is deploying Adaptive-PraSLE on physical IoT motes and measuring performance under real radio conditions. Simulation results are positive but cannot substitute for hardware validation with actual interference patterns, battery discharge curves, and multi-hop wireless links.

Mobile network support. All experiments used static node placements. Many IoT and robotic applications involve mobile nodes where topology changes frequently. Extending the evaluation to mobile scenarios and potentially adapting the protocol to handle neighbor churn (nodes entering and leaving communication range) would make the protocol applicable to a wider range of scenarios.

Adaptive weight tuning. The current scoring function uses fixed weights (30/30/20/20 for energy, link quality, connectivity, and CPU). An online learning mechanism that adjusts these weights based on observed network conditions could improve leader selection quality. For example, in a network where energy is abundant but link quality varies, the protocol could automatically increase the link quality weight.

Security extensions. The current protocol assumes a benign environment. A malicious node could falsify its score to become leader or disrupt elections by broadcasting incorrect values. Adding message authentication and Byzantine fault tolerance would be necessary for deployments in adversarial environments. The self-stabilizing property provides some resilience (the protocol eventually recovers from any transient state), but persistent attackers require explicit defenses.

Heterogeneous networks. The evaluation assumed all nodes have identical hardware. Real deployments often mix devices with different capabilities: some nodes may have larger batteries, better radios, or more processing power. Evaluating how the composite scoring function behaves with heterogeneous hardware and whether the weight distribution needs adjustment for mixed networks is an open question.

Reset cycle parameter tuning. The current implementation uses a fixed reset interval of $R = 3$ election cycles. This value bounds worst-case recovery time but limits steady-state efficiency to one cycle between resets. Investigating adaptive reset intervals, for example increasing R when the network is stable and decreasing it when failures are detected, could improve the trade-off between message overhead and fault recovery time.

Integration with routing protocols. Leader election does not operate in isolation. In RPL-based IoT networks [22], the elected leader often serves as the DODAG root. Investigating how Adaptive-PraSLE interacts with RPL's topology construction and how leader changes affect routing stability could lead to a more integrated protocol stack for IoT deployments.

Bibliography

- [1] H. Garcia-Molina, "Elections in a distributed computing system", *IEEE Transactions on Computers*, vol. C-31, no. 1, pp. 48–59, Jan. 1982. doi: 10.1109/TC.1982.1675885. [Online]. Available: <http://vis.usal.es/rodrigo/documentos/papers/BullyAlgorithm.pdf>.
- [2] E. Chang and R. Roberts, "An improved algorithm for decentralized extrema-finding in circular configurations of processes", *Communications of the ACM*, vol. 22, no. 5, pp. 281–283, May 1979. doi: 10.1145/359104.359108. [Online]. Available: <https://dl.acm.org/doi/10.1145/359104.359108>.
- [3] L. Lamport, "Paxos made simple", *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, Dec. 2001.
- [4] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm", in *USENIX Annual Technical Conference (ATC)*, USENIX Association, 2014, pp. 305–319.
- [5] M. Conard and A. Ebneenasir, "A practical self-stabilizing leader election for networks of resource-constrained IoT devices", in *17th European Dependable Computing Conference (EDCC)*, IEEE, 2021, pp. 127–134. doi: 10.1109/EDCC53658.2021.00025.
- [6] E. W. Dijkstra, "Self-stabilizing systems in spite of distributed control", *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, Nov. 1974. doi: 10.1145/361179.361202.
- [7] Contiki-NG Contributors, *Contiki-NG: The OS for next generation IoT devices*, <https://github.com/contiki-ng/contiki-ng>, Accessed: 2025-03-05, 2024.
- [8] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th. Addison-Wesley, 2011, isbn: 978-0132143011.
- [9] N. A. Lynch, *Distributed Algorithms*. San Francisco, CA: Morgan Kaufmann, 1996, isbn: 978-1558603486.
- [10] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process", *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, Apr. 1985. doi: 10.1145/3149.214121.

- [11] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks", in *33rd Annual Hawaii International Conference on System Sciences (HICSS)*, IEEE, 2000, p. 10. doi: 10.1109/HICSS.2000.926982.
- [12] K. Albee, N. Ohi, B. Coltin, I. A. Nesnas, and R. Soussan, "Lunar leader: Persistent, optimal leader election for multi-agent exploration teams", in *23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, IFAAMAS, 2024, pp. 35–43.
- [13] S. Dolev, *Self-Stabilization*. Cambridge, MA: MIT Press, 2000, isbn: 978-0262041782.
- [14] C. Bormann, M. Ersue, and A. Keranen, "Terminology for constrained-node networks", RFC 7228, May 2014. doi: 10.17487/RFC7228. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7228>.
- [15] G. Anastasi, M. Conti, M. Di Francesco, and A. Passarella, "Energy conservation in wireless sensor networks: A survey", *Ad Hoc Networks*, vol. 7, no. 3, pp. 537–568, 2009. doi: 10.1016/j.adhoc.2008.06.003.
- [16] IEEE, *IEEE standard for low-rate wireless networks*, Institute of Electrical and Electronics Engineers, 2020. doi: 10.1109/IEEESTD.2020.9144691.
- [17] P. Baronti, P. Pillai, V. W. Chook, S. Chessa, A. Gotta, and Y. F. Hu, "Wireless sensor networks: A survey on the state of the art and the 802.15.4 and ZigBee standards", *Computer Communications*, vol. 30, no. 7, pp. 1655–1695, 2007. doi: 10.1016/j.comcom.2006.12.020.
- [18] Texas Instruments, *CC2538 powerful wireless microcontroller system-on-chip for 2.4-GHz IEEE 802.15.4, 6LoWPAN, and ZigBee applications*, SWRS096D, 2015. [Online]. Available: <https://www.ti.com/lit/ds/symlink/cc2538.pdf>.
- [19] A. Dunkels, F. Österlind, N. Tsiftes, and Z. He, "Software-based on-line energy estimation for sensor nodes", in *4th Workshop on Embedded Networked Sensors (EmNets)*, ACM, 2007, pp. 28–32. doi: 10.1145/1278972.1278979.
- [20] M. Zuniga and B. Krishnamachari, "Analyzing the transitional region in low power wireless links", in *1st IEEE International Conference on Sensor and Ad Hoc Communications and Networks (SECON)*, IEEE, 2004, pp. 517–526. doi: 10.1109/SAHCN.2004.1381954.
- [21] D. S. De Couto, D. Aguayo, J. Bicket, and R. Morris, "A high-throughput path metric for multi-hop wireless routing", *Wireless Networks*, vol. 11, no. 4, pp. 419–434, 2005. doi: 10.1007/s11276-005-1766-z.
- [22] T. Winter *et al.*, "RPL: IPv6 routing protocol for low-power and lossy networks", RFC 6550, Mar. 2012. doi: 10.17487/RFC6550. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6550>.
- [23] N. Baccour *et al.*, "Radio link quality estimation in wireless sensor networks: A survey", *ACM Transactions on Sensor Networks*, vol. 8, no. 4, pp. 1–33, 2012. doi: 10.1145/2240116.2240123.
- [24] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, O. Landsiedel, and N. Tsiftes, "The Contiki-NG open source operating system for next generation IoT devices", *SoftwareX*, vol. 18, p. 101 089, 2022. doi: 10.1016/j.softx.2022.101089.
- [25] A. Dunkels, B. Grönvall, and T. Voigt, "Contiki – a lightweight and flexible operating system for tiny networked sensors", in *29th Annual IEEE International Conference on Local Computer Networks*, IEEE, 2004, pp. 455–462. doi: 10.1109/LCN.2004.38.

- [26] G. Montenegro, N. Kushalnagar, J. Hui, and D. Culler, "Transmission of IPv6 packets over IEEE 802.15.4 networks", RFC 4944, Sep. 2007. doi: 10.17487/RFC4944. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4944>.
- [27] A. Dunkels, O. Schmidt, T. Voigt, and M. Ali, "Protothreads: Simplifying event-driven programming of memory-constrained embedded systems", *ACM SenSys*, pp. 29–42, 2006. doi: 10.1145/1182807.1182811.
- [28] F. Österlind, A. Dunkels, J. Eriksson, N. Finne, and T. Voigt, "Cross-level sensor network simulation with COOJA", in *31st IEEE Conference on Local Computer Networks*, IEEE, 2006, pp. 641–648. doi: 10.1109/LCN.2006.322172.
- [29] R. G. Gallager, P. A. Humblet, and P. M. Spira, "A distributed algorithm for minimum-weight spanning trees", *ACM Transactions on Programming Languages and Systems*, vol. 5, no. 1, pp. 66–77, Jan. 1983. doi: 10.1145/357195.357200.
- [30] G. Le Lann, "Distributed systems – towards a formal approach", in *IFIP Congress*, 1977, pp. 155–160.
- [31] D. Hirschberg and J. Sinclair, "Decentralized extrema-finding in circular configurations of processors", *Communications of the ACM*, vol. 23, no. 11, pp. 627–628, 1980. doi: 10.1145/359024.359029.
- [32] IEEE, *IEEE standard for token ring access method and physical layer specifications*, Institute of Electrical and Electronics Engineers, 1998. doi: 10.1109/IEEESTD.1998.88568.
- [33] M. U. Rahman, "Leader election in the internet of things: Challenges and opportunities", *arXiv preprint arXiv:1911.00759*, 2020. doi: 10.48550/arXiv.1911.00759.
- [34] S. Kutten, G. Pandurangan, D. Peleg, P. Robinson, and A. Treharn, "On the complexity of universal leader election", *Journal of the ACM*, vol. 62, no. 1, 7:1–7:27, 2015. doi: 10.1145/2699440.
- [35] S. Kutten, W. K. Moses Jr., G. Pandurangan, and D. Peleg, "Singularly near optimal leader election in asynchronous networks", in *Proceedings of the 35th International Symposium on Distributed Computing (DISC)*, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 27:1–27:18. doi: 10.4230/LIPIcs.DISC.2021.27.
- [36] L. Lamport, "The part-time parliament", *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, May 1998. doi: 10.1145/279227.279229.
- [37] M. Castro and B. Liskov, "Practical byzantine fault tolerance", pp. 173–186, 1999.
- [38] A. Arora and M. Gouda, "Closure and convergence: A foundation of fault-tolerant computing", *IEEE Transactions on Software Engineering*, vol. 19, no. 11, pp. 1015–1027, 1993. doi: 10.1109/32.256850.
- [39] D. Angluin, J. Aspnes, Z. Diamadi, M. J. Fischer, and R. Peralta, "Computation in networks of passively mobile finite-state sensors", 4, vol. 18, 2006, pp. 235–253. doi: 10.1007/s00446-005-0138-3.
- [40] D. Alistarh and R. Gelashvili, "Recent algorithmic advances in population protocols", *ACM SIGACT News*, vol. 49, no. 3, pp. 63–73, Sep. 2018. doi: 10.1145/3289137.3289150.

- [41] J. Beauquier, P. Blanchard, and J. Burman, "Self-stabilizing leader election in population protocols over arbitrary communication graphs", in *17th International Conference on Principles of Distributed Systems (OPODIS)*, Springer, 2013, pp. 38–52. doi: 10.1007/978-3-319-03850-6_4.
- [42] K. Altisen, A. Cournier, S. Devismes, A. Durand, and F. Petit, "Self-stabilizing leader election in polynomial steps", *Information and Computation*, vol. 254, pp. 330–366, 2016. doi: 10.1016/j.ic.2016.10.002.
- [43] K. Altisen, A. Cournier, G. Defalque, and S. Devismes, "Self-stabilizing leader election in directed networks", in *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing (PODC)*, ACM, 2024, pp. 312–323. doi: 10.1145/3662158.3662778.
- [44] K. Altisen, S. Devismes, A. Durand, C. Johnen, and F. Petit, "Stabilizing leader election with weak assumptions on network dynamics", in *Proceedings of the 23rd International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, Springer, 2021.
- [45] Y. Sudo, F. Ooshita, H. Kakugawa, T. Masuzawa, A. K. Datta, and L. L. Larmore, "Time-optimal self-stabilizing leader election in population protocols", in *Proceedings of the 40th ACM Symposium on Principles of Distributed Computing (PODC)*, ACM, 2021, pp. 33–42. doi: 10.1145/3465084.3467898.
- [46] Y. Mo, J. Beal, and S. Dasgupta, "A self-stabilizing algorithm for leader election in networks of arbitrary topology", *Automatica*, vol. 146, p. 110 556, 2022. doi: 10.1016/j.automatica.2022.110556.
- [47] O. Younis and S. Fahmy, "HEED: A hybrid, energy-efficient, distributed clustering approach for ad hoc sensor networks", *IEEE Transactions on Mobile Computing*, vol. 3, no. 4, pp. 366–379, Oct. 2004. doi: 10.1109/TMC.2004.41.
- [48] M. A. Haddar, "SEALEA: Scalable and energy aware k-leaders election algorithm in IoT wireless sensor networks", *Wireless Personal Communications*, vol. 125, pp. 209–229, 2022. doi: 10.1007/s11277-022-09547-8.
- [49] S. Lindsey and C. S. Raghavendra, "PEGASIS: Power-efficient gathering in sensor information systems", in *IEEE Aerospace Conference*, vol. 3, IEEE, 2002, pp. 1125–1130. doi: 10.1109/AERO.2002.1035242.
- [50] A. Manjeshwar and D. P. Agrawal, "TEEN: A routing protocol for enhanced efficiency in wireless sensor networks", in *15th International Parallel and Distributed Processing Symposium (IPDPS)*, IEEE, 2001, pp. 2009–2015. doi: 10.1109/IPDPS.2001.925197.
- [51] A. Manjeshwar and D. P. Agrawal, "APTEEN: A hybrid protocol for efficient routing and comprehensive information retrieval in wireless sensor networks", in *2nd International Workshop on Parallel and Distributed Computing Issues in Wireless Networks and Mobile Computing*, IEEE, 2002, pp. 195–202. doi: 10.1109/IPDPS.2002.1016600.
- [52] A. Bounceur, M. Bezoui, R. Euler, N. Kadjouh, and F. Lalem, "BROGO: A new low energy consumption algorithm for leader election in WSNs", in *Proceedings of the 10th International Conference on Developments in eSystems Engineering (DeSE)*, IEEE, 2017, pp. 218–223. doi: 10.1109/DeSE.2017.18.

- [53] A. Bounceur, M. Bezoui, and R. Euler, "LOGO: A new distributed leader election algorithm in WSNs with low energy consumption", in *Proceedings of the EAI International Conference on Future Internet Technologies and Trends (ICFITT)*, Springer, 2018, pp. 1–14. doi: 10.1007/978-3-319-73712-6_1.
- [54] A. Bounceur, M. Bezoui, R. Euler, F. Lalem, N. Kadjouh, and M. Hammoudeh, "A wait-before-starting algorithm for fast fault-tolerant and low energy leader election in WSNs dedicated to smart-cities and IoT", in *Proceedings of the IEEE SENSORS Conference*, IEEE, 2017. doi: 10.1109/ICSENS.2017.8234201.
- [55] A. Bounceur, M. Bezoui, L. Lagadec, R. Euler, *et al.*, "DoTRo: A new dominating tree routing algorithm for efficient and fault-tolerant leader election in WSNs and IoT networks", *Mobile Networks and Applications*, vol. 25, pp. 1892–1903, 2020. doi: 10.1007/s11036-020-01599-z.
- [56] P. Thubert, "Objective function zero for the routing protocol for low-power and lossy networks (RPL)", RFC 6552, Mar. 2012. doi: 10.17487/RFC6552. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6552>.
- [57] O. Gnawali and P. Levis, "The minimum rank with hysteresis objective function", RFC 6719, Sep. 2012. doi: 10.17487/RFC6719. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6719>.
- [58] V. Jacobson, "Congestion avoidance and control", *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, Aug. 1988. doi: 10.1145/52325.52356.
- [59] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems", *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, Mar. 1996. doi: 10.1145/226643.226647.
- [60] T. D. Chandra, V. Hadzilacos, and S. Toueg, "The weakest failure detector for solving consensus", *Journal of the ACM*, vol. 43, no. 4, pp. 685–722, Jul. 1996. doi: 10.1145/234533.234549.
- [61] D. Canepa and M. G. Potop-Butucaru, "Stabilizing flocking via leader election in robot networks", INRIA, Tech. Rep. RR-6268, 2013.
- [62] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model", *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 25–34, 1987. doi: 10.1145/37402.37406.
- [63] A. Favier, N. Guittonneau, *et al.*, "CEL: Topology-aware leader election for dynamic networks", in *Proceedings of the 21st IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM) / Pacific Rim International Symposium on Dependable Computing (PRDC)*, IEEE, 2020. doi: 10.1109/PRDC49599.2020.00020.
- [64] A. Favier *et al.*, "Centrality-based eventual leader election in dynamic networks", in *Proceedings of the 20th IEEE International Symposium on Network Computing and Applications (NCA)*, IEEE, 2021. doi: 10.1109/NCA53618.2021.9685390.
- [65] K. M. Khalid *et al.*, "Multi-leader election in dynamic sensor networks", *EURASIP Journal on Wireless Communications and Networking*, vol. 2017, p. 187, 2017. doi: 10.1186/s13638-017-0961-9.

Appendix

A Simulation Environment

All experiments use the Cooja network simulator with Contiki-NG. This section documents the simulation parameters that are not covered in the main text.

A.1 Radio Model

Cooja's Unit Disk Graph Medium (UDGM) models radio propagation as a circular disk around each node. Two parameters control connectivity:

- **Transmission range:** The maximum distance at which a node can successfully deliver a packet. All nodes within this range receive the transmission.
- **Interference range:** Set equal to the transmission range in all experiments. Nodes within this range experience interference from ongoing transmissions.

For convergence, fault tolerance, and partition experiments, both success ratios are set to 1.0 (no packet loss). For noise experiments, both TX and RX success ratios are set to the target level (0.9, 0.7, or 0.5).

A.2 Topology Enforcement via Radio Range

Logical topologies are enforced by positioning nodes at fixed coordinates and setting the UDGM transmission range so that only intended neighbors are within reach. Table A.1 shows the transmission ranges used for each topology.

The clique range covers the full grid diagonal so all nodes can reach each other. The line range ($1.3 \times$ spacing) reaches immediate neighbors but not nodes two hops away. The mesh range ($1.2 \times$ spacing) reaches orthogonal neighbors at 50 m but not diagonal neighbors at ≈ 70.7 m. The ring range ($1.3 \times$ arc length) reaches adjacent ring positions only.

Table A.1: UDGM transmission range by topology (10-node configuration)

Topology	Node Placement	Spacing (m)	TX Range (m)
Clique	Grid ($\lceil\sqrt{N}\rceil$ columns)	50	198.3
Line	Horizontal line at $y = 100$	40	52.0
Ring	Circle of radius $50 + 3N$	arc-dependent	65.3
Mesh	Grid ($\lceil\sqrt{N}\rceil$ columns)	50	60.0

A.3 Mote Configuration

All nodes are emulated as Contiki motes in Cooja, providing instruction-level emulation of the target hardware. The mote startup delay is 1 second (staggered), and each mote runs the same firmware binary compiled with the appropriate algorithm and topology flags.

A.4 Simulation Control

Simulations run in headless mode (no GUI) using Cooja’s ScriptRunner plugin. A JavaScript control script forwards all mote log output to standard output and terminates the simulation after a configurable timeout (set via the C00JA_TIMEOUT environment variable, default 300 seconds). The experiment runner scripts override this timeout per configuration.

A.5 Source Code

The complete source code for all algorithm implementations, simulation configurations, and experiment automation scripts is available at: <https://github.com/Pushpit07/PraSLE-Simulation-in-Contiki-NG-Cooja>

B Complete Configuration Reference

Table B.2 lists all compile-time constants in the Adaptive-PraSLE configuration header. These supplement the key parameters shown in Chapter 5, Table 5.3.

C Experiment Configuration

C.1 Simulation Duration

Each experiment type uses topology-dependent simulation durations to account for slower message propagation in line and ring topologies. Table C.3 lists the durations.

Table B.2: Complete Adaptive-PraSLE configuration constants

Constant	Default	Description
<i>Feature Toggles</i>		
ADAPTIVE_ENERGY_AWARE	1	Enable energy-aware scoring
ADAPTIVE_LINK_QUALITY_AWARE	1	Enable link-quality scoring
ADAPTIVE_LEADER_ROTATION	1	Enable controlled handover
ADAPTIVE_TIMEOUTS	1	Enable adaptive timeout tuning
ADAPTIVE_BACKUP_LIST	1	Enable backup leader list
ADAPTIVE_RESET_CYCLES	1	Enable periodic reset cycles
<i>Scoring Weights (% , must sum to 100)</i>		
ENERGY_WEIGHT	30	Energy component weight
LINK_QUALITY_WEIGHT	30	Link quality component weight
CONNECTIVITY_WEIGHT	20	Neighbor count component weight
MIN_NEIGHBORS_FOR_LEADER	2	Min neighbors to be a leader candidate
CPU_WEIGHT	20	CPU availability component weight
CPU_HIGH_THRESHOLD	80%	CPU usage considered “busy”
<i>Energy Parameters</i>		
ENERGY_CRITICAL_THRESHOLD	20%	Battery level triggering forced handover
ENERGY_LOW_THRESHOLD	40%	Battery level to avoid leader selection
ENERGY_UPDATE_INTERVAL	5 s	Period between energy estimate updates
INITIAL_BATTERY_LEVEL	100	Simulated initial battery percentage
BATTERY_DRAIN_RATE	1	Drain per second of radio activity (%)
<i>Link Quality Parameters</i>		
RSSI_GOOD_THRESHOLD	−60 dBm	Links above this are considered good
RSSI_POOR_THRESHOLD	−85 dBm	Links below this are considered poor
ETX_MAX_ACCEPTABLE	4	Max ETX before neighbor deemed unreliable
LQI_MIN_ACCEPTABLE	50	Min LQI value (0–255 scale)
FRESHNESS_MIN_THRESHOLD	4	Min freshness counter to trust link (0–16)
FRESHNESS_CONFIDENCE_SCALE	1	Enable freshness as confidence multiplier
RSSI_UNKNOWN	−127	Sentinel value for unknown RSSI
<i>Adaptive Timeout (Jacobson RTT)</i>		
TIMEOUT_MIN_SECONDS	0.15	Floor for adaptive timeout
TIMEOUT_MAX_SECONDS	5.0	Ceiling for adaptive timeout
TIMEOUT_ALPHA	0.125	EWMA smoothing factor ($\alpha = 1/8$)
TIMEOUT_SAFETY_MARGIN	1.2	Multiplier on computed RTO
INITIAL_RTT_ESTIMATE_MS	100	Initial smoothed RTT guess
INITIAL_RTT_VARIANCE_MS	25	Initial RTT variance guess
<i>Leader Rotation</i>		
MIN_LEADERSHIP_TERM	30 s	Minimum time before rotation allowed
HANDOVER_ACK_TIMEOUT	3 s	Time to wait for handover acknowledgment
MAX_HANDOVER_RETRIES	3	Retries before fallback to next backup
<i>Backup List</i>		
BACKUP_LIST_SIZE	3	Number of backup leaders maintained
BACKUP_HEARTBEAT_INTERVAL	3 s	Period between leader heartbeats
BACKUP_FAILURE_THRESHOLD	3	Missed heartbeats before declaring failure
<i>Reset Cycle</i>		
RESET_CYCLE_COUNT	3	Reset every N election cycles

Table C.3: Simulation duration by experiment type, topology, and network size (seconds). Each topology row applies to all algorithm variants using that topology (PraSLE, Adaptive-PraSLE). Line and ring topologies require longer durations at large network sizes due to slower message propagation. The classical Ring algorithm uses the same durations as line topologies.

Experiment / Topology	5 nodes	10 nodes	50 nodes	100 nodes
Convergence (clique, mesh)	60	60	90	180
Convergence (ring)	60	60	100	180
Convergence (line, Ring algorithm)	60	60	120	240
Fault tolerance (clique, mesh)	60	120	150	240
Fault tolerance (ring)	120	120	160	240
Fault tolerance (line, Ring algorithm)	120	120	180	300
Noise (clique, mesh)	60	60	90	180
Noise (ring)	60	60	100	180
Noise (line, Ring algorithm)	60	60	120	240
Network partition (clique, mesh)	60	60	90	180
Network partition (ring)	60	60	100	180
Network partition (line, Ring algorithm)	60	60	120	240

C.2 Trial Configuration

Each experiment configuration runs 100 independent trials. Each trial uses a unique random seed generated from a combination of the system clock and trial number, ensuring statistically independent results. The seed is injected into the Cooja simulation file before each run, overriding the template's default seed.

C.3 Convergence Detection

Convergence is detected by parsing mote log output. Each node prints a log message when it first determines its leader. The experiment script monitors for the ALL_CONVERGED marker, which is emitted when every node in the simulation has reported agreement on the same leader. The convergence time is the simulated time (in milliseconds) at which this marker appears.

C.4 Fault Injection

In fault tolerance experiments, the leader node is crashed at a fixed time after simulation start. The crash time is calculated as the simulation duration divided by two (e.g., at 60 seconds for a 120-second simulation). The crash is implemented by removing the leader mote from the Cooja simulation via the scripting API. Recovery time is measured from the crash event until all surviving nodes agree on a new leader.

C.5 Network Partition Layout

Partition experiments split the network into two equal groups by placing them in separate physical regions. Group A nodes are positioned in a grid starting at coordinates (20, 20), while Group B nodes are placed in a grid starting at (20, 600). The 580-meter vertical separation exceeds the maximum UDGM transmission range, ensuring complete isolation between groups. Each group independently elects its own leader.

C.6 Noise Experiment Configuration

Three noise levels are tested by setting the UDGM success ratios as shown in Table C.4.

Table C.4: Noise experiment success ratio configuration

Label	TX Success Ratio	RX Success Ratio	Nominal Packet Loss
noise90	0.9	0.9	$\approx 10\%$
noise70	0.7	0.7	$\approx 30\%$
noise50	0.5	0.5	$\approx 50\%$

Both TX and RX success ratios are set to the same value. The effective single-hop packet loss rate equals $1 - (TX \times RX)$, so noise90 produces approximately 19% actual loss, noise70 approximately 51%, and noise50 approximately 75%. However, following convention in the thesis, we refer to these by their nominal loss rates (10%, 30%, 50%). Raw experimental data (CSV files) is available in the source repository linked in Section A.